

Development of an Embedded HPEC Telemetry Unit and Dashboard Integration for the TUKZIE Rev 0 Platform



Presented by:
Samson Okuthe
Student number: OKTSAM001

Prepared for:
Simon Winberg
Co-supervisor: Sampath Jayalath
Department of Electrical Engineering
University of Cape Town

Submitted to the Department of Electrical Engineering at the University of Cape Town in partial fulfilment of the academic requirements for a Bachelor of Science degree in Electrical and Computer Engineering.

October 7, 2026

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced.
3. This report is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as their own work or part thereof.

Signature:



Samson Okuthe (OKTSAM001)

Date: October 7, 2026

Word count of the body of the report (Chapters 1 to 7, excluding tables, figures, captions and code listings): 20 449.

Terms of Reference

This project (SW-7) is to develop a high-performance embedded computing (HPEC) telemetry unit for the TUKZIE Rev 0 electric cargo trike, integrated with the vehicle's existing dashboard, capable of acquiring synchronised inertial, battery, positioning and camera data, characterising the vehicle's dynamic response at the edge, preserving telemetry under intermittent cellular connectivity, and presenting a minimal set of safety-relevant information to the driver on that dashboard.

Acknowledgments

I thank my supervisor, A/Prof. Simon Winberg, and co-supervisor, Sampath Jayalath, for their guidance, and the TUKZIE Flagship Programme and its vacation-work teams, whose earlier work on the vehicle's dashboard, sensing and drivetrain this project builds on.

Abstract

Draft abstract, to be revised once field results are available.

This report describes the development of an embedded telemetry unit for the TUKZIE Rev 0, a 72 V electric cargo trike, and its integration with the vehicle's existing dashboard. An ESP32-S3 with an integrated LTE modem acquires data from two inertial measurement units at 200 Hz, the vehicle's battery-management system over Bluetooth Low Energy, and GNSS, computes time-domain ride-characterisation features at the edge, logs telemetry locally, and publishes it over MQTT. A Raspberry Pi 4, time-synchronised to the inertial stream by a pulse wire, runs a pretrained object detector that reports road users in coarse distance bands, with three time-of-flight sensors for close range, and a page written for the existing dashboard shows its live view and alerts in place of the windshield head-up display in the original brief. Bench validation to date shows deterministic 200 Hz acquisition with no dropped samples, per-sensor calibration reducing an 8% disagreement between nominally identical accelerometers to within measurement noise, GNSS positions a median of 2.7 m from a reference, automatic recovery of the cellular link within about 5 s of the signal returning, a complete read-back of the local log, and a median of 102 ms from camera readout to detection at 19.5 frames per second. On-vehicle field testing, which the ride-characterisation research questions and the glance-time question depend on, is the principal outstanding work.

Contents

1	Introduction	1
1.1	Background and motivation	1
1.2	Objectives of this study	2
1.2.1	Problems to be investigated	2
1.2.2	Requirements and verification mapping	3
1.2.3	Purpose of the study	3
1.3	Scope and Limitations	4
1.3.1	Scope	4
1.3.2	Limitations	5
1.4	Plan of development	6
2	Literature Review	7
2.1	Review scope and evaluation framework	7
2.2	Light electric freight vehicles and last-mile logistics	7
2.3	Pavement roughness and vehicle ride smoothness	8
2.4	Inertial signal processing and vibration separation	9
2.5	Edge computing and real-time telemetry architecture	10
2.6	Driver-display design, attention, and visual workload	10

2.7	Cellular synchronisation and data integrity	11
2.8	Proximity sensing on low-speed vehicles	12
2.9	Synthesis and research gap	12
3	Methodology	14
3.1	Development and verification approach	14
3.2	Embedded hardware baseline	14
3.3	Microcontroller startup verification	15
3.4	A7670X UART integration	16
3.5	Modem power-up sequencing	16
3.6	AT-command verification procedure	16
3.7	GNSS polling verification	17
3.8	Baseline verification criteria	17
3.9	Single-sensor inertial integration (firmware v0.3.0)	18
3.10	Flash and PSRAM configuration verification	18
3.11	Dual-inertial deterministic acquisition architecture	20
3.11.1	Design requirements	20
3.11.2	Task and queue structure	20
3.11.3	Sensor configuration	21
3.12	Inter-board time synchronisation	22
3.12.1	Pin assignment error found before wiring	22
3.12.2	End-to-end validation	22
3.12.3	A physical connection fault found during a later session	23
3.12.4	Combined video and sync capture	23

3.12.5	Lens-cover calibration test	24
3.13	GNSS response parsing and field-mapping validation	24
3.13.1	Initial field-order assumption and its failure	24
3.13.2	Revised mapping and its remaining limitation	25
3.13.3	Outdoor fix and confirmed field layout	25
3.13.4	Modem warm-up behaviour	26
3.14	Acquisition-integrity instrumentation	27
3.15	Ride-characterisation signal processing	27
3.15.1	Validation against a synthetic signal	28
3.15.2	Per-sensor magnitude calibration	28
3.15.3	Speed-normalised roughness index	29
3.15.4	Dual-IMU fusion	30
3.16	Battery-management system interface over BLE	31
3.16.1	Hardened interface design	33
3.16.2	A scanning fault found during testing	33
3.16.3	First live contact with a powered pack	33
3.16.4	A second assumption invalidated by live testing	34
3.16.5	Status at time of writing	34
3.16.6	Checksum fault found on the vehicle	34
3.17	Environmental sensing	35
3.18	Motor speed tap	35
3.19	Full-system integration on the target hardware	36
3.19.1	A misdiagnosed fault: distinguishing a hardware failure from a software configuration mismatch	36

3.19.2	Result: simultaneous operation on the target hardware	37
3.20	Camera subsystem	38
3.20.1	Camera identification	38
3.20.2	A media-provisioning fault: incomplete partitioning	38
3.20.3	Network provisioning	38
3.20.4	A recurring connectivity fault on re-provisioned media	39
3.20.5	Verification	39
3.20.6	Camera casing and mounting	40
3.20.7	Hazard-detection scope extension	40
3.20.8	Camera and time-of-flight fusion	54
3.21	Cellular telemetry publishing	55
3.21.1	Local logging to internal flash	56
3.21.2	End-to-end validation against a real broker	57
3.21.3	Result-code checking and automatic reconnection (firmware v0.5.0)	58
4	Results	59
4.1	Inertial acquisition integrity (RQ3)	59
4.2	Sensor calibration	59
4.3	Ride characterisation (RQ1, RQ4)	59
4.4	Battery-management telemetry	60
4.5	Camera, synchronisation and proximity sensing	60
4.6	Cellular synchronisation and data integrity (RQ3)	60
4.7	Driver display (RQ2)	60
4.8	Full-system test	60

5	Discussion	61
6	Conclusions	62
7	Recommendations	63
7.1	Generic road-obstacle detection	63
A	Graduate Attributes, AI Use and Project Repository	71
A.1	Graduate attribute evidence	71
A.2	Project repository	72
A.3	Declaration of Artificial Intelligence Use	73
B	Supporting Figures	74
C	Test Evidence	76

Chapter 1

Introduction

1.1 Background and motivation

Rapid urbanisation is increasing demand for efficient passenger transport and urban goods delivery. The United Nations projects that approximately 68% of the world's population will live in urban areas by 2050 [2]. This growth, together with the need to reduce transport emissions and operating costs, has encouraged the development of lightweight electric-drive vehicles for last-mile logistics. Within this context, the TUKZIE Flagship Programme seeks to engineer an energy-aware, secure and autonomous-ready vehicle platform for last-mile logistics and urban freight applications. The TUKZIE Rev 0, a 72V electric cargo trike, represents a testbed for integrating advanced telemetry and driver-assistance systems in resource-constrained environments.

The relevance of this platform is particularly clear in sub-Saharan African cities, where three-wheelers already support urban goods movement. A 2025 study of medium-size cargo delivery routes in Dar es Salaam found that an electric tricycle reduced operating cost per kilometre by approximately 45.5% relative to a motorcycle, 62.5% relative to a petrol tuk-tuk, and 86% relative to a light-duty car [3]. The reported values are route- and price-specific rather than universal forecasts, but they show why locally appropriate electric freight platforms warrant further engineering and field validation.

Vehicle telemetry is often only logged and analysed afterwards, which delays maintenance and prevents real-time decisions on the vehicle. Real-time safety alerts matter because road traffic injuries remain a leading cause of death worldwide, with most of the deaths in low- and middle-income countries [1]. Presenting telemetry to the driver in real time has its own cost, however: any driver display, head-down or head-up, draws the driver's gaze or attention away from the road, and cluttered presentation increases attention capture [5, 8]. NHTSA guidance recommends that individual visual-manual glances away from the roadway should not exceed two seconds, with no more than twelve seconds of cumulative off-road glance time for a task [6]. This provides a measurable human-factors constraint on any driver display: what is shown must be sparse and legible enough to be read within a short glance, wherever the display is placed.

Recent advances in High-Performance Embedded Computing (HPEC) have enabled edge-based signal processing in vehicular applications, allowing for real-time sensor fusion and on-vehicle analytics [9]. Many light vehicles, the TUKZIE Rev 0 among them, already carry a touchscreen dashboard on which

such analytics can be presented; Head-Up Displays (HUDs) offer a forward-view alternative [10, 11], with optical and attention costs of their own (Chapter 2). However, integrating on-vehicle analytics with driver presentation for low-cost, three-wheeled electric vehicles in developing contexts remains an open challenge.

This study addresses this gap by developing an HPEC telemetry unit for the TUKZIE Rev 0, integrated with the vehicle's existing dashboard, enabling real-time ride-smoothness characterisation and safety alerts to the driver. The original brief specified a windshield HUD. During the project it was replaced, with the supervisor's agreement, by integration with the existing dashboard: the dashboard already shows the information a HUD would have carried, a second display would compete for the driver's attention [18], and the programme's move towards autonomy makes the camera's hazard alerts more useful than a second screen (Section 3.20.7).

The intended outcome is an integrated prototype demonstrated on the target or a representative vehicle under relevant operating conditions. This corresponds to a target of Technology Readiness Level 6, at which a representative prototype is demonstrated in a relevant environment [7]. It does not imply production qualification or approval for unrestricted road deployment.

1.2 Objectives of this study

1.2.1 Problems to be investigated

Transportation agencies and fleet operators face significant challenges in monitoring road health and vehicle performance cost-effectively. Reference-grade profiling equipment requires specialist instrumentation and operation, which limits the frequency and coverage of surveys in resource-constrained settings [13, 12]. Standard in-car displays often contribute to cognitive overload and attention capture, where drivers divert their gaze from the road to process telemetry [18]. Furthermore, in resource-constrained environments, there is a gap for a modular, low-latency system capable of edge-based signal processing for vehicle dynamic-response characterisation while providing a safe head-up driver interface.

The engineering problem is therefore to acquire synchronised data reliably on a high-vibration, power-constrained vehicle; distinguish road-induced motion from vehicle-specific dynamics; maintain bounded processing and display latency; and preserve telemetry during intermittent cellular connectivity. These functions must be integrated without obstructing the driver's view or making safety-critical behaviour dependent on a remote service [22].

Specifically, this study investigates the following research questions:

1. How accurately and repeatably can a low-cost edge-processing system characterise the trike's dynamic response to road- and powertrain-induced excitation in real time, using inertial sensors on a 72V electric cargo trike?
2. Which information hierarchy, layout, and update strategy can present safety-critical telemetry and hazard alerts on the vehicle's existing dashboard while limiting visual clutter and driver distraction?
3. What end-to-end latency, data-loss, and recovery performance can be achieved across sensor acquisition, edge processing, local logging, and 4G/LTE synchronisation?

4. Which time- and frequency-domain features most effectively separate road-induced vibration from repeatable motor and powertrain vibration?

1.2.2 Requirements and verification mapping

Table 1.1 maps each research question to a concrete system requirement and the method by which it is, or will be, verified. This mapping is used consistently through Chapter 3 (Methodology) and is intended to be revisited directly in Chapter 4, rather than treated as a fixed statement made only here.

Table 1.1: Research questions mapped to system requirements and verification method.

RQ	Requirement	Verification method	Status
RQ1	Dual-IMU acquisition at ≥ 160 Hz (ISO 2631-1 bound) with per-sensor magnitude agreement to within measurement noise	Deterministic 200 Hz acquisition (Section 3.11); per-sensor calibration against true gravity, cross-checked under a dynamic disturbance (Section 3.15.2)	Bench-verified
RQ2	The existing dashboard presents safety-critical alerts without exceeding NHTSA off-road glance-time guidance	Sparse alert page in the dashboard's own status colours, tested against the live camera (Section 3.20.7); on-vehicle glance-time observation	Page bench-verified; glance time outstanding
RQ3	Bounded end-to-end latency, quantified data loss, and recovery across acquisition, edge processing, local logging, and cellular sync	Queue drop-counters and jitter instrumentation (Section 3.14); local LittleFS logging decoupled from MQTT connection state; end-to-end MQTT publish against a real broker (Section 3.21)	Acquisition, camera latency, signal-loss recovery and local-log read-back bench-verified; on-vehicle testing outstanding
RQ4	Time/frequency-domain features that separate road-induced from powertrain-induced vibration	Closed-form self-test of feature computation against a synthetic signal (Section 3.15); real-vehicle feature comparison against known road/powertrain events	Self-test only; real-vehicle validation outstanding

1.2.3 Purpose of the study

The significance of investigating these problems lies in bridging the gap between expensive, proprietary road-profiling equipment and the need for cost-effective, real-time monitoring solutions. By leveraging High-Performance Embedded Computing and low-cost inertial sensors, this study aims to provide a modular, low-latency system that delivers head-up telemetry while limiting unnecessary downward glances and avoiding additional visual clutter.

The findings will directly inform the development of the TUKZIE Rev 0 platform and contribute to the broader field of sustainable urban mobility by demonstrating a replicable, open-architecture approach to

vehicle telemetry. This work aligns with the UCT Radar Remote Sensing Group’s focus on advanced sensing and the TUKZIE Flagship Programme’s mandate for energy-aware, secure vehicle platforms.

Furthermore, this research addresses the practical needs of fleet operators in developing regions, where infrastructure monitoring budgets are constrained and traditional data collection methods are often infeasible. By enabling continuous, vehicle-specific ride-condition screening, the proposed system could support route comparison, targeted inspection, and more informed maintenance planning.

1.3 Scope and Limitations

1.3.1 Scope

This study focuses exclusively on the TUKZIE Rev 0, a 72 V electric cargo trike manufactured by Jinpeng [21]. Direct inspection of the motor and controller nameplates recorded a three-phase, permanent-magnet, sensored hub motor rated at 72 V DC and 35 A, paired with a controller rated to a 90 A current limit, giving a calculated rated input power of approximately 2.5 kW [61]. The vehicle’s own manufacturer identification plate (Figure B.1, Appendix B), photographed directly on the platform, independently confirms the manufacturer and model (Jiangsu Jinpeng Group Co., Ltd, model L2e-U) and additionally records a rated output of 3.9 kW, a rated speed of 42 km/h, and a maximum load of 639 kg. The system integrates the following components:

- A hardware module housing inertial measurement units (IMUs) and a Sensirion SEN55 environmental sensor, which measures particulate matter, volatile organic compounds (VOC), nitrogen oxides (NOx), humidity, and temperature (Section 3.17).
- A forward-facing camera, time-synchronised to the inertial acquisition path, providing a visual record against which detected dynamic-response events can be labelled and interpreted (for example, distinguishing a pothole from an intersection or a deliberate manoeuvre), and running a pretrained object-detection model to report predefined road-relevant object categories and an approximate distance band (Section 3.20.7).
- A non-visual, non-machine-learning proximity sensor (an 8×8-zone time-of-flight distance sensor) providing live nearby-object distance and rough left/centre/right position to the driver, adopted as a second, physically independent measurement that operates regardless of the camera detector’s own status and involves no camera image or learned model.
- A multithreaded software architecture for real-time sensor fusion, telemetry processing, and dynamic-response indexing.
- A camera page for the vehicle’s existing dashboard, showing the live view and the active hazard alerts in the dashboard’s own status colours, in place of the optical windshield Head-Up Display of the original brief (Section 3.20.7).
- A 4G/LTE communication link for central database synchronisation.

The project is aligned with the UCT Radar Remote Sensing Group’s focus on advanced sensing and specifically addresses the “ride-smoothness” mandate of TUKZIE by isolating road-induced vibration

from nominal motor and powertrain vibration using statistical feature extraction. The scope is limited to the TUKZIE Rev 0 platform and utilises low-cost inertial and environmental sensors processed at the edge using microcontroller and single-board computer platforms [12].

The software architecture follows a modular design, with separate threads for:

- High-frequency sensor polling (both IMUs at 200 Hz, above the 160 Hz minimum in Table 1.1).
- Compute-heavy signal-processing algorithms (ride characterisation).
- The driver display (the dashboard page, on the dashboard's own computer).
- Networking stack (MQTT over 4G LTE Cat-1).

This separation ensures that no process blocks another [22].

1.3.2 Limitations

The following limitations are acknowledged:

- The study is limited to a single vehicle platform (TUKZIE Rev 0), which may affect generalisability to other vehicle types or configurations.
- Sensor calibration assumes nominal operating conditions; significant environmental variations (extreme temperatures and humidity) may affect accuracy.
- Edge-processing latency measurements are conducted in controlled test environments; real-world performance may vary with network conditions and electromagnetic interference.
- The driver display is the vehicle's existing head-down dashboard. A windshield head-up display, which would keep the driver's gaze nearer the road [10], is not built in this project.
- Time constraints limit extensive field testing to urban routes only; rural and off-road conditions are not evaluated.
- The system is designed as a removable module to ensure that the vehicle can be quickly returned to a standard setup without obstructions [22].
- The 4G/LTE communication depends on network availability; data synchronisation may be delayed in areas with poor coverage.
- The camera's object-detection capability uses a pretrained, general-purpose model without vehicle-specific training, reports only object category, approximate distance band, detection confidence, timestamp, and alert status, and does not retain raw camera frames or identify individual people; it does not track objects across frames or perform lane-level blind-spot geometry, and is not used for driver monitoring or any safety-critical vehicle function. The time-of-flight rangefinder described above provides a second, independent proximity measurement alongside it.

1.4 Plan of development

This report is organised as follows:

Chapter 2: Literature Review. Reviews telemetry systems for electric vehicles, edge-computing architectures for vehicular applications, inertial road-roughness assessment [12, 23], head-up display design [10, 11, 28], and proximity sensing on low-speed vehicles.

Chapter 3: Methodology. Describes the hardware platform, firmware architecture, sensor integration, calibration, camera and proximity subsystems, and cellular telemetry, together with the bench validation carried out at each stage, including faults found and how they were resolved.

Chapter 4: Results. Presents acquisition integrity, calibration, ride-characterisation, battery, camera, proximity and cellular results by subsystem, followed by a full-system test on the vehicle.

Chapter 5: Discussion. Interprets the results against the literature and the research questions.

Chapter 6: Conclusions. Answers each research question and states the contribution and its limitations.

Chapter 7: Recommendations. Identifies future work, including generic road-obstacle detection.

Chapter 2

Literature Review

2.1 Review scope and evaluation framework

The proposed system combines four fields that are often treated separately: pavement and ride-quality sensing, embedded signal processing, vehicular telemetry, and driver-display design, drawing mainly on Head-Up Display (HUD) research. The literature is therefore evaluated against the needs of the TUKZIE Rev 0 rather than only by reported classification accuracy. Relevant evidence must also address computational cost, sampling requirements, sensitivity to vehicle speed and mounting position, real-time latency, communication loss, and driver workload. This framework is important because a technique that performs well on a passenger car, smartphone, or laboratory simulator may not transfer directly to a lightweight three-wheeled cargo platform.

2.2 Light electric freight vehicles and last-mile logistics

Light electric freight vehicles can reduce the energy, space, and emissions associated with urban delivery, but their advantage depends on route length, payload, consolidation strategy, electricity supply, and the vehicle used for comparison. In Dar es Salaam, Sebwa et al. compared an electric cargo tricycle with motorcycles, petrol tuk-tuks, and a light-duty car on selected medium-size delivery routes [3]. The electric tricycle had the lowest reported operating cost, with savings ranging from 45.5% against motorcycles to 86% against the light-duty car. The study also reported energy use of up to 0.54 kWh for a one-way trip and a 97.59% energy saving relative to the light-duty-car comparison [3]. It avoided the direct tailpipe emissions produced by the petrol vehicles. These results provide valuable regional evidence, although the underlying fuel prices, route choices, assumed payload, trip length, comparison basis, and omission of electricity-generation emissions limit direct transfer to Cape Town.

Evidence from Portland reaches a similar conclusion using a different system boundary. Saenz et al. combined operational GPS and warehouse data with vehicle, battery, fuel, and electricity life-cycle contributions, finding greenhouse-gas reductions of 51–72% for an electric-tricycle logistics service relative to conventional urban logistics [4]. The range is more informative than a single headline value: it shows that benefits change when partner operations and supporting logistics are included. For TUKZIE, the literature therefore justifies the electric cargo-trike context but does not by itself demonstrate that one

prototype will reduce costs or emissions. Both studies also show where such figures come from: Saenz et al. needed operational GPS and warehouse records to estimate the benefit, and the savings reported by Sebwa et al. are specific to the routes, energy per trip and prices they measured. An operator can confirm such benefits for its own vehicles only by measuring the same quantities in service: energy and battery state per route, distance travelled, utilisation, and the condition of the vehicle and its load. On-vehicle telemetry is the instrument that supplies these quantities, and this is the link between the logistics literature and the present project, whose unit measures battery state, position, ride exposure and connectivity on the actual platform.

Three-wheel geometry and a compact chassis also make vehicle response an engineering concern rather than merely a logistics detail. Changes in payload position, tyre pressure, speed, and road surface can affect both stability and measured vibration. This strengthens the case for recording operating state alongside IMU measurements and for avoiding the direct transfer of passenger-car roughness thresholds to the TUKZIE platform.

2.3 Pavement roughness and vehicle ride smoothness

Road roughness and ride smoothness are related but not interchangeable concepts. The International Roughness Index (IRI) is calculated from a measured longitudinal road profile using a standard quarter-car simulation [13]. It provides a transferable engineering measure when the profile measurement and calibration procedures are controlled. By contrast, an accelerometer mounted on a vehicle measures the combined response of the road input, tyres, suspension, chassis, payload, speed, sensor orientation, and mounting location. Consequently, raw vertical acceleration cannot be interpreted as IRI without a validated mapping and suitable reference measurements.

González et al. demonstrated that road profiles can be classified from simulated axle and body accelerations, establishing the value of vehicle-response measurements for roughness estimation [14]. The approach is attractive because it reuses vehicle dynamics rather than requiring a dedicated laser profiler. Its limitation is equally important: the inferred roughness depends on the assumed vehicle model and operating conditions. A model calibrated for a conventional four-wheeled vehicle cannot be transferred uncritically to the TUKZIE trike, whose wheel layout, suspension response, payload distribution, and low operating speed differ.

Smartphone and low-cost inertial sensing studies extend this response-based approach. Sattar et al. found that accelerometers and positioning sensors can support scalable road surface monitoring, but identified device placement, orientation, speed, sampling rate, and inconsistent ground truth as persistent sources of variation [15]. Alatoom and Smadi addressed cost and portability through an embedded framework using microcontroller and single-board computer platforms [12]. Their architecture is closer to the present project than a smartphone-only system because it permits controlled sensor mounting, deterministic sampling, and local processing. Nevertheless, published low-cost systems generally target passenger vehicles or generic road-anomaly detection; they do not establish a calibrated ride-smoothness measure for a three-wheeled electric cargo vehicle.

These findings support a deliberately bounded claim for this project. The TUKZIE unit should first produce a repeatable, vehicle-specific ride-smoothness index and event log. Correlation with IRI may be investigated when reference profile data are available, but the system should not label its output as IRI solely from uncalibrated chassis acceleration. Repeatability over repeated traversals, sensitivity to speed

and payload, and agreement with a reference sensor or labelled route provide more defensible validation criteria.

2.4 Inertial signal processing and vibration separation

An IMU signal contains gravitational acceleration, orientation changes, road excitation, vehicle-body motion, motor and drivetrain vibration, sensor noise, and transient events. The processing pipeline must therefore preserve road-related information while removing components that would make comparisons between runs misleading. Orientation compensation or transformation into a vehicle-fixed frame is required before features from the nominally vertical axis are compared. Resampling and timestamp checks are also necessary because feature values become unreliable when samples are irregular or missing.

The literature uses both time-domain and frequency-domain features. Root-mean-square acceleration, standard deviation, peak-to-peak amplitude, crest factor, jerk, and threshold counts are computationally inexpensive and interpretable. Frequency-domain energy and power spectral density can distinguish sustained narrow-band vibration from broadband road impacts. Bui et al. show that signal-processing-driven rough-road detection can be executed on an edge microcontroller, supporting the feasibility of local classification without continuous cloud processing [23]. However, threshold-based methods can overfit a particular vehicle and speed, while more complex classifiers can hide the physical reason for a detection and demand a larger labelled dataset.

Recent smartphone research illustrates both the opportunity and this limitation. Yiliguogqi et al. used vertical acceleration and roll-derived features, including standard deviation and interquartile range, with an SVM at a 10 Hz sampling rate to distinguish roughness classes [24]. The low sampling frequency and compact feature set are attractive for constrained hardware. The reported experiments classified the defined road-quality levels correctly in 80–100% of trials across four vehicles and repeated short road segments [24]. However, that range is tied to the authors' phones, routes, labels, IRI thresholds, and limited test segments; it cannot be treated as an expected accuracy for the TUKZIE without retraining and independent validation. SVM or XGBoost models should therefore be considered as comparison methods after a transparent statistical baseline has been established, not as substitutes for calibration.

For the TUKZIE platform, separation of road excitation from motor and powertrain vibration requires an empirical baseline. Stationary tests at several motor speeds can identify repeatable spectral components associated with the powertrain. Coast-down and repeated route tests can then show which components persist without powered traction and which vary with the road surface. Windowed statistics should be normalised or stratified by vehicle speed, since the same spatial road feature produces a different temporal excitation as speed changes. Rather than relying on one feature, a small interpretable feature vector combining vertical RMS acceleration, jerk, robust peak measures, and energy in selected frequency bands offers a defensible balance between sensitivity and computational cost.

Whole-body vibration standards provide useful terminology but do not replace road-profile measurement. ISO 2631-1 defines methods for evaluating human exposure to whole-body vibration [16]. Its frequency-weighted measures can inform a comfort-oriented index, whereas IRI remains a road-profile statistic. Reporting these as separate outputs would prevent the common error of treating passenger comfort, vehicle response, and pavement geometry as the same quantity.

2.5 Edge computing and real-time telemetry architecture

Edge computing moves computation closer to the sensors to reduce response time, bandwidth use, and dependence on a remote service [17]. These benefits directly match the TUKZIE use case: the driver display and safety alerts must continue to operate when cellular coverage is poor, and transmitting every raw high-frequency sample would consume more bandwidth and energy than transmitting compact features and events. The embedded HPEC concept also enables several sensor streams to be timestamped and fused before data leave the vehicle [9].

The term “high performance” must nevertheless be interpreted relative to the platform. The relevant performance is not maximum throughput, but predictable completion of time-sensitive work within a constrained power and thermal budget. A multithreaded design can isolate high-rate sensor acquisition from feature extraction, driver-display rendering, local storage, and networking, as required by the project brief [22]. Threads alone do not guarantee real-time behaviour: shared locks, blocking input/output, memory allocation, and unbounded queues can still cause missed samples or stale displays.

A producer–consumer architecture is therefore appropriate. The acquisition task should timestamp samples and place them into bounded buffers; processing tasks should consume complete windows; the driver display should read the most recent validated state without waiting for network activity; and the communication task should transmit queued summaries asynchronously. Overload behaviour must be explicit. Dropping an old display frame may be acceptable, whereas silently losing an IMU window or corrupting timestamps is not. Performance should be reported as distributions of acquisition jitter, processing time, display age, queue occupancy, and end-to-end latency rather than as an average alone.

Pérez-González et al. provide a useful contemporary comparison through a field-tested electric-racing telemetry system that combines ESP32 acquisition, Raspberry Pi 4B edge analytics, Node-RED visualisation, and LoRaWAN communication [25]. Its modular division between deterministic acquisition and higher-level processing supports the architecture proposed here. The racing study reports a 99% packet-delivery ratio and sub-second telemetry latency during its field trials [25]. These results demonstrate feasibility for that track, gateway placement, message rate, and radio configuration; they do not establish LoRaWAN as universally superior in moving vehicles. LoRaWAN offers low-power long-range packets when gateways are available, whereas 4G/LTE provides wider public-network reach and direct Internet connectivity at the cost of subscription, coverage dependence, and less predictable delay. The comparison supports retaining local logging and asynchronous communication regardless of which radio is selected.

2.6 Driver-display design, attention, and visual workload

Any driver display, head-down or head-up, takes the driver’s eyes or attention away from the road. NHTSA’s guidelines limit single glances away from the road to 2 s and a task to 12 s of total glance time, and treat a vehicle waiting at a traffic light as being driven [6]. A head-up position does not remove the problem: Liu reported that HUD use changes attention demand and driving performance, so display location and cognitive processing must be evaluated together [5], and Smith et al. found that conventional head-down-display assessment methods do not fully capture HUD behaviour, so both driving and secondary-task performance should be considered [19]. The design problem is therefore what is shown and how, not only where.

For a screen such as a vehicle’s dashboard, published guidance gives quantities against which a design can be checked. ISO 15008 sets character size by visual angle, with 20 arc minutes recommended and 16 acceptable, as reported in [70] and [71]; at about 70 cm from a 10.1-inch panel of 1280 by 800 pixels, 20 arc minutes is a capital height of about 24 pixels. Typeface matters as well: Reimer et al. measured 10.6% less total glance time with a humanist typeface than with a square one among male drivers [70]. In a driving simulator, larger touch keys improved performance up to about 17.5 mm [72], and Euro NCAP’s 2026 driver-engagement protocol sets a minimum touch-target size and assesses physical controls for key functions [73]. A study of production infotainment systems found that entering a navigation destination took about 40 s on average [74], far beyond a 12 s budget, which argues for keeping such tasks for when the vehicle is parked.

Complexity is a central risk on either kind of display. Currano et al. showed that HUD complexity influences driver perceptions of automated vehicles [20], while cognitive-load experiments on connected HUDs demonstrate that adding screens or information channels can increase mental demand [18]. Park and Im found symbol count to be the most influential interface factor in their semi-automated-driving study and recommended fewer than six symbols at a time [26], a useful initial ceiling rather than a universal threshold, since the meaning, salience, location, duration and driving context of the symbols also matter. The TUKZIE driver display should therefore use a stable information hierarchy: persistent speed and battery state, brief prioritised safety alerts, and navigation cues only when required. Decorative graphics, rapidly changing values, and simultaneous low-priority notifications should be excluded. The system should also fail safe when data are stale, removing or clearly marking invalid values rather than continuing to show them as current.

A windshield HUD was the brief’s original choice and was assessed before the dashboard was adopted. Its design space involves location, visual depth, contrast, information density, and alignment with the external scene [28], and misaligned or excessive graphics compete with hazards for attention [8]. Its optics add further constraints: the two windshield surfaces produce a double image that wedge glass or interlayers correct only for one viewing geometry [10, 27], the eye box in which the whole image is visible must be measured for each installation, and legibility depends on ambient light and viewing position [10]. On a removable, low-cost projector on a trike, with vibration and strong daylight, each of these is harder than in an integrated production vehicle. Where the vehicle already has a dashboard carrying speed and battery state, a HUD also adds a second information channel competing for the same attention [18]. These are the reasons the project integrated with the existing dashboard instead (Section 3.20.7).

Evaluation should combine technical and human-centred measures. Technical measures include rendering time, update rate, luminance or contrast under representative lighting, and the age of displayed telemetry. Human-centred evaluation can use task completion, missed-event counts, glance behaviour, perceived workload, and structured feedback during controlled low-risk trials.

2.7 Cellular synchronisation and data integrity

Cellular communication extends local telemetry to fleet and maintenance applications, but coverage and delay are outside the embedded controller’s direct control. The architecture should consequently separate the real-time control path from remote synchronisation. The driver display and local event detection must not wait for a 4G/LTE response. Each record should carry a monotonic sample timestamp, a route or session identifier, sensor-quality flags, and a sequence number so that the server can detect gaps and duplicates.

MQTT is suitable for compact publish–subscribe telemetry, whereas HTTP is useful for configuration, bulk upload, and conventional database interfaces. Protocol choice alone does not ensure reliability. A store-and-forward queue, bounded retry policy, acknowledgement strategy, and reconnection test are required. The appropriate performance measures are local data completeness, queued backlog, synchronisation delay, duplicate rate, and recovery after an induced outage. These measures directly address the project’s remote data-synchronisation deliverable [22].

2.8 Proximity sensing on low-speed vehicles

Live nearby-object alerting is a recognised requirement even for low-speed vehicles, not only high-speed autonomous ones. De Simone et al. used ultrasonic range data, classified by a supervised neural network, to add obstacle avoidance to retrofitted autonomous agricultural machines [64], showing that low-cost range sensing can support obstacle awareness on slow-moving vehicles. Closer to this project’s own vehicle class, Noomwongs et al. developed a low-speed autonomous micro electric vehicle that combines RTK-GNSS localisation with a 2D LiDAR for real-time obstacle avoidance, explicitly because vision-based approaches demand expensive sensors and computing power that such vehicles lack, a constraint this project shares [65], confirming that proximity/obstacle awareness is treated as a genuine requirement for this vehicle category in the literature, not only for highway-speed autonomous cars.

This project’s own live-alerting requirement is addressed with a time-of-flight (ToF) rather than an ultrasonic or vision-based sensor, and a solid-state multi-zone ToF sensor (the VL53L5CX) was selected over a single-point alternative specifically for its ability to resolve direction (left/centre/right) from a single device, discussed further in Chapter 3. An independent laboratory characterisation of this exact sensor [63] is used directly in that chapter to set realistic, evidence-based expectations for its accuracy and reliability, rather than relying on the manufacturer’s datasheet figures alone.

2.9 Synthesis and research gap

Existing studies establish that low-cost inertial sensors can identify road anomalies, edge devices can process sensor windows locally, and in-vehicle displays can be read within glance limits when they are sparse and legible. They also expose three limitations. First, acceleration-based roughness estimates are strongly dependent on the vehicle, speed, mounting, and calibration. Second, edge architectures are often evaluated by algorithm accuracy or mean latency without reporting jitter, queue behaviour, and outage recovery. Third, display guidance and evidence come almost entirely from passenger cars; no published design rationale for the dashboards of electric three-wheelers was found in the sources reviewed for this project.

No reviewed work integrates these concerns for a lightweight electric cargo trike in an emerging-market context, with its vibration, payload variation, limited on-board computing, and intermittent connectivity. Passenger vehicles integrate telemetry and driver displays at a cost far above this class, while low-cost road-sensing and IoV studies usually investigate their subsystems separately. This project contributes an integrated, low-cost architecture for that platform: deterministic dual-IMU acquisition with measured jitter and loss, interpretable on-board ride features, camera detection fused with time-of-flight ranging, store-and-forward LTE telemetry, and presentation of the results on the vehicle’s existing dashboard, with stale or missing data shown as absent rather than simulated. Each of these properties is defined

so that it can be measured; the bench measurements are reported in Chapter 3, and the results chapter (Chapter 4) states which have been measured on the vehicle.

Chapter 3

Methodology

3.1 Development and verification approach

Development followed a staged verification approach in which each hardware and software interface was tested before higher-level functions were introduced. This reduces the number of simultaneous fault sources and provides a traceable sequence from processor startup to cellular communication. The first stage established the microcontroller development environment. The second stage verified the hardware UART connection to the cellular modem. Sensor acquisition, task scheduling, storage, networking, and the driver display were added only after this baseline had been shown to operate repeatably.

Figure 3.1 shows the system as built on 6 October 2026, as an overview of the sections that follow: the telemetry unit, the Raspberry Pi 4 with the camera and the time-of-flight sensors, and the vehicle's existing dashboard on the Raspberry Pi 5, with the links between them and their rates.

3.2 Embedded hardware baseline

The initial platform is the Makerfabs ESP32-S3 4G LTE CAT1 A7670X board. It combines an ESP32-S3-WROOM-1-N16R8 microcontroller with an A7670X LTE Cat-1 modem and provides 16 MB flash memory, 8 MB PSRAM, native USB, modem USB, a microSD-card interface, and GNSS capability [58]. This integrated board was selected for the baseline because it permits the processor, modem, storage, and GNSS interfaces to be tested without constructing a separate modem carrier circuit.

Figure B.2 (Appendix B) reproduces the manufacturer's own labelled port and pin diagram for this board. It is referenced directly in Section 3.19 to explain a genuine Serial-routing fault (the board exposes an ESP32-S3 native-USB port and a separate CH340K USB-TTL port as physically distinct connectors, not one port with two modes), and in Section 3.20's accompanying build configuration to record that the microSD-card slot's chip-select line (GPIO10, via R23) was traced on the underlying schematic and found not populated, which is why local telemetry logging in this project uses internal flash rather than the card slot shown here.

3.3. MICROCONTROLLER STARTUP VERIFICATION

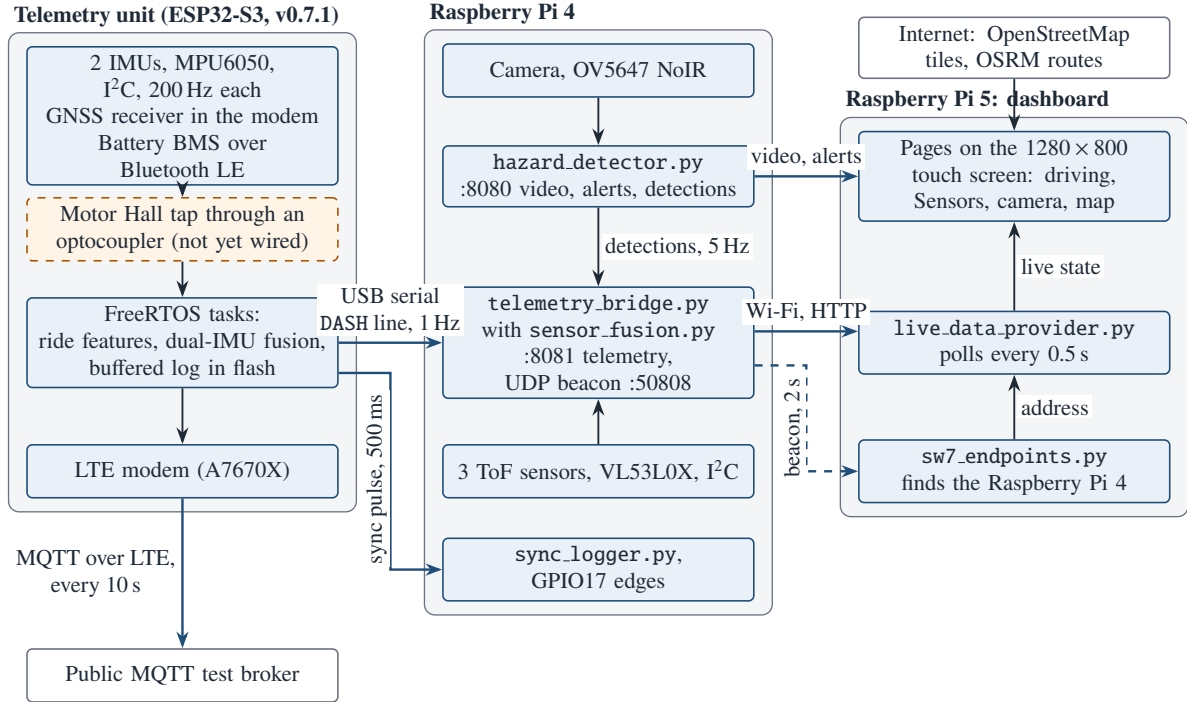


Figure 3.1: The system as built on 6 October 2026. Programs are named by their files in the repository. Dashed outline: built but not yet connected on the vehicle.

Firmware was developed in Visual Studio Code using PlatformIO and the Arduino framework. The PlatformIO target was configured as `esp32-s3-devkitc-1`. Native USB CDC was enabled using the following build definitions:

```
ARDUINO_USB_MODE=1
ARDUINO_USB_CDC_ON_BOOT=1
```

These settings expose the ESP32-S3 native USB connection as a serial interface during bench development. They are configuration choices for the selected board and framework, not general requirements for every ESP32-S3 implementation.

3.3 Microcontroller startup verification

A baseline firmware build, identified as version 0.1.0, was used to verify compilation, flash programming, boot, and host serial output. The program printed a fixed system banner and periodic status messages so that resets and interrupted execution could be identified. The runtime loop avoided long delays so that later interfaces could be added without redesigning a blocking program structure.

During bench testing, firmware could wait for the native USB serial connection before printing the startup banner. An unbounded `while (!Serial)` loop is unsuitable for vehicle deployment because the unit must boot when no host computer is connected. The deployment implementation therefore uses a bounded startup timeout rather than an indefinite wait.

The baseline acceptance criteria were:

- successful compilation without errors;
- successful firmware upload;
- one expected startup banner per reset;
- continuous execution without repeated resets; and
- readable serial output at 115200 bit/s.

3.4 A7670X UART integration

The modem was connected through a dedicated ESP32-S3 hardware UART. In the Makerfabs reference design, ESP32 GPIO 47 is the UART receive input and GPIO 48 is the UART transmit output. The modem power-key and reset controls are connected to GPIO 4 and GPIO 5, respectively [58]. The UART was configured for 115200 bit/s, eight data bits, no parity, and one stop bit. GPIO 47 therefore receives data transmitted by the modem, while GPIO 48 transmits commands to the modem.

The initial test deliberately excluded MQTT, HTTP, FreeRTOS task partitioning, and sensor drivers. Its purpose was limited to confirming modem power-up, UART framing, command transmission, and response reception.

3.5 Modem power-up sequencing

Initial versions of the UART test assumed that the A7670X would already be active when the ESP32-S3 started sending commands. After a later reset, the ESP32-S3 printed each command but received no modem response. Inspection of the board reference design showed that the modem power key is controlled through GPIO 4. Firmware version 0.2.0 was therefore updated to reproduce the documented power-key operation automatically. GPIO 4 was driven high for three seconds and then returned low. GPIO 5, connected to the modem reset input, was held low during normal operation. The firmware then allowed five seconds for modem startup before beginning the AT-command sequence [58].

Automating this operation removes the need to press the modem power button whenever the embedded unit restarts. It also makes the startup procedure repeatable for later vehicle deployment. The revised sequence was verified after an autonomous restart by recording the modem-ready indication, an OK response to AT, SIM readiness, and successful home-network registration responses.

3.6 AT-command verification procedure

After modem startup, the firmware issued the following commands in sequence:

1. AT, to test basic command and response communication;
2. AT+CPIN?, to query SIM readiness;
3. AT+CSQ, to obtain the modem's received-signal-quality indication;
4. AT+CEREG?, to query LTE/EPS network registration;
5. AT+COPS?, to identify the selected network operator; and
6. AT+CPSI?, to inspect the serving radio system and connection state.

The command meanings and response fields follow the SIMCom A76XX command set [59]. A response of OK to AT confirms command-channel communication only. SIM readiness, LTE registration, and packet-data service require their respective responses and must not be inferred from the handshake alone.

Following the automated sequence, the firmware entered a bidirectional serial bridge. Characters received from the host serial port were forwarded to the modem UART, and modem responses were returned to the host. Both directions were checked independently in each loop iteration so that manual commands could be issued without a separate firmware build. This bridge later proved useful beyond bench debugging: Section 3.13 uses it to inject AT commands directly and observe raw modem responses, decoupled from the firmware's own polling logic.

3.7 GNSS polling verification

The modem's GNSS engine was enabled by issuing AT+CGNSSPWR=1. Firmware then polled AT+CGNSSINFO at three-second intervals and forwarded each response to the USB serial terminal. Early polls were expected to fail while the GNSS subsystem was still starting. A successful interface test required the modem-ready indication, an OK response to the information query, and a correctly framed +CGNSSINFO record. A valid navigation fix additionally required populated time, position, and fix-status fields during an outdoor test with the GNSS antenna connected. This distinction prevents an empty but correctly framed response from being reported as a position measurement.

3.8 Baseline verification criteria

Table 3.1 defines the evidence required before network protocol development began. Successful completion of these tests establishes the processor-to-modem control path. It does not by itself validate Internet connectivity, MQTT delivery, communication latency, or recovery after an outage; those functions are evaluated separately using a defined broker, payload, QoS level, encryption configuration, and induced link failures.

Table 3.1: Baseline microcontroller and modem verification criteria.

Test	Required evidence	Acceptance condition
Processor boot	Versioned startup banner and continuous status output	No repeated reset loop
Modem power-up	GPIO 4 pulse followed by startup delay	Modem becomes responsive without manual button input
UART handshake	Modem response to AT	Response contains OK
SIM status	Response to AT+CPIN?	SIM reports ready
Signal indication	Recorded AT+CSQ response	Valid, non-unknown RSSI field
LTE registration	Response to AT+CEREG? and AT+CPSI?	Registered on the intended network
GNSS interface	Repeated AT+CGNSSINFO responses	Correctly framed response received after GNSS readiness
GNSS position fix	Populated fix, time, latitude, and longitude fields	Valid outdoor fix recorded
Manual bridge	Commands and responses transferred in both directions	No lost or corrupted command lines

3.9 Single-sensor inertial integration (firmware v0.3.0)

A single MPU6050 six-axis inertial sensor was added to the baseline as an interim step, connected over I²C on GPIO 8 (SDA) and GPIO 9 (SCL). A dedicated FreeRTOS task, pinned to a separate core from the modem task, polled the sensor and printed acceleration values to the serial console. This version established that the sensor could be read alongside an active modem task without interfering with cellular communication, but it did not timestamp samples, did not enforce a deterministic sampling period, and provided no mechanism for a second sensor or for downstream consumers. These limitations motivated the acquisition redesign described in Section 3.11.

3.10 Flash and PSRAM configuration verification

The ESP32-S3-WROOM-1-N16R8 module fitted to the board provides 16 MB of flash and 8 MB of PSRAM [58], but the stock PlatformIO board definition for `esp32-s3-devkitc-1` assumes a smaller flash, no-PSRAM variant. This was confirmed directly from the build tool’s own platform report, which identified the active board configuration as an 8 MB, no-PSRAM devkit rather than the physically fitted module. Left uncorrected, this mismatch under-allocates the flash partition table and leaves the PSRAM uninitialised.

A staged, single-variable change procedure was used to correct this rather than applying all changes at once, so that any resulting fault could be attributed to a specific configuration change:

1. **Baseline measurement.** The firmware was built and its reported flash partition size recorded (3,342,336 bytes), confirming the smaller default partition table was in effect.
2. **Flash size and partition table override.** `board.build.flash.size` was set to 16 MB and

`board_build.partitions` to a matching 16 MB partition table. This alone produced a boot loop, with the processor resetting approximately once per second at a fixed program counter.

3. **PSRAM configuration added.** A PSRAM memory-type override (`qio_opi`, denoting quad-I/O flash with octal-I/O PSRAM) was added on top of the flash change. The identical boot loop persisted, at the same program counter, ruling out PSRAM configuration as the cause on its own.
4. **Isolation by full reversion.** All board-configuration overrides were removed, returning the project to the original, unmodified configuration. The boot loop did not recur, and the firmware ran correctly, including live communication with the cellular modem. This isolated the fault to the flash-size and partition-table override rather than to firmware logic, application code, or a hardware fault.

Direct hardware identification via the upload tool’s own chip-detection routine (independent of the PlatformIO board manifest) confirmed the module genuinely reports *Embedded PSRAM 8 MB*, consistent with the fitted N16R8 part. The ROM bootloader’s own boot banner consistently reported the flash interface as operating in dual-I/O (DIO) mode. This is the most probable explanation for the boot loop: the attempted `qio_opi` configuration requests quad-I/O flash access, which is inconsistent with a flash chip or wiring that supports only dual-I/O access, producing an invalid image header rather than a working boot sequence.

Given this evidence, the working stock configuration was retained rather than continuing to pursue the flash and PSRAM expansion during this development session. This was a deliberate, time-bounded decision rather than an unresolved fault: at the time of testing, firmware flash usage was approximately 9% of the default 3.2 MB application partition, so the additional flash and PSRAM capacity was not required for the acquisition architecture described in this chapter. Recovering the full 16 MB flash and 8 MB PSRAM is retained as identified future work, with a dual-I/O flash and octal-I/O PSRAM configuration (`dio_opi`) identified as the next configuration to test, on the basis of the DIO evidence above.

This process is reported in full, including the unsuccessful configuration attempts, because the diagnostic method (isolating a single hardware/software boot fault by reverting one configuration variable at a time and using an independent, tool-level hardware read to check an assumption) was the substantive engineering outcome, distinct from whether the specific PSRAM configuration ultimately succeeded on the first attempt.

The flash question was later resolved when local logging (Section 3.21.1) required the 16 MB partition table. With that table in place and the flash size left at the development-board default of 8 MB, the board entered the same class of boot loop, restarting immediately after the ROM handed over to the bootloader. Setting the flash size to 16 MB and the flash mode to DIO, as the ROM banner reports, resolved it: the firmware booted normally and the 3.5 MB LittleFS logging partition mounted for the first time. This is consistent with the DIO diagnosis above, and it confirms that the flash size and flash mode must be set together with the partition table. The PSRAM memory-type setting was left at its default, since this firmware does not use PSRAM.

3.11 Dual-inertial deterministic acquisition architecture

3.11.1 Design requirements

The single-sensor implementation of Section 3.9 was replaced with an architecture intended to support the ride-characterisation methodology described in Chapter 2: two inertial sensors sampled at a deterministic rate, each sample individually timestamped and sequence-numbered, decoupled from any consumer (local logging, radio transmission, or serial reporting) via a bounded queue.

The target sampling rate of 200 Hz was derived from ISO 2631-1, which weights whole-body vibration content up to 80 Hz [16]; the sampling theorem therefore requires a rate above 160 Hz, and 200 Hz was selected to provide margin above this bound without unduly increasing the load on the I²C bus or the consuming task.

3.11.2 Task and queue structure

Each inertial sensor is served by an identical acquisition task, parameterised by a small configuration structure (sensor identifier, output queue handle, sensor driver instance, and a readiness flag) so that the timing-critical code path exists once rather than being duplicated per sensor. The two sensors are connected on the ESP32-S3's two independent I²C peripherals (one sensor per bus) rather than sharing a single bus, which avoids both an address collision (both sensors default to the same I²C address) and bus-bandwidth contention between the two sampling streams.

Figure 3.2 shows the task layout as it stands in firmware v0.7.1, including the battery, motor-speed, ride-characterisation and dashboard-line tasks added in later sections.

Each acquisition task uses `vTaskDelayUntil` rather than `vTaskDelay` to hold its sampling period. `vTaskDelayUntil` measures the delay from the previous *target* wake time rather than from the moment the current iteration happens to finish, so that variable execution time in a given iteration does not accumulate into long-run drift of the sampling rate.

No acquisition task performs serial output. Writing to the serial console blocks for a variable, non-trivial duration, which would have consumed part of the sampling period and defeated the purpose of measuring it; all reporting is deferred to a lower-priority stats task described in Section 3.14. When a queue is full, the producing task drops the new sample and increments a counter rather than blocking, so that a slow consumer can never hold up sample acquisition.

Each sample carries a sensor identifier, a monotonically increasing sequence number, and a timestamp from `esp_timer_get_time()`, an ESP-IDF monotonic microsecond clock unaffected by wall-clock or network time adjustments. Table 3.2 lists the sample structure.

3.11. DUAL-INERTIAL DETERMINISTIC ACQUISITION ARCHITECTURE

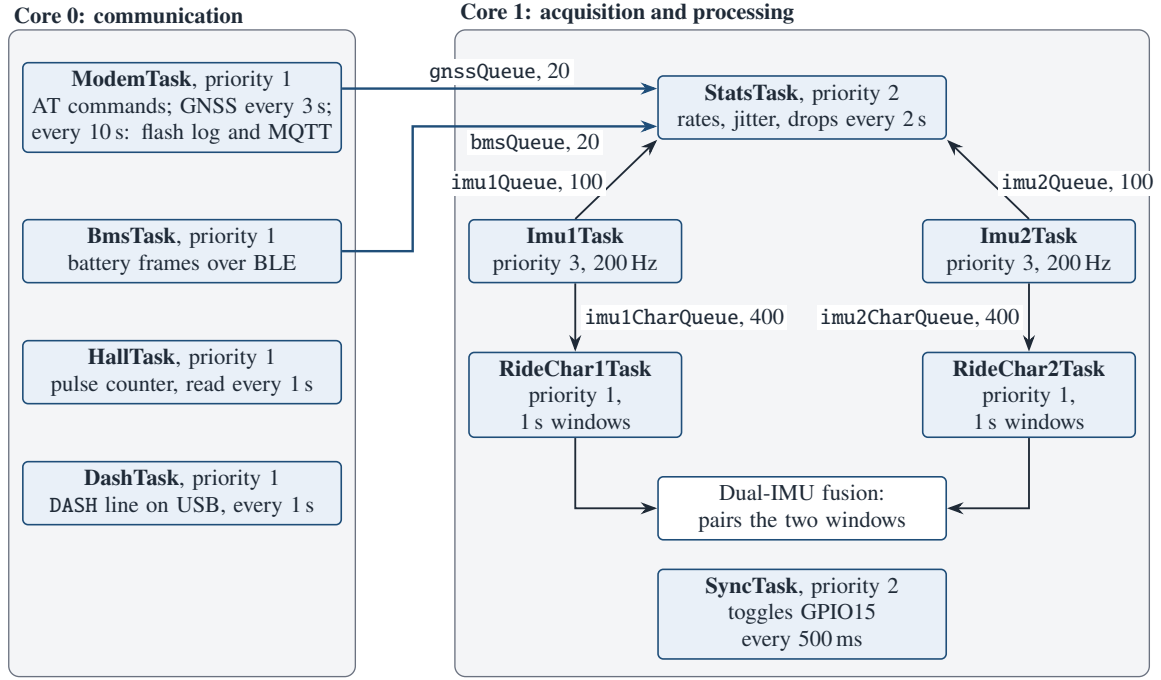


Figure 3.2: FreeRTOS tasks and queues in firmware v0.7.1. Priorities as set in the firmware (higher runs first); each queue is labelled with its name and capacity in items. Queues between the two cores are drawn in blue.

Table 3.2: Inertial sample structure.

Field	Type	Description
sensor_id	uint8	0 for the first sensor, 1 for the second
sequence	uint32	Per-sensor sample counter
timestamp_us	int64	esp_timer_get_time() at acquisition
ax, ay, az	float	Acceleration, m/s ²
gx, gy, gz	float	Angular rate, rad/s

3.11.3 Sensor configuration

Each sensor is initialised with an accelerometer range of $\pm 8g$, chosen to give headroom against clipping during pothole or kerb impacts while retaining resolution; this value is provisional and will be revisited once field data indicates the actual peak levels produced by this vehicle. The gyroscope range is set to $\pm 500^\circ/\text{s}$. The onboard low-pass filter bandwidth is explicitly set to its widest available option (260 Hz) rather than left at the sensor driver's default, because ISO 2631-1 requires content up to 80 Hz and a narrower default filter setting would have attenuated exactly the frequency band this project needs to measure. The widest setting has a cost: 260 Hz is above the 100 Hz Nyquist frequency of the 200 Hz sampling, so content between 100 and 260 Hz is not removed and can alias into the measured band. The next narrower setting, 94 Hz, would begin to attenuate the 80 Hz band of interest. Sampling the sensor faster and filtering and decimating to 200 Hz in software would avoid both, and is identified as future work.

3.12 Inter-board time synchronisation

Correlating video, inertial, and other future sensor streams (Chapter 1) across the ESP32-S3 and a Raspberry Pi requires the two boards' independent clocks to be related to one another. A dedicated FreeRTOS task toggles a GPIO output every 500 ms (a 1 s period, so two edges per second), using `vTaskDelayUntil` for the same drift-resistant timing behaviour as the acquisition tasks, and logs its own `esp_timer_get_time()` value at each edge. A companion process on the Raspberry Pi (outside the scope of this chapter) is intended to watch the same physical line on a GPIO input and record the edge arrival time against its own clock; comparing the two logs post-hoc yields the clock offset and drift between the boards.

The pulse task's timing was verified directly from captured serial output: successive logged edges were consistently 500,000 to 500,001 μ s apart over an extended capture, indicating no measurable drift over the observed window and confirming the mechanism is precise enough to serve as a shared time reference before it is connected to a second board.

3.12.1 Pin assignment error found before wiring

The GPIO initially selected for this output was chosen as a placeholder without reference to the board's schematic: the Makerfabs ESP32-S3 4G LTE CAT1 A7670X module carries peripherals beyond the microcontroller itself, and not every silicon GPIO is brought out to an accessible header. Direct physical inspection of the assembled board showed no such pin present, which was then confirmed against the manufacturer's published schematic (obtained from the Makerfabs hardware repository, since the placeholder pin had been chosen before this document was available): the board exposes exactly two thirteen-pin headers, and the pin in question is not among the signals brought out on either. The fault was caught before any wiring was attempted, avoiding the risk of a connection to an internal or unexposed net.

The schematic also confirmed, as a useful secondary result, that both IMU2's I2C pins (Section 3.11) and IMU1's original I2C pins are genuinely present on these headers, the former explicitly silkscreened as I2C signals and the latter as general-purpose GPIO usable for I2C via the microcontroller's pin matrix. The sync output was reassigned to a general-purpose pin on the same header as IMU2's I2C connection, avoiding pins with a strapping or alternate-function role that could interfere with the board's boot sequence, and allowing both signals to be wired to the Raspberry Pi through a single connector.

3.12.2 End-to-end validation

With the corrected pin wired to the Raspberry Pi's GPIO17 and a shared ground between the two boards, the mechanism was tested end-to-end for the first time: a listener process on the Raspberry Pi (using `gpiozero` to detect rising and falling edges, logging each against the Pi's own monotonic clock) was run concurrently with a capture of the ESP32's own serial log. Over a combined window, the Raspberry Pi logged 28 edges against the ESP32's 25 logged edges in its shorter capture window, with inter-edge intervals on the Raspberry Pi side clustering at 500.1–500.3 ms, consistent with the ESP32's own 500.000 ms period plus the expected interrupt-handling and Python-level logging overhead on the Raspberry Pi side. This confirms the physical connection, pin assignment, and cross-board signalling

approach are sound.

One interval fell well outside this cluster: a single 413.6 ms gap, immediately followed by a 499.8 ms gap, against a run of otherwise consistent ~ 500 ms intervals. This is reported rather than discounted, but is not yet explained; the most likely candidate, given it occurred once immediately after the physical connection was first made, is a transient contact bounce at the newly-wired connection rather than a fault in either board's timing logic, though this has not been confirmed and should be watched for over a longer capture before being dismissed.

3.12.3 A physical connection fault found during a later session

In a later lab session, the same test (concurrent capture of the Raspberry Pi's edge log and the ESP32's own serial log) unexpectedly returned zero edges logged on the Raspberry Pi side over a full capture window, despite the ESP32's own log confirming the pulse was still toggling correctly at the microcontroller pin. Isolating this to one board rather than the other in this way pointed directly at the physical link between them rather than either board's own logic. Visual inspection of the jumper wires between the two boards, which had been handled while repositioning the camera earlier in the same session, found the connection had been disturbed; reseating it and re-running the same concurrent test immediately restored correct operation (edges logged on both sides at the expected rate). This is noted as a real, if mundane, reliability consideration for eventual on-vehicle deployment: a jumper-wire connection was sufficient for bench testing but was shown, within a single session, to be vulnerable to incidental handling, which is a more benign disturbance than the vibration expected during actual vehicle operation. A more mechanically secure connector is recommended before this link is relied upon during a moving test.

The edge logger was later rewritten (`CameraDetection/sync_logger.py`) to take each edge's time from the kernel through the GPIO character device (`libgpiod`), on the same monotonic clock as the camera's frame timestamps, rather than from a Python callback. In two 60 s runs on 30 September 2026 the first recorded 159 edges instead of about 120: every out-of-range interval fell within one 16 s stretch, with bursts of extra edges as little as 0.03 ms apart, and the remaining 39 s were clean. This matches the disturbed contact seen before and is a second instance of the same vulnerability. The second run recorded 121 edges with a median interval of 500.06 ms and a range of 499.72 to 500.41 ms, a spread of under 0.7 ms, compared with 500.1 to 500.3 ms clusters and one unexplained 413.6 ms gap from the earlier Python-timestamped logger. For analysis, edges that do not fit the 500 ms pattern can be discarded, since the pulse period is known.

3.12.4 Combined video and sync capture

A single script combining video recording (`rpicam-vid`, with per-frame presentation timestamps written via `--save-pts`) with the same GPIO edge-logging behaviour described above was run for a 15 second test. This produced, in one output directory: an 18.3 MB H.264 video file; a frame-timestamp file recording 439 frames (consistent with the sensor's approximately 29 fps at 1920×1080); a sync-edge log recording 33 edges, consistent with the ESP32's 2 Hz toggle rate over the same duration; and a small reference file recording the script's own start time on both the monotonic and wall clocks.

This confirms the combined capture pipeline runs without error and produces internally consistent output, but one calibration gap remained before frame-level correlation against sync edges could be trusted: the

frame-timestamp file reports each frame's time relative to the first encoded frame (starting at 0 ms), not against the same monotonic clock used for the sync log, and the recorded script-start time necessarily precedes the camera pipeline's actual initialisation. The offset between these two time bases required a dedicated calibration test.

3.12.5 Lens-cover calibration test

A deliberate visual event, fully covering the lens by hand for several seconds during a live recording, was used to bound this offset. The moment of covering was marked by the operator sending an explicit message at the instant of covering, immediately answered by querying the Raspberry Pi's wall clock; this reported time was then compared against the recording's own accounting of the same instant (the run's recorded start time plus the elapsed time, from the frame-rate, to the frame at which the recorded video visibly changes).

An initial attempt at this test, coordinated by the operator reacting to a countdown given in advance, failed to produce a usable result: the visual event did not appear in the footage at all in one case, and in a second case appeared several seconds later than expected, most likely because reacting to an externally-given countdown introduces its own uncontrolled delay. Making the operator the one to trigger and report the event, rather than reacting to an external cue, resolved this: the reported covering time and the frame at which the image visibly changed (confirmed by inspection to be a uniform, featureless colour consistent with a hand directly over the lens, not a partial shadow or lighting change) agreed to within under one second, a gap consistent with ordinary human-reaction-plus-network round-trip latency in the reporting method itself rather than an intrinsic fault in the capture pipeline.

This bounds the previously-unmeasured offset at under approximately one second, a substantially tighter and more confident result than the prior "of the order of several hundred milliseconds, unconfirmed" statement, though it does not eliminate the offset entirely: a tighter bound would require an event whose timing does not depend on a human reporting it (for example, an LED driven directly from the sync-pulse GPIO and captured optically), which was not available with the hardware on hand and is noted as a possible refinement rather than a requirement, given that sub-second alignment is adequate for the post-hoc correlation role this camera is intended to serve.

3.13 GNSS response parsing and field-mapping validation

The GNSS interface verified in Section 3.7 confirmed that a correctly framed +CGNSSINFO response could be obtained, but the firmware at that stage only forwarded the raw response text to the serial console; no structured position, altitude, or speed data was extracted. This section describes the design and validation of a parser to extract that structure.

3.13.1 Initial field-order assumption and its failure

A commonly published SIMCom A76XX +CGNSSINFO response format was initially assumed, consisting of thirteen comma-separated fields: fix mode; GPS, GLONASS, and BeiDou satellite counts; latitude

and hemisphere; longitude and hemisphere; date; UTC time; altitude; speed; and course. The parser was written defensively against this assumption rather than trusting it unconditionally: the number of comma-separated fields in each response is checked before any field is interpreted, and the raw response text is always logged alongside the parsed result, so that an incorrect assumption would be visible in the log rather than silently producing plausible-looking but wrong values.

This defensive design proved necessary. When tested against the live modem using the manual serial bridge established in Section 3.6, a genuine no-fix response was captured as:

```
+CGNSSINFO: , , , , , , , ,
```

This is eight commas, i.e. nine fields, not the assumed thirteen. Every poll was consequently logged as an unexpected field count rather than silently misparsed.

3.13.2 Revised mapping and its remaining limitation

The field count was corrected to nine, with a working-hypothesis mapping of fix mode, latitude, hemisphere, longitude, hemisphere, date, UTC time, altitude, and speed. This mapping is explicitly a hypothesis rather than a confirmed result: an all-empty response of the form captured above confirms the field *count* but carries no information about field *order*, since every field is empty. Confirming the order requires a response captured with a genuine satellite fix, which was not obtained during this indoor test session. Validating the field mapping against an outdoor fix, and confirming the units of the reported speed field (which differ between knots and km/h across firmware revisions of this modem family), is carried forward as part of the GNSS position-fix test defined in Table 3.1.

With the corrected field count, the parser was re-tested over an extended run and produced a consistent, correctly parsed no-fix result on every poll (fix mode zero, no populated coordinate fields), with the raw response continuing to match the expected nine-field, all-empty structure. This is taken as validating the field *count* correction; the field *order* remains to be confirmed as described above.

A later check against the manufacturer’s command manual (version 1.09) sharpened rather than resolved this question. That version documents a sixteen-field +CGNSSINFO response (fix mode; GPS, GLONASS and BeiDou satellite counts; latitude and hemisphere; longitude and hemisphere; date; UTC time; altitude; speed; course; and three dilution-of-precision values), with a no-fix response of fifteen commas, and notes that other firmware variants report a different field set [59]. The fitted modem’s nine-field response therefore does not match that documented version, which means the field order cannot simply be taken from the manual either; the outdoor fix remains the deciding test. The same manual specifies the speed field in knots, which, if it also holds for the fitted firmware, must be converted before the speed-normalised roughness index in Section 3.15.3 is computed.

3.13.3 Outdoor fix and confirmed field layout

The field order was settled on 29 and 30 September 2026 with the board on a rooftop, the GNSS patch antenna facing a clear sky, and a reference position taken from a phone at the same spot (–33.9800701, 18.4654199). A with-fix reply from the fitted modem has seventeen fields, for example

```
+CGNSSINFO: 3,15,,09,02,33.9801178,S,18.4654350,E,290926,214739.00,
            82.4,0.000,69.44,1.68,0.86,1.44
```

that is, fix mode; GPS, GLONASS, BeiDou and Galileo satellite counts; latitude and hemisphere; longitude and hemisphere; date; UTC time; altitude; speed; course; and PDOP, HDOP and VDOP. Latitude and longitude are in decimal degrees, not the degrees-and-minutes form the parser had assumed. The layout matches the sixteen-field form in the manual with one extra satellite count (Galileo), and while a fix was settling the modem also sent an eighteen-field variant with an empty course and one extra trailing value, whose meaning is not documented. The nine-field no-fix reply seen indoors is simply this reply with its empty satellite block collapsed. The earlier mapping, fitted to that empty reply, read the satellite counts as the position, so the first outdoor fix was reported at 0.25 and 0.10 degrees.

The parser was rewritten in firmware v0.5.1 to 0.5.3 to find the latitude hemisphere letter and read every other field from its position relative to it, which fits all three observed layouts. A reply is accepted as a fix only if it has one of these layouts, if the coordinates, date and time have the expected form, and if the position does not imply a jump of more than 100 m/s from the previous accepted fix; this rejected replies in which characters had been dropped, which occurred when a second AT query competed with the firmware's own polling during the first capture. A console command prints the modem's raw reply on every poll, so the parser can be checked without sending extra commands.

With the corrected firmware, a six-minute capture gave a three-dimensional fix 105 s after a cold start (51 s in an earlier session), with 23 to 25 satellites and an HDOP of about 0.9 to 1.5. Over 104 fixes, the position was a median of 2.7 m from the phone's, with 95% within 11.0 m and a maximum of 12.3 m; since the phone's own position has an error of a few metres, this is consistent with normal GNSS accuracy rather than a measured absolute error. The speed field read between 0 and 5.8 while the board was stationary, mostly in the first minute of the fix, so its units (knots according to the manual) and its noise at low speed are still to be confirmed against a known speed on the vehicle. Until then the speed-normalised index of Section 3.15.3 is not computed from real data.

The same session exposed a fault in the modem power-up of Section 3.5. The power key toggles the modem, and the modem keeps its supply when only the microcontroller restarts, so the unconditional pulse at every boot switched an already-running modem off after a reprogramming: every GNSS poll and MQTT command then went unanswered for a whole capture. Earlier sessions had hidden this because the board had often been reset twice. Since firmware v0.5.2 the power key is pulsed only if the modem does not answer an AT command at boot. The fault may also explain some of the GNSS timeouts attributed to warm-up below.

3.13.4 Modem warm-up behaviour

A related, initially confusing observation was that GNSS polls timed out entirely (no response of any kind, not even an empty +CGNSSINFO line) for approximately the first 60–90 seconds after AT+CGNSSPWR=1 was issued, despite the modem responding normally and immediately to other AT commands in the same session. Because the modem's power-key sequence (Section 3.5) ran on every firmware reset, this warm-up period was re-triggered on every reflash during development, which initially made the behaviour appear intermittent. A related fault in that sequence was later found and fixed (Section 3.13.3). Testing against a modem instance that had already been running for several minutes, via the manual serial bridge, confirmed that polling succeeds reliably once this warm-up period has elapsed. This is

consistent with normal GNSS receiver behaviour, in which an internal startup or calibration routine delays the command interface for position queries independently of the shared AT command processor used for cellular functions, and is treated as expected behaviour rather than a fault, though it should be accounted for in any test procedure that enables GNSS shortly before recording data.

A later recorded session showed how this appears in the firmware’s own statistics. The modem’s start-up command sequence takes about 27 seconds after boot, and only then does GNSS polling begin, every 3 seconds. Every poll in the session then timed out, as expected during warm-up and indoors, and because the statistics count only polls that return a response, they reported zero polls throughout. A zero poll count therefore does not by itself mean that polling has stopped; the log lines reporting a poll timeout distinguish the two cases.

3.14 Acquisition-integrity instrumentation

A dedicated, lower-priority task drains all three telemetry queues (two inertial, one GNSS) and periodically reports the evidence needed to characterise the acquisition path’s real-time behaviour: achieved sample rate, minimum/mean/maximum inter-sample interval, and dropped-sample counts, per sensor, over a rolling two-second window; and GNSS poll and valid-fix counts over the same window. This task was itself the subject of a fault found during this development session: the GNSS queue was initially never drained by any consumer, since draining was added for the two inertial queues but omitted for GNSS. At the poll rate of one sample per three seconds and a queue depth of twenty, this produced a queue-full condition after approximately one minute of runtime, reported by the (non-blocking) producer as a dropped sample rather than causing a fault. The omission was corrected by including the GNSS queue in the same drain cycle as the two inertial queues.

This instrumentation directly produces the acquisition-integrity evidence reported in Chapter 4: inter-sample jitter distributions, dropped-sample counts, and queue behaviour under load, without requiring a separate measurement pass once the vehicle is in motion.

3.15 Ride-characterisation signal processing

A dedicated task, one instance per IMU, computes time-domain ride-characterisation features from each one-second window of samples: root-mean-square acceleration, standard deviation, peak-to-peak amplitude, crest factor, mean absolute jerk, and a count of samples exceeding an adaptive threshold (mean plus two standard deviations). Features are computed on the acceleration-magnitude signal ($\sqrt{a_x^2 + a_y^2 + a_z^2}$) rather than a single nominally-vertical axis, since the sensor mounting has not yet been calibrated to a known vehicle-fixed frame; magnitude is rotation-invariant and therefore meaningful regardless of mounting orientation, at the cost of not identifying which axis a disturbance came from. This is a deliberate first-pass simplification, not a final design choice, and orientation compensation remains identified future work. Because the magnitude includes gravity, its root-mean-square value sits near 9.8 m/s^2 at rest; the standard deviation, which is the root-mean-square about the window mean, is therefore the measure of vibration level used in the plots and the resting figures below. The frequency weighting that ISO 2631-1 applies before computing a weighted RMS is not yet implemented.

Each IMU’s acquisition task feeds this new characterisation task through its own dedicated queue, independent of the queue already drained by the acquisition-integrity task described above, so that adding this consumer could not alter the already-validated jitter and dropped-sample accounting for the acquisition path itself.

3.15.1 Validation against a synthetic signal

Before the physical IMUs were connected, the feature-computation function was validated independently of any sensor hardware: a pure sine wave of known amplitude was generated in software, for which RMS, peak-to-peak amplitude, and crest factor have closed-form expected values ($A/\sqrt{2}$, $2A$, and $\sqrt{2}$ respectively, for amplitude A). Run once at boot regardless of whether an IMU is present, this self-test computed RMS, peak-to-peak, and crest factor all within 5% of their analytical values against a 200-sample, 5 Hz test signal. This confirms the arithmetic in the feature-computation code is correct; it does not, and cannot yet, confirm that the resulting features usefully separate road-induced vibration from motor and powertrain vibration on the real vehicle, which requires the physical IMUs and is carried forward to on-vehicle testing, following the separation approach reviewed in Section 2.4.

3.15.2 Per-sensor magnitude calibration

Once both physical IMUs were wired and running simultaneously, a systematic discrepancy was observed between them: on the bare development board on 18 September 2026, with both sensors stationary on the same rigid platform, IMU1 consistently reported a resting acceleration magnitude of 10.35 to 10.39 m/s^2 and IMU2 9.55 to 9.56 m/s^2 . Since both sensors were stationary and subject to gravity alone, both should report a magnitude close to 9.80665 m/s^2 : IMU1 read about 5.7% high and IMU2 about 2.6% low, so the two disagreed by about 8%. This is consistent with the sensitivity and bias tolerance permitted by the accelerometer’s own datasheet between nominally identical units, rather than a wiring or software fault. Two boards from the same batch simply do not agree on the same physical state without correction.

Because the feature-computation pipeline described above operates on acceleration magnitude only (Section 3.15), the appropriate scope of correction is a single scalar multiplicative factor per sensor, applied to the magnitude signal before feature computation, rather than a full per-axis bias-and-scale calibration: RMS, standard deviation, peak-to-peak amplitude, and mean absolute jerk all scale linearly under such a correction, while crest factor and the threshold-crossing count are unaffected, since both are scale-invariant under a uniform multiply. This corrects exactly the discrepancy observed without attempting to correct per-axis bias or cross-axis coupling, which would require a multi-orientation calibration procedure and is not needed since no downstream code consumes individual axes.

The calibration factor for each sensor is computed once, the first time that sensor is detected with no stored calibration present: the firmware samples 200 readings (one second at the 200 Hz acquisition rate), computes the mean magnitude, and derives a scale factor as the ratio of true gravitational acceleration to that mean. This necessarily assumes the sensor is stationary during this window, which holds for the bench bring-up session in which calibration first ran, but cannot be assumed on every future boot once the sensor is mounted to a vehicle that may be powered on while already in motion or under vibration from other equipment. The computed factor is therefore stored in non-volatile storage and reused on every subsequent boot rather than recomputed blind; a computed factor falling outside a sanity range of 0.5–2.0 is rejected and not stored, on the basis that a result that far from unity is more likely to indicate

the sensor was not actually stationary during calibration than a genuine hardware discrepancy of that magnitude.

The factors stored by that board's first calibration were 0.9773 for IMU1 and 0.9901 for IMU2, and afterwards both sensors' resting readings were 9.80 to 9.81 m/s², agreeing with each other to within measurement noise (previously about 8% apart). The output lines of that calibration were not captured, but the factors imply mean magnitudes of 10.03 and 9.90 m/s² at the moment of calibration, not the values observed earlier the same day, so the uncalibrated readings had shifted between the two sessions. The likely cause, not confirmed, is that the sensors were moved or re-seated in between: a single magnitude factor corrects the reading only for the orientation in which it was measured, because the per-axis offsets and gains remain uncorrected. The same effect appears on the target board, whose first boot with both IMUs connected measured mean magnitudes of 9.44 and 9.57 m/s² (factors 1.0390 and 1.0245), and whose readings with those factors were 9.93 and 10.10 m/s² in a later session (Section 3.15.4). The practical consequences are that the sensors must be calibrated in their final mounted orientation on the vehicle, which the firmware's recalibration command allows (Section 3.21), and that a multi-position calibration correcting each axis would remove the dependence on orientation. This was further checked under dynamic conditions, not only at rest: physically perturbing the sensor platform produced a large, simultaneous spike in both sensors' computed features (peak-to-peak amplitude rising from a quiescent baseline of approximately 0.2 m/s² to 18.4 and 10.2 m/s² respectively, crest factor rising from approximately 1.01 to 2.17 and 1.43, mean absolute jerk rising from approximately 8 m/s³ to 160 and 162 m/s³), followed by a decay back through progressively smaller windows to the original quiescent baseline. The two sensors continued to track each other closely throughout this dynamic event, not only in the stationary case the calibration factor was derived from, which is stronger evidence that the correction generalises than the static check alone would provide. This also serves as a first confirmation, independent of the earlier closed-form self-test, that the pipeline correctly distinguishes a genuine physical disturbance from the quiescent baseline on real hardware; it is a bench-top hand disturbance rather than a real road event, and is reported as evidence the detection logic behaves sensibly on genuine kinematic input, not as a substitute for the outstanding real vehicle field test.

3.15.3 Speed-normalised roughness index

The features above describe the vehicle's response rather than the road alone: the transfer from road profile to measured acceleration depends on vehicle speed, so the same road section driven faster produces a larger RMS. Li et al. classify road roughness from the mean square of unsprung-mass acceleration divided by vehicle speed, on the basis of the relationship between the road's power spectral density, the acceleration response, and speed [62]. Following that approach, each one-second window additionally computes an index equal to the mean-square acceleration magnitude divided by the most recent GNSS speed.

Two differences from that study are acknowledged. The sensors here are to be mounted on the vehicle body rather than on the unsprung mass, so the suspension's own filtering sits between the road and the measurement, and the index uses the acceleration magnitude rather than a single vertical axis. The index is left undefined, rather than reported as a misleading value, when there is no valid GNSS fix or when speed is below a minimum threshold, since dividing by a near-zero speed produces an arbitrarily large number. Its units follow the GNSS speed field, which the manufacturer's manual gives in knots but which has not yet been confirmed on the fitted modem (Section 3.13), so until then it supports relative comparison within one dataset only. It has not yet been computed against a real GNSS fix.

3.15.4 Dual-IMU fusion

The two inertial sensors were initially reported as two independent streams. They are now also fused into a single estimate. Both are the same MPU6050 part, in the same configuration, calibrated to the same scale, so their measurement noise is taken as equal; under that assumption, inverse-variance weighting reduces to the plain mean, which is the minimum-variance unbiased combination of the two readings. The RMS, standard deviation, peak-to-peak amplitude, mean absolute jerk, and speed-normalised index are therefore averaged across the sensors. Crest factor and the threshold-crossing count are ratios and counts that do not combine meaningfully by averaging, so they remain per-sensor.

Because the sensors are mounted at different points on the vehicle, they do not measure an identical quantity, and averaging alone would hide that difference. Each fused window therefore also reports a disagreement figure: the absolute difference between the two sensors' standard deviations, divided by their mean (with a small floor to avoid dividing by near-zero values at rest). The two acquisition tasks fill their windows independently, so a pair is fused only if the two windows ended within half a window of each other on the shared microsecond clock, guaranteeing at least 50% overlap. If one sensor stops producing windows for three seconds, the fused output falls back to the remaining sensor and records that it has done so, rather than silently averaging in stale data.

The fusion logic is checked by a boot-time self-test on synthetic windows, covering the averaging, the disagreement figure, the rule that the speed index requires both sensors, rejection of a badly misaligned pair, and fallback to a single sensor; all five checks passed on the target board. In the recorded bench session described below (Figure 3.3), every window was fused from both sensors with a skew below 1 ms, the disagreement figure ranged from 0.00 to 1.68, and acquisition continued at 200 Hz on both sensors with no dropped samples. That session loaded stored calibration factors of 1.0390 and 1.0245, not the 0.9773 and 0.9901 measured when calibration was first performed, on the bare development board used for IMU bring-up (Section 3.15.2). The factors are stored in each board's own non-volatile memory, so the target board calibrated itself at its first boot with both IMUs connected, measuring mean magnitudes of 9.44 and 9.57 m/s² (Sections 3.15.2 and 3.19). With those factors the median resting magnitudes were 9.93 and 10.10 m/s², about 1.3% and 2.9% above standard gravity. Both findings are reported as found. The sensors are to be recalibrated while stationary before field testing, using a console command added to the firmware for that purpose, and the result checked against the stored factors.

Live telemetry dashboard and a recorded bench session

To view the telemetry as it is produced, a browser dashboard was written that reads the unit's serial output directly over USB, using the Web Serial interface in Chromium-based browsers, and needs no installation. It parses each line the firmware prints and shows the fused and per-sensor ride features, the sample rate and dropped-sample count of each IMU, GNSS and BMS status, free memory, and the result of the boot self-tests, with rolling plots of vibration level and sensor disagreement (Figure B.3, Appendix B). The same page can replay a saved log through the same parsing, so a recorded session can be reviewed exactly as it appeared live.

The separate bench pages for each subsystem were later combined into a single bench console. It reads the telemetry unit over the same serial link and groups its output into ride vibration, IMU acquisition, battery, position, cellular, and logging and system, next to the camera's live view and alerts from the Raspberry Pi. It also sends the firmware's test commands (described in Section 3.21), filters the serial

log by subsystem, saves a timestamped copy of a session, and replays recorded logs. Panels for the time-of-flight and air-quality sensors are included and parse a proposed line format, ready for when those sensors are connected. The console only reads data, so it does not affect the firmware or the fusion; while connected it holds the serial port, so it must be closed before the board is reprogrammed. It was checked by replaying the recorded session described below (Figure C.1, Appendix C).

A 150-second session was then recorded from power-up, with the board left still and then disturbed by hand, and saved as a log file. The planned protocol was to disturb the board between 50 and 70 seconds on audible cues, but the cues were not heard, so in the recording the board was still from 0 to about 75 seconds and disturbed from about 75 to 128 seconds; the log is annotated to say so. Figure 3.3 shows the session replayed through the dashboard. The start of the log records both boot self-tests passing:

```
[ 0.22s] [CAL] imu1: loaded stored scale=1.0390
[ 0.22s] [CAL] imu2: loaded stored scale=1.0245
[ 0.22s] Overall: PASS (ride-feature self-test)
[ 0.22s] Mean of both sensors: PASS
[ 0.23s] Disagreement figure: PASS
[ 0.23s] Speed index needs both: PASS
[ 0.23s] Misaligned pair rejected: PASS
[ 0.23s] Stale sensor fallback: PASS
[ 0.23s] Overall: PASS (fusion self-test)
```

Across the session, all 149 fused windows combined both sensors, with a skew below 1 ms (each window's fused result is printed once by each sensor's task, so the log holds 222 fused lines for 149 windows). While the board was still (the first 70 s), the fused vibration level had a median of 0.041 m/s^2 and a maximum of 0.067 m/s^2 ; during the disturbance it rose to a peak of 9.35 m/s^2 , and the largest peak-to-peak amplitude in any window was 77.6 m/s^2 , before returning to the resting level. Both IMUs sampled at 200.0 to 200.5 Hz throughout, with no dropped samples.

3.16 Battery-management system interface over BLE

The vehicle's battery pack reports voltage, current, state of charge, capacity, cell and MOSFET temperature, and protection-fault status over a Bluetooth Low Energy interface conforming to the JBD/Jiabaida smart BMS protocol. A prior implementation of this interface existed but had not been integrated into the acquisition architecture described in this chapter, and was found on review to have three defects relevant to a vehicle deployment: it created a new BLE client object on every connection attempt without releasing the previous one, leaking memory at a rate of one client per reconnection cycle; it re-established the BLE connection from scratch every ten seconds regardless of whether the previous connection was still valid, rather than holding a persistent connection; and it framed incoming notification data using only a start byte and a trailing terminator byte, which is unreliable because the terminator's value can legitimately occur inside the data payload itself.

3.16. BATTERY-MANAGEMENT SYSTEM INTERFACE OVER BLE

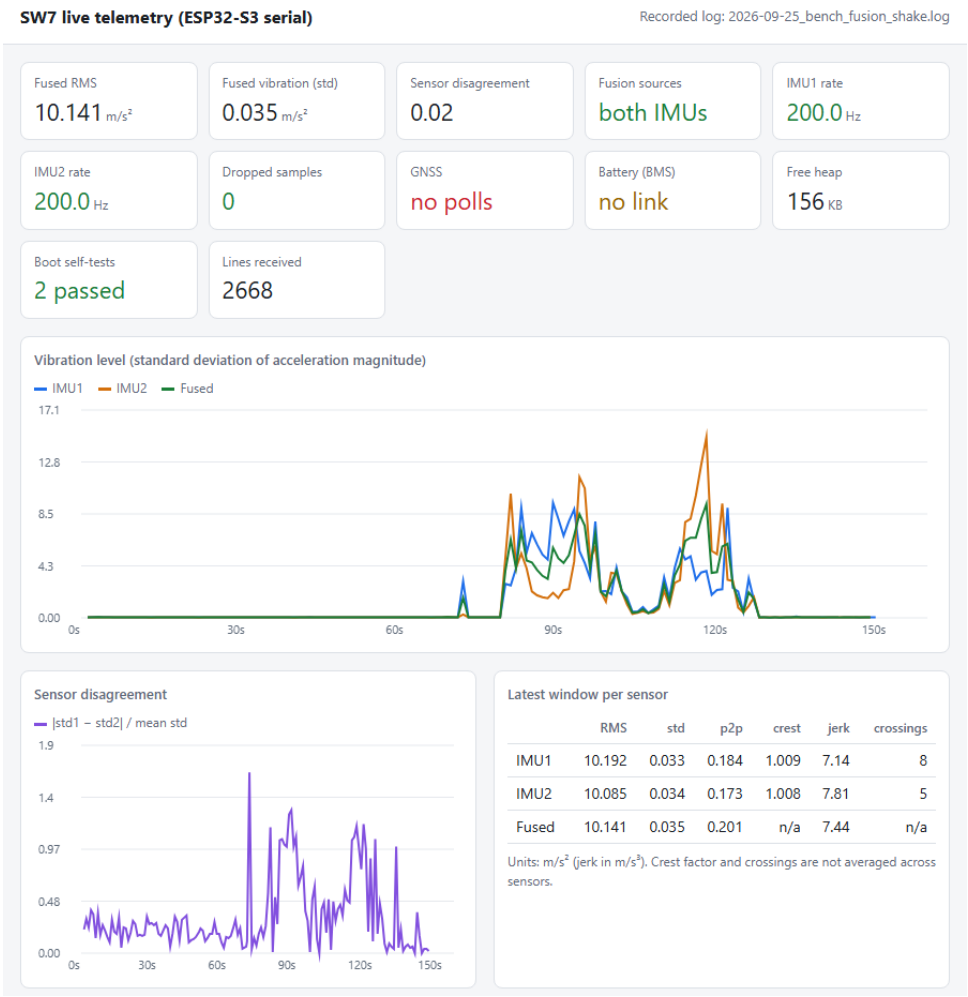


Figure 3.3: A 150-second bench session recorded from power-up and replayed through the dashboard: still until about 75 s, disturbed by hand until about 128 s, then still again.

3.16.1 Hardened interface design

The interface was rewritten around a single BLE client object, created once and reused for every connection attempt, with reconnection triggered only by an explicit disconnect callback rather than a fixed timer. Frame reassembly now uses the protocol's declared data-length field (byte index 3) to compute the expected total frame length before attempting to parse a frame, rather than searching for a terminator value that is not guaranteed to be unique. Each frame is additionally validated against a checksum computed as the two's-complement of the sum of the register, status, and length-declared data bytes, following community-documented reverse-engineering of the JBD protocol; a persistent checksum failure is intended as a signal to re-derive this formula from a freshly captured frame rather than as proof the frame itself is corrupt, since no vendor datasheet was available to confirm the formula directly. A secondary plausibility check (pack voltage, state of charge, and current within ranges consistent with a 72 V nominal pack) is applied independently of the checksum, since a checksum-valid but implausible reading is more likely to indicate a genuine fault condition worth logging than a parsing defect. Every advertised BLE device seen during scanning is logged, not only the configured target address, so that a stale or incorrect address is visible in the log rather than producing silent, unexplained non-connection.

3.16.2 A scanning fault found during testing

During bench testing, the interface was observed to log no BLE advertisements at all, from any device, over an extended scanning window in an environment known to contain multiple active BLE devices. Inspection of the reconnection logic showed that continued scanning had been made conditional on a target-address variable that is only ever set once the target device has already been found, meaning the retry path could never execute on a first run in which the target had not yet been located. Removing that condition, so that scanning resumes whenever the interface is disconnected and not already mid-scan, produced immediate and continuous device discovery on retest. This distinguishes a genuine absence of the target device from a defect preventing the interface from scanning for anything at all, a distinction that would not have been available without the diagnostic logging described above.

3.16.3 First live contact with a powered pack

In a subsequent lab session, the pack was powered on for the first time during testing. The configured MAC address was confirmed present in the scan log, this time carrying an advertised name (DB24SA01L24S150ABU) not seen while the pack was off. Read as a JBD-style model designation, the 24S and 150A components of this name are consistent with a 24-cell-series lithium pack rated to 150 A, which is independently useful evidence: it does not match the vehicle's main traction pack, which a separate inherited team's safety documentation records as lead-acid with no BMS. The pack this interface targets is therefore most likely a separate lithium auxiliary pack (plausibly the sensor/accessory supply described by another inherited subsystem's documentation) rather than the vehicle's main battery, a distinction still to be confirmed with that subsystem's owners.

3.16.4 A second assumption invalidated by live testing

With the pack advertising, `connect()` was observed to succeed on the underlying BLE link, but the subsequent `getService(BMS_SERVICE_UUID)` call returned null, so the connection was dropped before any characteristic was read. This is a second instance of the same class of fault as the scanning defect above: an assumption (that the notify/write characteristics are nested under service UUID `0xff00`) that had never been tested against a genuinely live pack. It is notable that an independently-written reference implementation, targeting the same MAC address and the same characteristic UUIDs (found in an inherited teammate's codebase), never encounters this problem, because it looks up the notify/write characteristics directly rather than first requiring a specific parent service UUID: a difference in library behaviour (Bleak's high-level API versus this project's direct use of the ESP32 Arduino BLE library) rather than a difference in understanding of the protocol. The interface was corrected to search every service discovered on the connection for the required characteristic UUIDs, rather than assuming which service they belong to, matching the more permissive behaviour of the reference implementation.

Separately, this same cross-check against an independently-written parser confirmed, byte for byte across every decoded field (pack voltage, current, remaining and nominal capacity, protection-fault flags, state of charge, and NTC temperature readings), that this project's own field-offset mapping is correct. This had previously been reverse-engineered from community documentation rather than a vendor datasheet; an independent implementation arriving at identical offsets is meaningful corroborating evidence, though it should be noted that the reference implementation does not itself validate the frame checksum, so the checksum formula specifically remains self-derived and not independently confirmed.

3.16.5 Status at time of writing

After the characteristic-lookup fix, the connection was observed to succeed intermittently rather than reliably: across repeated automatic retry cycles, most `connect()` attempts failed outright, with one confirmed success (which was the case that exposed the service-UUID fault above). A full power-cycle of the microcontroller did not change this pattern, which rules out a wedged BLE stack on the microcontroller side as the explanation. The pack continued to advertise consistently throughout, which argues against the most likely alternative explanation (another client already holding the pack's single available BLE connection), though this was not conclusively ruled out either. The interface's connection-management and frame-validation logic is therefore now corrected and partially exercised against the real, powered pack, but a fully reliable end-to-end connection, sufficient to capture and checksum-validate a complete real telemetry frame, remains outstanding, and is not yet claimed as complete.

3.16.6 Checksum fault found on the vehicle

On 30 September 2026 the board was brought to the vehicle with the pack switched on. The pack advertised under the configured address with the name `DB24SA01L24S150ABU` and the service UUID `0xff00`, which confirms the configured address and service. Its signal was weak: -93 dBm at the laptop and -88 dBm on a phone held next to the vehicle, close to the limit at which a Bluetooth Low Energy link can be held, which is consistent with the intermittent connections described above. In one session the firmware connected once and received three frames, and rejected all three as failing the checksum.

The checksum formula, flagged above as self-derived, was then found to be wrong. The firmware summed

the register byte, the status byte, the length byte and the data; the JBD protocol's checksum covers only the status byte, the length byte and the data. The query frame the firmware itself sends shows the rule: DD A5 03 00 FF FD 77 carries the checksum $0x10000 - (0x03 + 0x00) = 0xFFFD$, which excludes the leading A5 in the same position as the register byte of a reply. Including that byte made every genuine frame fail by exactly three. Firmware v0.5.4 corrects the sum and prints the raw bytes of any frame that still fails, so that the formula can be checked directly against captured frames. The live serial watch after the reprogramming did not see the pack again, but the stored log, read back the next day, shows that the corrected firmware did connect later in that session: fourteen consecutive records, about 140 s, carry a reading with a valid checksum, 74.78 V, 0.00 A and 60% state of charge, after which the connection was lost again. This confirms the corrected checksum against real frames. The reading is plausible for a 24-cell pack at rest (3.12 V per cell), but it has not yet been compared with the pack's own app. The weak signal also has a consequence for mounting: the telemetry unit, whose antenna is a small trace on the board, may have to sit close to the pack's Bluetooth module.

3.17 Environmental sensing

Environmental and air-quality measurement is provided by a Sensirion SEN55 sensor module (part number SEN55-SDN-T), which reports particulate matter, volatile organic compounds (VOC), nitrogen oxides (NO_x), relative humidity, and temperature over I²C. The module was selected so that environmental data could be recorded alongside the ride-characterisation and positioning data on the same timeline, allowing conditions along a route to be compared with the vehicle's measured response. It has been ordered from a local supplier; at the time of writing it has not yet been received or integrated into the firmware.

3.18 Motor speed tap

Separating road-induced from powertrain-induced vibration (RQ4) is easier when the motor speed is known, because vibration from the motor appears at frequencies that follow its speed. The motor's Hall-effect sensors give a pulse train whose rate is proportional to that speed, so a tap on one Hall wire is planned. The Hall signal is at 5 or 12 V and is referenced to the drivetrain's ground, so it is not connected to the microcontroller directly but through an optocoupler, which passes the pulses as light and keeps the two circuits electrically separate. The modules received are bestep single-channel PC817 boards, with input terminals, a series resistor and an indicator LED on the vehicle side, and VCC, OUT and GND terminals with a pull-up resistor on the logic side. The input voltage rating depends on the fitted series resistor, which is still to be read, and which Hall wire to tap and its voltage are still to be measured on the vehicle with the lab manager.

Firmware v0.6.0 counts the pulses with the microcontroller's hardware pulse counter, which counts both edges without software running for each pulse and so does not affect the IMU tasks, with a 12.5 μ s glitch filter. A low-priority task reads and clears the counter once a second; the edge rate is published, and the motor speed in revolutions per minute is computed as the edge rate divided by twice the number of pole pairs, since one Hall wire completes one cycle per pole pair per revolution. The pole-pair count is not yet known, so the speed is reported as null until it is set. Until the tap is wired, the counter reads the sync-pulse pin itself as a bench loopback, against which the count can be checked exactly. In the bench run of 1 October 2026 it counted 666 edges in 333.3 s between two status readings, 2.00 edges per

second, matching the pulse's 500 ms toggle exactly. The same firmware also reads the modem's signal quality every 10 s and publishes it with the telemetry, so that coverage along a route can be related to any failed publishes; on the bench it read 24 to 26 on the modem's 0 to 31 scale, about -65 dBm.

3.19 Full-system integration on the target hardware

The work described above (dual-IMU acquisition and calibration, the BMS interface, cellular/GNSS networking) had each been validated individually, but on two different physical boards: the two IMUs on a bare ESP32-S3 development board used for isolated bring-up, and the cellular modem, GNSS, and BMS interface on the project's actual target board (a Makerfabs ESP32-S3 A7670X module, the only board with the required cellular and GNSS hardware present). This section reports moving the IMUs onto the target board itself, so that ride-characterisation, BMS, GNSS, and cellular telemetry publishing all run together on the single physical board the vehicle deployment requires.

3.19.1 A misdiagnosed fault: distinguishing a hardware failure from a software configuration mismatch

After moving the physical IMU connections onto the target board, the firmware produced no serial output whatsoever beyond the ROM bootloader's own boot banner, across repeated reflashes, physical resets, and reseated wiring. Initial hypotheses, tested and each ruled out in turn, included: a hung I2C bus preventing the acquisition task from ever reaching its first diagnostic print (ruled out by adding explicit `Serial.flush()` calls immediately after `Serial.begin()`, before any I2C code executes, which still produced no output); insufficient time for native-USB enumeration before the host attempted to read (ruled out by adding a fixed startup delay, which did not change the result); and a damaged connection from the physical rewiring (ruled out by confirming the board's power LED and temperature were normal).

The eventual explanation was a software configuration mismatch rather than a hardware fault. The project's build configuration had, in an earlier session, been changed to route the Arduino `Serial` object over hardware UART0 rather than native USB, specifically to fix serial output on the bare bring-up board, which reaches the host over UART0 via an external USB-to-serial bridge chip. The target board's own schematic (obtained directly from the manufacturer's public repository, not assumed) shows a materially different design: it exposes three separate USB-C connectors, one wired to the ESP32-S3's native USB peripheral and a second wired to hardware UART0 via an onboard CH340K bridge chip, on an entirely separate physical port from the first. With `Serial` routed to UART0, application output was being sent out the second, unconnected port, while only the ROM bootloader's fixed-routing boot banner (which uses the native USB peripheral regardless of application configuration) reached the port actually connected to the host. Restoring the native-USB routing configuration for this board resolved the fault immediately and completely.

This is reported in some detail because the debugging process itself is the relevant methodological point: a symptom consistent with several plausible causes (hardware fault, timing issue, software fault) was resolved by systematically eliminating each using a targeted, falsifiable test, rather than assumed from the most recently changed variable. The two boards this project alternates between require opposite settings for this configuration, which is now documented directly in the build configuration file to prevent this specific fault recurring when switching boards again.

3.19.2 Result: simultaneous operation on the target hardware

With the configuration fault resolved, both IMUs, the BMS BLE scanner, and the cellular modem's AT command interface were confirmed running simultaneously on the target board. Both IMUs report as found and configured, each streaming a full window (400-401 samples per 2-second reporting interval, against a 200 Hz target) with zero dropped samples, and each auto-calibrating on first detection via the procedure described in Section 3.15.2 (measured scale factors of 1.0390 and 1.0245 for this particular pair of sensors on this board, both stored to non-volatile storage and correctly reloaded on subsequent boots without recalibrating). Concurrently, the modem responded to AT+CPIN? with +CPIN: READY, confirming the SIM is present and unlocked, and logged genuine network-attach events (+CGEV: EPS N ACT 1, +CGEV: ME PDN ACT 8) rather than the modem being silent or erroring, while the BMS scanner continued logging real nearby BLE advertisements throughout. This is the first point at which every major subsystem covered in this chapter has been exercised concurrently on the single physical board the vehicle deployment requires, rather than validated in isolation on separate hardware.

The I²C lines were also checked with a multimeter on 30 September 2026, with the board powered and the bus idle: both read about 3.3 V, so the IMU modules' pull-up resistors go to 3.3 V and the microcontroller's inputs, which are not 5 V tolerant, are not at risk. The check is to be repeated on the vehicle wiring after mounting.

Figure 3.4 shows the bench setup used for this integration, with each component labelled by its role in the system. The Makerfabs board forms the telemetry unit, the Raspberry Pi 4 hosts the camera, and the two are joined by the sync-pulse wire described in Section 3.12.

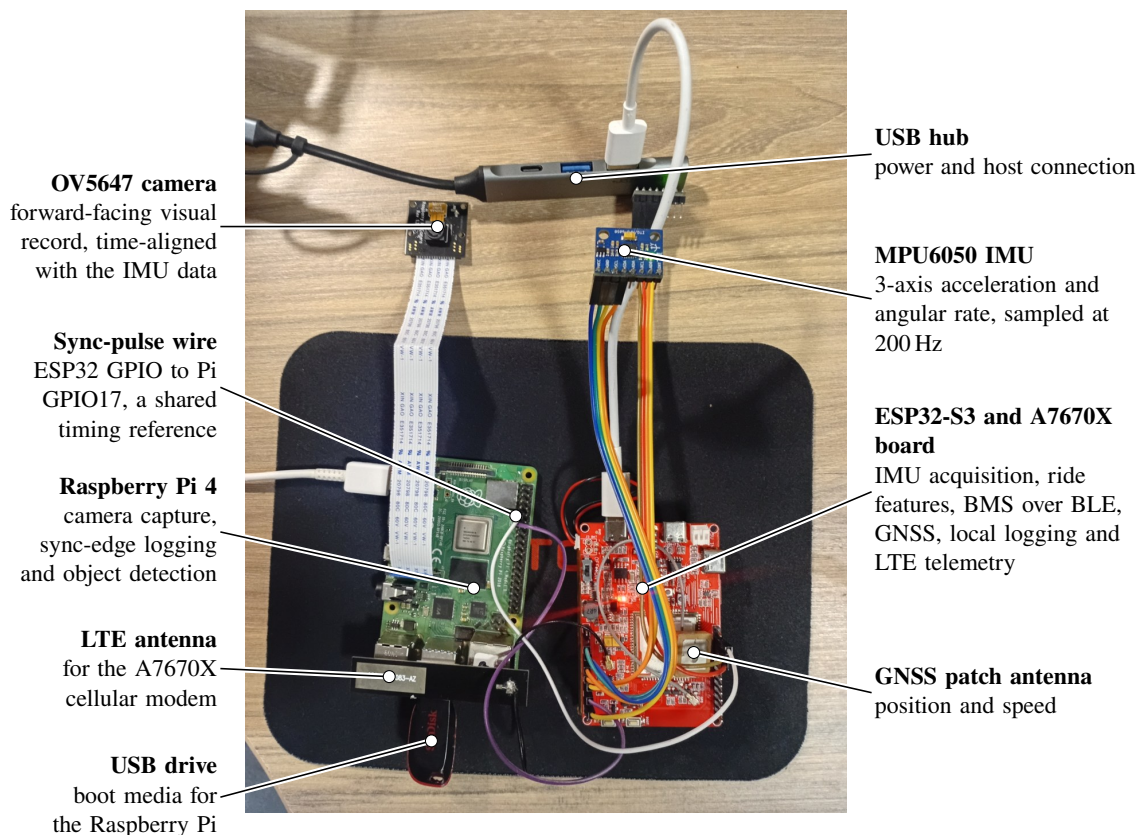


Figure 3.4: Bench setup of the telemetry system, with each component labelled by its role.

3.20 Camera subsystem: Raspberry Pi provisioning and network verification

A forward-facing camera, intended to provide a synchronised visual record against which inertial events can be labelled (Chapter 2), is hosted on a Raspberry Pi 4 rather than the project's Raspberry Pi 5. This was a deliberate choice: the Raspberry Pi 5 already runs the inherited vehicle dashboard from prior vacation work, and only one storage card was available for the two boards. Provisioning the Raspberry Pi 4 from its own dedicated USB drive, rather than the Raspberry Pi 5's card, avoids modifying or risking a working, inherited deployment in order to develop unrelated capture software.

3.20.1 Camera identification

Querying the connected camera directly (`rpikam-hello --list-cameras`) identified the sensor as an OV5647, corresponding to the original 5-megapixel Raspberry Pi Camera Module rather than a v2, v3, or High Quality module. This determines the achievable frame rate and resolution combinations available to later experimental work; the sensor supports, among other modes, 1920×1080 at approximately 32.8 fps and 1296×972 at approximately 46.3 fps, either of which is adequate for the post-hoc correlation role originally intended for this camera; the later real-time detection role (Section 3.20.7) runs at a lower 640×480 resolution.

3.20.2 A media-provisioning fault: incomplete partitioning

A second Raspberry Pi 4 USB drive, provisioned later to give the camera subsystem its own dedicated boot media independent of the first, was initially written incorrectly: inspecting the resulting drive showed a single FAT32 partition containing only boot files, with no second, larger partition present for the root filesystem. This is consistent with copying the boot-partition files directly onto the drive rather than writing a complete disk image through an imaging tool, since a plain file copy has no means of creating the ext4 root partition the boot configuration expects. The fault was caught by inspecting the drive's partition table directly rather than only its file listing, which showed one partition where two were expected; `cmdline.txt` on the faulty drive also referenced a root-partition identifier that did not exist anywhere on the disk, which would have caused the device to fail at boot rather than produce a subtler runtime fault. Re-writing the same target drive through Raspberry Pi Imager's normal write flow produced the expected two-partition layout (a ~512 MB boot partition and a ~29 GB root partition), which was confirmed both by inspecting the partition table and by cross-checking that the disk's own signature matched the partition identifier referenced in the regenerated `cmdline.txt`.

3.20.3 Network provisioning

Two network configurations were attempted before a working configuration was found. Joining the existing university network directly was unsuccessful, consistent with existing project documentation noting that the Raspberry Pi 5 is similarly not reachable over the general university network and instead relies on a dedicated mobile hotspot. A local Windows-hosted mobile hotspot was then configured, but the connecting laptop reported that it was sharing the connection over the 5 GHz band; no device joined the hotspot despite correct network credentials being applied to the Raspberry Pi. This is consistent

with a documented characteristic of the Raspberry Pi 4’s wireless interface, which requires a wireless regulatory (country) domain to be configured before 5 GHz operation is enabled at all; without this, the interface cannot use or discover a 5 GHz-only network, independent of whether the correct credentials are present. Forcing the hotspot to the 2.4 GHz band resolved this without requiring any change to the Raspberry Pi’s own configuration, and the device joined the network immediately following its next reboot.

A related, separate fault was found once the device was network-reachable: the secure-shell service did not respond on its expected port despite the device being fully booted. This was traced to the remote-access configuration not having been re-confirmed during a second provisioning pass that had been intended only to update the wireless network credentials, and was resolved directly on the device (`systemctl enable --now ssh`). This is noted because it illustrates a general risk in staged device provisioning: revisiting one configuration field to correct it does not guarantee that unrelated fields set in an earlier pass remain applied, and each should be independently verified rather than assumed.

3.20.4 A recurring connectivity fault on re-provisioned media

After the media-provisioning fault above was corrected and the device re-flashed, network connectivity had to be re-established from scratch, and proved considerably less straightforward the second time. The hotspot itself was first found to have stopped broadcasting entirely between sessions (the hosting laptop’s virtual network adapter reported a disconnected media state despite appearing enabled in software), which was resolved by re-enabling it directly. With the hotspot broadcasting again, the device still did not detect it. Querying the device’s wireless regulatory state directly (`iw reg get`) showed a discrepancy: the global regulatory domain proposed by the kernel boot parameter correctly showed the expected country code, but the wireless radio itself (`phy#0`) reported a separate, generic “world” regulatory domain with DFS channels marked unset. This is consistent with a known characteristic of the Broadcom/Cypress wireless chip used on this board, in which the kernel’s regulatory-domain hint is not always propagated to the radio’s firmware, and a country code set through the driver-level configuration path (via `raspi-config` or an equivalent wireless-configuration mechanism) is required in addition to the kernel-level hint.

Both the country-code fix and a re-confirmation that the hotspot’s own broadcast band was set to 2.4 GHz (Section 3.20 above) were applied together before connectivity was restored, and it is honestly reported here that the specific sufficient fix was not isolated, since testing was not repeated with each change in isolation once a working configuration was found. What is established with confidence is that the device’s own radio hardware and driver stack were not blocked by any lower-level fault: `rftkill` reported no soft or hard block on either the Bluetooth or wireless interfaces throughout, ruling out the simpler explanation that the radio itself was disabled.

3.20.5 Verification

With network and remote-access issues resolved, the device was confirmed reachable, correctly identified by hostname, running Raspberry Pi OS Trixie (the same release used by the project’s Raspberry Pi 5, avoiding a software-version mismatch between the two boards), reporting no under-voltage or thermal throttling condition, and reporting adequate free storage available for recorded video. The combined video-capture and inertial-synchronisation software described in Section 3.12 was subsequently implemented

and validated on this device once connectivity was restored.

A minimal live-preview capability (an MJPEG stream served over HTTP, viewable in any browser without additional software on the viewing machine) was also implemented as an operational aid, allowing the camera's physical aim and framing to be checked remotely before a recording session, independent of the main capture pipeline described above. This capability proved necessary rather than a convenience: without it, a recording session was found only after the fact to have been aimed at the ceiling rather than at any relevant scene, since the camera currently has no fixed mounting bracket and had shifted during handling. The live preview allows this to be caught and corrected before, rather than after, a recording is made, but the underlying cause, the lack of a rigid mount, remains open. The 3D-printed camera casing and corner mount described in Section 3.20.6 were adopted to address this.

3.20.6 Camera casing and mounting

Section 3.20.5 above identifies the absence of a rigid camera mount as an open reliability issue: without one, a recording session was found only after the fact to have been aimed at the ceiling. As a first design, a parametric mounting bracket was modelled in OpenSCAD, sized to the camera module and lens, with a rectangular cable-routing slot and a circular lens aperture. This bracket was not used for the build; the casing that was 3D printed is described below.

An initial render of the design appeared correct in preview, but rendering the full constructive-solid-geometry model (rather than only previewing it) revealed that the cavity cut for the lens aperture extended through the full thickness of the mounting face, consuming the material intended to retain the camera module rather than leaving a lip for it to seat against. This was corrected by bounding the cavity cut height to leave the intended retention lip in place, and re-verified with a full render rather than the preview alone, since the fault was only visible once the model was fully evaluated. Figure B.4 (Appendix B) shows the corrected design.

The camera is instead housed in a three-part casing designed as part of this project and 3D printed, shown in Figure 3.5 alongside the rest of the bench setup. A bottom shell (29.2 by 28.2 by 7.6 mm) and a top cover (4.8 mm) enclose the camera module, protecting it and its ribbon connection, and a separate 30-degree corner mount (32.3 mm tall) provides the surface by which the casing is attached to the vehicle. A second printed casing houses a second camera. The casing has been printed but not yet fixed to the vehicle, so rigid mounting of the camera remains to be completed during vehicle integration.

3.20.7 Hazard-detection scope extension

The camera's role was extended, on supervisor direction, beyond post-hoc visual correlation to include a live hazard-awareness function: reporting the presence, and an estimated rough distance, of road-relevant objects (pedestrians, cyclists, other vehicles) in the camera's field of view. This is deliberately scoped to what the Raspberry Pi 4 can sustain without a dedicated machine-learning accelerator, rather than a full autonomous-driving perception stack: a pretrained, COCO-trained SSD-MobileNet-v1 object detector (quantized to 8-bit integers as a TensorFlow Lite model, requiring no project-specific training data or training run) classifies objects in each frame, and distance is estimated from a known-object-width heuristic using the standard pinhole-camera relationship between an object's real-world width, its width in pixels, and the camera's focal length.

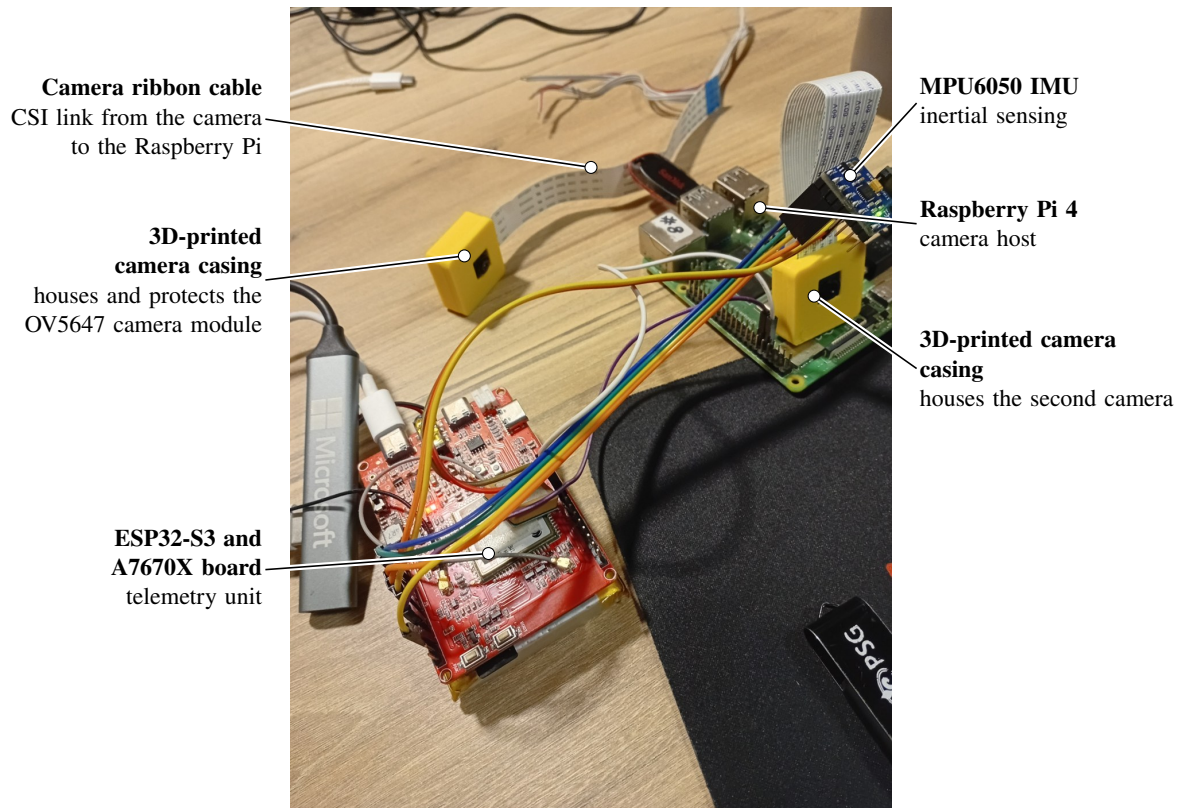


Figure 3.5: The cameras fitted in their 3D-printed casings, shown with the rest of the bench setup.

This estimate has two acknowledged limitations, stated here rather than left implicit. First, the focal length used has not yet been calibrated against this specific camera and lens; a placeholder value is used until a calibration shot (an object of known width at a known distance) is captured. Second, each object class uses one fixed average real-world width (for example, 1.8 m for a car), which is a reasonable order-of-magnitude estimate but not a per-instance measurement, since real vehicles vary in width and are not always presented front-on to the camera. Reported distances should therefore be read as indicative rather than precise until both limitations are addressed.

Implementation and data handling

This component is the only part of the system that uses machine learning: a pretrained, quantised convolutional object-detection network. Its data handling was fixed in code before it first processed live camera frames, on the bench on 28 September 2026.

That data handling is deliberately minimal. The detector reports only a fixed set of object categories (motor vehicles, motorcycles, bicycles, pedestrians, and animals), grouped into coarse distance bands (immediate, warning, and monitoring) rather than precise distances. An object class must be detected within any reportable band for three consecutive frames before an alert is raised, which suppresses single-frame flicker; an active alert is re-logged if the object holds a nearer band for three frames, and clears only after the class has been absent for five consecutive frames. For each alert, only five fields are recorded: object category, distance band, detection confidence, timestamp, and alert status. Each camera frame exists only in memory until the next frame replaces it and is never written to disk. No face or number-plate recognition, and no identification of any individual person or vehicle, is performed.

All inference takes place locally on the Raspberry Pi, and the detector has no control over any vehicle function: its output is an advisory alert only. For bench checks, an optional live view draws the detections on the camera image and serves it over the local network; it is not stored.

It is worth stating the boundary clearly in the other direction as well, because the vocabulary invites confusion: the ride-characterisation work described in Section 3.15, including its event detection and classification, is entirely classical time-domain signal processing against hand-set numerical thresholds. It contains no learned parameters and no trained model of any kind.

A complementary, non-machine-learning proximity sensor

A separate request arose during this project for a live alert to the driver when an object is near the vehicle, distinct from the post-hoc, camera-based hazard labelling described above. A VL53L5CX time-of-flight sensor (ST Microelectronics) was selected to provide this. It gives a second, physically distinct measurement of nearby objects, involves no camera image or learned model at all, and operates independently of the camera detector, so the alert does not depend on the detector's own status.

Unlike a single-point time-of-flight device, the VL53L5CX resolves distance across an 8×8 grid of independent zones within a 63° diagonal field of view, reporting a distance measurement per zone via I²C at a rated range of up to 4 m. This was chosen over the narrower single-point VL53L1X specifically for left/centre/right spatial discrimination: a single-point sensor can only report that *something* is ahead within its narrow cone, whereas the zoned output can distinguish which side of the vehicle's path an object occupies. The sensor reports raw per-zone distance only, with no object classification or image capture involved. At the time of writing, the sensor has been sourced (Pololu carrier board, confirmed in stock locally for same-day order) but not yet received or integrated; the alerting logic (a per-zone distance threshold, driving either a GPIO output or a value published alongside the rest of the telemetry described in Section 3.21) is identified as near-term future work once hardware arrives.

The rated 4 m range and per-zone accuracy are datasheet figures obtained under controlled, indoor lighting. Ambient infrared from direct sunlight is known to reduce the effective range of time-of-flight sensors below their rated figure, and target reflectivity affects the range at which a given object is reliably detected. An independent laboratory characterisation of this exact sensor [63] quantifies the reflectivity effect directly rather than leaving it as a general expectation: against a white target, the sensor's reported distance was found to be near-linear with true distance (slope 1.005 mm/mm) with a standard deviation of only a few millimetres across most of the tested range, whereas against a black target the standard deviation grew to nearly 50 mm beyond 400 mm, and the proportion of valid (non-rejected) readings fell to 18.8–34.4% at the tested distances, against 92–100% across the study's main experiments. The same study also reports a transient warm-up period of approximately 15 minutes after power-on, during which reported bias drifts as the sensor's internal temperature stabilises. Both effects are expected to be at least as pronounced outdoors on this vehicle, where sunlight adds a further reduction in range beyond what this indoor characterisation captures. This includes dark-coloured road users such as pedestrians in dark clothing, a realistic and safety-relevant case given the finding above, so the sensor's true achievable range and reliability under this project's actual operating conditions is treated as an open question to be measured directly once received, rather than assumed from either the datasheet or this indoor reference study.

Why one sensor, not several

A single-point time-of-flight sensor (such as the VL53L1X considered initially) would need to be duplicated across several fixed pointing directions to give any left/centre/right discrimination, since each unit reports only one aggregate reading for whatever falls within its narrow cone. The VL53L5CX avoids this by construction: it already resolves its 63° field of view into a 8×8 grid of independently reported zones from a single I²C device, so the spatial-discrimination requirement that would otherwise justify multiple sensors is met by one. A second unit would only be justified by a requirement this design does not currently have, such as covering a field of view wider than 63° or sensing in more than one direction from the vehicle (for example, forward and reverse simultaneously); neither is part of the current scope, so one sensor is judged sufficient for this version of the system.

Planned bench validation procedure

Once received, the sensor will be validated on the bench before integration, following the same staged, falsifiable approach used elsewhere in this chapter rather than assumed to work from the datasheet: (1) confirm I²C communication and firmware/driver identification against the expected device ID; (2) reproduce, on this specific unit, the reflectivity-dependent behaviour reported in [63] using a plain white and a plain black test target at a fixed known distance, to confirm this unit's own valid-reading rate and bias rather than assuming the published figures transfer directly; (3) sweep a single target through the sensor's rated range at fixed increments to build a measured distance-versus-reported-distance curve, analogous to the self-test already used for the ride-characterisation feature computation (Section 3.15); (4) confirm the per-zone output correctly localises a target moved across the field of view (left, centre, right) rather than only validating the aggregate case; and (5) confirm the warm-up transient reported in [63] either is present on this unit or is shown not to materially affect the alert threshold used, before the sensor is relied upon for any on-vehicle test.

Stand-in sensor until the VL53L5CX arrives

Because the VL53L5CX had not arrived by 30 September 2026, the time-of-flight sensors from a UCT micromouse sensor board are used in the meantime. The board carries three small sensor modules, one on each of its front and two angled side edges, so they face left, ahead and right: the several-fixed-directions arrangement described above, in its simplest form. The micromouse schematics give the part as the VL53L0X (VL53L0CXV0DH1), a single-zone sensor with a range of about 2 m, shorter than the VL53L5CX's 4 m. Each module has a six-pin header (interrupt, shutdown, data, clock, ground and 3.3 V) with pull-up resistors on the shutdown and interrupt lines but none on the I²C lines. All three sensors start at the same I²C address, so each shutdown line goes to its own pin and the sensors are woken one at a time and given separate addresses. The sensors are connected to the Raspberry Pi 4 rather than the microcontroller: the Pi sits at the front beside the camera, its I²C bus is free, and the microcontroller's 200 Hz acquisition is left undisturbed. A probe script identifies the fitted part from its model-identification register, since the VL53L0X, VL53L1X and VL6180X share one address and one footprint, and a reader script gives the left, centre and right distances. On 1 October 2026 the whole micromouse sensor board was connected to the Raspberry Pi through its own header (3.3 V, ground, the two I²C lines and the three shutdown lines; the board's LED supply left unconnected). The probe confirmed the VL53L0X, and the reader woke the three sensors in turn, gave each its own address and read all three at about 31 readings per second each. Over 45 s on the bench the centre sensor read 267

to 300 mm and the right sensor 24 to 38 mm, both steady, while the left sensor returned 8190 on every reading, the sensor's code for no target within range, rather than failing to answer. Covering each sensor in turn with a fingertip, left, centre and then right, brought the matching reading down to 20 to 46 mm in the same order, which confirms both that each sensor responds and that the software's left, centre and right labels match the physical sensors. Distances have not yet been checked against a tape measure. How these three sensors are combined with the camera is described in Section 3.20.8.

Status at time of writing

The detector was first run against the live camera on the bench on 28 September 2026 (Raspberry Pi 4, 640×480 frames, LiteRT runtime), sustaining about 7 frames per second over two 60 s runs (430 and 419 frames). The first run exposed two input-handling faults. The label file's leading placeholder entry shifted every class name by one, so a person in view was never reported, and the camera's RGB888 buffers are stored in blue, green, red order while the model expects red, green, blue [60]. After both were corrected, a person in view was detected with confidence between 0.50 and 0.73; the two fixes were applied together, so their individual effects were not separated. The same run showed an alert clearing on a single missed frame, so one continuously present person produced 16 activations within 47 s, which led to the clear-side hysteresis described above. Only the person class has so far been confirmed on live frames, distance bands still rely on an uncalibrated focal length, and the detector has not yet been tested outdoors or on the vehicle. A further open question, raised with the project supervisor but not yet answered, is what specifically should count as a reportable hazard for this vehicle and use case (road-surface defects specifically, versus general object presence of any kind); the current implementation reports a fixed set of general COCO object classes as a starting point, pending that clarification.

Alert hysteresis

The alert flicker seen in the second live run came from the clearing rule: an alert cleared on the first frame in which its class was missing, so detections close to the 0.5 confidence threshold switched it off and on repeatedly. The rule was changed so that an active alert clears only after its class has been absent for five consecutive frames, about 0.7 s at the measured frame rate. An active alert is also logged again if the object holds a nearer distance band for three consecutive frames, the same debounce as an onset, so that an approaching object is recorded as it gets closer while a single noisy frame cannot escalate it. Within a frame, the nearest detection of a class is used, and confidence only decides between detections in the same band. These rules were checked offline, by replaying a scripted sequence of detections through the unchanged detection loop with the camera and model replaced by stand-ins: a person present throughout with single missed frames, a single weak near detection, a sustained approach and a departure. Missed single frames no longer cleared the alert, the single near frame did not escalate it, the sustained approach did, and the alert cleared once the person had gone. The rules were later seen working on live frames: in the dashboard page test on 1 October 2026 the detector's log shows person alerts both raised and cleared.

Live view and latency measurement

To check that the detector responds to what is actually in front of the camera, and not only through its log, an optional live view was added. It draws each detection (class, confidence, estimated distance and band) on the camera image and serves the result as an MJPEG stream over HTTP, to a browser on the

Raspberry Pi itself by default or, when enabled, to other devices on the same local network. Frames exist only in memory and are never written to disk. In the first version the drawing and JPEG encoding ran inside the detection loop, and the detector was then observed to respond more slowly. Drawing and encoding were therefore moved to a separate thread that always takes the newest frame and skips older ones, so a slow viewer delays only the live view and never detection; handing a frame to that thread was measured offline at about 0.3 ms. The stream also uses a small network send buffer, so a slow link drops frames rather than showing an image that falls further and further behind.

To quantify edge processing latency, as the project brief requires, every frame's latency is now measured from the moment the sensor produced it. Each captured frame carries a `SensorTimestamp`, which the camera library documents as the time in nanoseconds since boot at which the first pixel was read out of the sensor, on the same clock as the monotonic clock used by the detector [60]. The detector records, per frame, the queueing time from readout to capture, preprocessing, inference, and alert logic, and the total from sensor readout to result. It prints the median, 95th percentile and maximum of each every five seconds and can write every frame's timings to a file. The measurement excludes the exposure time itself. Inference also now uses all four processor cores of the Raspberry Pi 4, and the frame is resized with bilinear rather than bicubic interpolation. The latency was measured on the Raspberry Pi on 30 September 2026 in three one-minute runs with the camera on the bench (`BenchTest/logs/2026-09-30_c4_latency/`), shown in Table 3.3. With four threads, the median time from sensor readout to result was 102 ms (95th percentile 120 ms, maximum 136 ms) at 19.5 frames per second. Inference took 36 ms, preprocessing 12 ms, and the alert logic under 0.2 ms; the largest share, about 52 ms, was the time from readout until the detector received the frame, which covers readout itself and the camera's image processing. With the live-view server running (no viewer connected) the rate fell to 17.3 frames per second and the median rose by 3 ms. With one thread, inference took 126 ms and the rate fell to 7.0 frames per second, the same as the earlier live tests, which is consistent with those runs having used a single thread. The processor reached 65.7 °C with no throttling. At 30 km/h a median of 102 ms corresponds to 0.85 m of travel, and 1.0 m at the 95th percentile. The three-frame onset debounce adds about two frame intervals, roughly 100 ms at 19.5 frames per second, so an alert is raised about 0.2 s after an object first appears, or about 1.7 m at 30 km/h. These are bench figures with no objects to track; the number of detections does not change the model's inference time, but the vehicle's own conditions (heat, supply) may.

Table 3.3: Camera detector latency on the Raspberry Pi 4 (ms, median / 95th percentile; 60 s per run).

Run	fps	Queue	Pre	Inference	Alert logic	Total
4 threads	19.5	52 / 71	12 / 14	36 / 39	0.1 / 0.2	102 / 120
4 threads, live view	17.3	57 / 78	12 / 12	36 / 39	0.1 / 0.2	105 / 127
1 thread	7.0	47 / 68	12 / 14	126 / 129	0.1 / 0.2	186 / 207

Alerts feed, dashboard page and start-up service

So that the vehicle's existing dashboard can show the camera's alerts, the detector also answers a request for its currently active alerts with a short JSON record containing only the five fields of the alert log, plus the current frame rate and latency. A page for the inherited PySide6 dashboard was written that shows the live view and these alerts in the dashboard's own status colours, polls the alerts twice a second, streams video only while the page is on screen, decodes only the newest frame, and reconnects on its own if the camera drops out. It uses only the Qt networking and widget modules, not the web-engine component that has crashed on the dashboard's Raspberry Pi 5. The page was tested offscreen on a laptop inside a copy of the dashboard's code: it picked the newest frame from a stream fed in uneven chunks, listed and cleared alerts, and stayed responsive when the camera address could not be reached. On 1 October 2026 it was run against the live detector on the Raspberry Pi 4 over the local network for 60 s: it showed the

live view 0.6 s after starting and decoded 638 frames in the 45 s the detector ran, 14.2 per second against the detector's 14.8, with no gap longer than 0.22 s; it displayed the frame rate and latency reported by the detector (about 15 frames per second and 76 to 87 ms) and changed to its offline state within a second of the detector stopping (Appendix C). In a second 40 s run with a person standing in front of the camera, the page listed a red "Person, Immediate" alert 0.5 s after starting, matching the box drawn on the video, at 15.4 frames per second and 109 to 118 ms. The person stayed in view throughout, so the alert clearing on the page was seen only in the simulated test and when the camera stopped; the detector's own log for the session shows both active and cleared person alerts. It has not yet been run on the dashboard's Raspberry Pi 5, and adding it to the dashboard needs four small edits to the dashboard team's files, which are left for that team to review. A service definition was also written so that the detector starts when the Raspberry Pi 4 boots and restarts if it stops; it has not yet been installed.

Telemetry link to the dashboard

An inspection of the dashboard's code on 2 October 2026 showed that it has no live input: every page is driven by built-in simulated data, there is no MQTT, serial or working CAN reader, and a single unused entry point accepts a complete live vehicle state, falling back to simulation if no live state arrives for 2.5 s. The 10 s MQTT cycle is too slow for that, and a driver's display should not depend on cellular coverage or a remote broker, so a local path was added (`DashboardIntegration/TELEMETRY_LINK.md`). Because the telemetry unit and the Raspberry Pi 4 share the front enclosure, already joined by the camera ribbon and the sync-pulse wire, the Raspberry Pi 4 acts as the vehicle's gateway: firmware v0.7.0 prints one compact JSON line per second on its USB serial port, a bridge program on the Raspberry Pi 4 reads it together with the three time-of-flight sensors and serves both over HTTP, and a live-data provider in the dashboard reads that once every 0.5 s, so the dashboard needs a single network link, to the Raspberry Pi 4, for the camera, the alerts and the telemetry. The provider fills the dashboard's battery state of charge, battery power, position, GNSS validity and front-obstacle distance; speed is left unmapped until the GNSS speed units are confirmed, and the front-obstacle distance uses the forward sensor only, so that a car parked beside the vehicle does not raise a front warning. The ride-quality figures, which the dashboard has no fields for, are shown on a new card on its diagnostics page. On the bench, 40 lines arrived in 40 s with no gap in the sequence numbers, every line valid JSON and at most 272 bytes, and both inertial sensors stayed at 200 Hz with no dropped samples (Appendix C). With the telemetry unit plugged into the Raspberry Pi 4, the bridge served the live record, then 0.37 s old, and the three distances to a laptop over the local network, with no malformed lines, and opening the port did not restart the telemetry unit. The dashboard side was tested offscreen on a laptop against a copy of the dashboard's newest version with the integration edits applied and a simulated bridge, including missing values, stale data, malformed replies and an unreachable bridge, in which cases nothing is passed on and the dashboard's own fallback takes over (simulation in the original dashboard; an explicit no-data state in the live-only mode described below). A separate copy of the dashboard with these edits was later started on the Raspberry Pi 5 (below), but a recorded end-to-end test there, from the telemetry unit to the screen, is still to be made. The edits to the dashboard team's files are listed for that team's review and have not been applied to their repository.

Live-only mode

On 3 October 2026 the Raspberry Pi 4 moved from the laptop's hotspot (192.168.137.82) to a phone hotspot (10.74.67.244). The dashboard was still set to the old address, could not reach the bridge, and fell back to its simulated data without any visible sign, so plausible but invented values were on the screen. A

related fault had already been seen on 2 October: while the battery-management system had not reported, the battery read 82 %, the simulator's last value, not a measurement. For a vehicle display both are worse than showing nothing, so a live-only mode was added (`DashboardIntegration/sw7_live_only.py`), switched on by the environment variable `TUKZIE_LIVE_ONLY=1`, which the start script sets. In this mode the simulation never runs. Until live data arrives, and within 2.5 s of it stopping, the dashboard publishes an explicit no-data state in which every sensor-derived field is empty and every validity flag false, so the pages show -- or "Unavailable" and a short message on screen says that live data was lost and no data is shown. Live states are built on the same blank state, so a field with no live source (speed, pedals, temperatures, tyres and so on) stays empty rather than taking a default. The state of charge is empty when the battery-management system has never reported, in both modes, so the 82 % figure can no longer appear. Gear, indicators, headlights and parking brake still come from the touch and keyboard inputs. Without the variable the dashboard behaves as its team wrote it. The change needs 34 small edits across 12 of the dashboard's files, kept with the other integration edits for the team's review. Tested offscreen on a laptop (3 October 2026): with the bridge down, every page showed -- or "Unavailable", painted without an error and never showed the simulator's values; the dashboard went live when a simulated bridge appeared and returned to the no-data state, not to simulation, when it stopped. Not yet tested on the Raspberry Pi 5 with the real bridge.

Finding the Raspberry Pi 4 on any network

No single address is reliable on a vehicle that may be tested on a laptop hotspot, a phone hotspot or a wired link, so the dashboard now finds the Raspberry Pi 4 by itself (`DashboardIntegration/sw7_endpoints.py`; `TELEMETRY_LINK.md`, section 4). One shared resolver serves both the camera page and the live-data provider. It tries, in order: the Raspberry Pi 5 itself (localhost), so that a single computer running everything needs no setup; an explicit override, if one is set; a fixed address on a planned Ethernet cable between the two computers (10.20.0.1 for the Raspberry Pi 4 and 10.20.0.2 for the Raspberry Pi 5, on a private subnet with no gateway so that Wi-Fi keeps internet access); the source address of a UDP broadcast beacon that the bridge sends every 2 s on port 50808 (beacons older than 10 s are ignored); the name `pi4-camera.local` through mDNS; and the last address that worked. A candidate is accepted when the bridge answers with valid JSON within 3 s (Figure 3.6). The limit was first 1 s, which failed on the phone hotspot on 3 October, where each round trip added 0.1 to 0.8 s. Three failed polls in a row, about 1.5 s, start a new search, and the ride card shows where the Raspberry Pi 4 was found and how (for example "Pi 4: 10.20.0.1 (wired)") or that it is searching. Empty searches first repeated every 2 s and filled the Raspberry Pi 5's log with 6,828 lookup warnings on 3 October, so the wait now doubles to 2, 4, 8, 16 and at most 30 s, while a beacon that arrives meanwhile is tried at once. The offscreen test checks the localhost, override and beacon steps, that an unrelated or old beacon is ignored, and that after the bridge stops the provider, the ride card and the camera page all follow the newly found host; it passes. The wired link has a setup script but its use on the vehicle is not yet recorded.

A pre-test check (`BenchTest/sw7_check.py`) is run from the laptop or the Raspberry Pi 5 before a test session. It reports pass or fail for each item: the Raspberry Pi 4 found and how, the camera answering with its frame rate, telemetry fresh (record under 3 s old), IMU rates and drops, all three time-of-flight sensors valid, the detector and bridge services running, the Raspberry Pi 4's under-voltage and throttling flags, the Raspberry Pi 5 reachable, and the dashboard running there with live-only mode set. It prints "READY" only if every critical item passes. Phone hotspots were found on 3 October to block mDNS and to drop pings between clients, so the check finds the Raspberry Pi 5 from a beacon that the Raspberry Pi 5 now sends, its last known address, the Raspberry Pis seen on the local network, and finally by scanning the local network for open ssh ports, confirming each candidate by asking its hostname over ssh. Its 16 offline tests pass. Against the real system on 4 October 2026, with the laptop on a phone

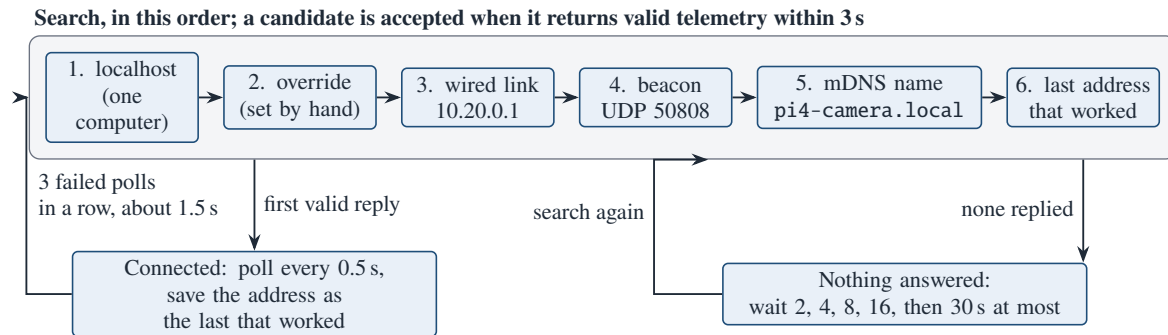


Figure 3.6: How the dashboard finds the Raspberry Pi 4 (`DashboardIntegration/sw7_endpoints.py`). The same resolver serves the camera page and the live-data provider.

hotspot, it reported ready: the Raspberry Pi 4 was found through its beacon, telemetry answered in 18 ms with the telemetry unit's record 0.5 s old, both inertial sensors at 200 Hz, all three distance sensors reading, the camera answering, both services running, and the Raspberry Pi 5 running the integrated dashboard in live-only mode. It also failed one item: the Raspberry Pi 4 reported under-voltage and throttling at that moment and since boot, which matches the detector running at 6.1 frames per second against 19.5 on a proper supply, and shows that the supply rating is a requirement, not a detail. Earlier the same day the check had exposed three faults that it then guided the fixes for: a 1 s probe timeout shorter than the hotspot's round trip, which was raised to 3 s; the dashboard Raspberry Pi not found by name or ping, because the phone hotspot drops both between clients, so the Raspberry Pi 5 now announces itself and the check also scans for open ssh ports; and the camera detector not running after a reboot, which was then installed as a service.

Deployment and start-up on the Raspberry Pi 5

The dashboard's map crashed the whole program on the Raspberry Pi 5, so it was diagnosed over ssh on 2 October 2026 without changing the team's installation (`DashboardIntegration/PI5_MAP_CRASH.md`). The installed dashboard was version 1.1, newer than the copy the integration edits were written against. The web engine on its own loaded and closed normally, offscreen and on the real display, but a second instance of the dashboard on the real display ended with a segmentation fault about 2 s after it was shown whenever the web map was selected, with online or offline map data; with the native map selected it did not crash and was still running after 30 s. A native crash cannot be caught by the page's Python error handling, and the web map was created in the background even when the native map was selected, so one edit stops it being created in that case. On 2 October a separate copy of version 1.1 was made on the Raspberry Pi 5 (`~/Dashboard_sw7`); the 13 integration edits of the time applied to it unchanged, the native map was selected, and the map page then opened without a crash. Its map area was empty indoors because there was no GNSS fix, which led to the street-tile map below.

Two further faults were found on the Raspberry Pi 5. The screen went blank after 600 s without a touch, from the X11 screensaver and display power management, which must not happen while driving; the start script now turns both off. The dashboard team measured their game-controller poller at about 53 % of one processor core, and the vehicle has a touch screen only, so a 49th edit lets the start script switch the poller off. A deploy command (`DashboardIntegration/deploy/deploy_dashboard.sh`) now repeats the whole procedure from the laptop: it copies the integration files, stops only the integrated copy, keeps the previous build, makes a fresh copy of the team's dashboard, applies the edits, selects the native map, records the deployed commit in the copy and restarts it. The patch program refuses to

touch the team's own folder and writes nothing unless every edit's anchor text occurs exactly once, so the team's dashboard is never changed and a failed run leaves the copy as it was. The edit set now holds 50 edits (14 for adding the pages and stopping the web map, 34 for live-only mode, one for the controller and one for the street-tile map). An auto-start entry starts the integrated copy at login after a 5 s wait for the desktop, with live-only mode on and no fixed addresses. On 3 October 2026 the deploy rebuilt the integrated copy on the Raspberry Pi 5 from the team's installed version 1.1 (the same code as their repository's latest commit) with every edit applying exactly once, and the team's own copy unchanged; the team's two auto-start entries were moved to a backup folder, so they can be restored, and replaced by one for the integrated copy. The 50th edit, for the street-tile map, has not yet been deployed.

Course and altitude (firmware v0.7.1)

A heading-up map needs the direction of travel, which the dashboard line did not carry. Firmware v0.7.1 adds two fields to it: `crs`, the GNSS course over ground in degrees clockwise from true north, and `alt`, the GNSS altitude in metres. Both are null without a fix or when the receiver gives no value, and the dashboard uses them only with a fix; a missing course is left empty rather than set to 0, which would mean north. A recorded bench check of the v0.7.1 output, like the one for v0.7.0 above, is not yet in the repository.

Street-tile map

The team's fallback map draws only vector roads from a small offline index around the campus, and the web map crashes on the Raspberry Pi 5, so a street map that does not use the web engine was written (`DashboardIntegration/sw7_tile_map.py`). It replaces the fallback map with the same interface, so one edit in the navigation page switches it in and an environment variable switches it back. It draws OpenStreetMap raster tiles directly with Qt's painter. It follows the OpenStreetMap tile usage policy: at most two requests at a time, an identifying user agent, only tiles on screen and only once the zoom has settled, and no bulk download. Tiles are kept in a disk cache and a tile under 30 days old is never fetched again, so roads already driven work without a network; after a network error it waits 15 s before retrying, or 5 min after a refusal from the server. A missing tile is replaced by a scaled-up lower-zoom tile or, failing that, by the team's vector roads, so the map is never blank, and the attribution is always shown. In follow mode the map turns so the direction of travel points up, with the vehicle as an arrow at 70 % of the height and zoom level 17. The course is trusted only while moving; when stopped, the last good heading is held for 20 s and the map then eases to north-up. Each fix, once a second, starts a smooth move of the marker over the measured update interval, and the redraw timer runs only while the map is visible and moving. Without a position the map stays at the last known place, or the campus, with the banner "Waiting for GPS fix".

To test the moving map without driving, a GPS track was generated along roads on the UCT campus (`CameraDetection/tests/replay_uct_route.txt`), from a driving route computed by the public OSRM demonstration server on OpenStreetMap data: 2,993 m, 650 points at one per second, at a target of 18 km/h. It is not recorded data. Every line is marked as a replay (firmware field `0.7.1-replay`, battery and motor fields empty, vibration made up), the bridge reports that it is replaying a file, and the map and the ride card then show an amber REPLAY badge, so a replayed position cannot be taken for a real one. The offscreen test feeds this track through the live-data provider to the map against a local stand-in tile server and checks the tiles requested and cached, heading-up follow, smooth movement, the no-fix banner, the REPLAY badge, the vector-road fallback with the tile server down, and touch

interaction; it passes against both dashboard versions. It was tested offscreen first and has since run on the Raspberry Pi 5 (Section 3.20.7); it has not yet followed a real fix.

Start-up as the default dashboard, and driver-facing changes (5 to 6 October 2026)

After a restart on 5 October 2026 the team's dashboard came up instead of the integrated copy. The Raspberry Pi 5 boots to a bare X display without a desktop session, so desktop auto-start entries are never read; the team's copy is started by a system service (`tukzie.service`). An equivalent service for the integrated copy (`DashboardIntegration/deploy/sw7-dashboard.service`) restarts it 5 s after a failure and gives it access to the sound server. It was enabled on 6 October in place of the team's service, whose unit file and folder were left unchanged, so one command reverses the switch. After the next restart the integrated copy was the one running, under the new service.

Further changes followed bench use on 6 October, each a separate edit to the integrated copy (63 edits in all):

- The driver and PIN page is skipped while an environment variable set by the start script is on. This is for bench work; the page is to be restored for the demonstration.
- The Settings page could be moved only by its narrow scroll bar on the touch screen, and it listed developer options next to the rider's. Finger-drag scrolling was added, the wording simplified, and the route provider, map engine, fallbacks, voice engine and kiosk options placed behind one "Show advanced settings" switch, with the stored preferences unchanged. Drag scrolling at first held each tap for Qt's default 0.25 s to tell a tap from a drag; the hold was set to zero.
- A Sensors page and a hazard banner were added (`DashboardIntegration/sensors_page.py`). Until then the time-of-flight distances and the fused hazards of Section 3.20.8 were not shown anywhere on the dashboard. The page shows the three distances as gauges (amber under 1.0 m, red under 0.5 m), the fused hazards nearest first, and the battery, position, motor and ride readings, with "—" for a missing value and the state of the link. The banner appears over every page when the fusion reports an immediate hazard, or when the link to the Raspberry Pi 4 has been lost for 5 s after working; touching it opens the page, and a button hides it for 30 s. In an offscreen run of the whole patched dashboard on the Raspberry Pi 5 against the live bridge, the page showed the ahead sensor at 0.43 m in red, the right at 0.69 m in amber and the left with no reading, and listed both readings as immediate obstacles from the time-of-flight sensors; the banner appeared over the driving page and was hidden on the Sensors page. It was deployed the same day; its use by touch has not yet been checked.

Touch response was reported as slow. The panel's USB touch controller (WaveShare WS170120, handled by the X server's libinput driver) was read directly while 25 taps were made: in every tap the panel reported contact in the same instant as the first position, so the panel adds no measurable delay. Contacts lasted 200 to 640 ms, and buttons in Qt act on release, so a long press reads as a slow response, and the visible pointer jumping to the finger adds to the impression. The processor was idle (load average 0.04), so drawing time has not been shown to matter. Acting on press in the navigation bar, hiding the pointer and timing a touch to the redraw are the next steps.

On the Raspberry Pi 5 the street map loaded, and a destination entered on 6 October was routed by the online OSRM service (3.3 km), but each new area appeared slowly, because tiles arrive two at a time

over a phone or laptop hotspot. A prefetch program (`DashboardIntegration/tools/prefetch_tiles.py`) caches the campus and the surrounding suburbs at zoom levels 11 to 16 only, since the tile policy forbids bulk downloads at zoom 17 and above, and it refuses large areas. Run on the Raspberry Pi 5 it saved 656 tiles (11.9 MB) in 320 s with none failed. Tiles at zoom 17 and above still load while driving; until they arrive the zoom 16 tile is scaled up.

To show route guidance indoors, a simulated drive was added to the map on 6 October. With a planned route and no real position in the previous 10 s, a button drives the vehicle marker along the route at 25 km/h, one position a second as the GNSS gives, in the heading-up follow view, with the part already driven drawn as done. A label reading “Simulated drive”, with the distance and time left, stays on the map the whole time, and nothing is written to the vehicle state, so speed, battery and the other pages keep showing real data or nothing; a real position stops it at once. Offscreen tests against the dashboard version installed on the Raspberry Pi 5 check each of these points. The team’s route list and arrival estimate do not yet follow the simulated position.

Voice advisories were silent. The two online speech engines the dashboard supports were not installed on the Raspberry Pi 5 and were added, but its only audio outputs are the two HDMI ports and the display offers no audio, so a speaker on USB or Bluetooth is still needed; voice output is untested. The Raspberry Pi 5 has 16 GB of memory and an AI HAT+ accelerator (Hailo-8, 26 TOPS) fitted. On 6 October the accelerator did not appear on the PCI Express bus, because the external connector is not enabled in the boot configuration. With it working, the detector could run on the Raspberry Pi 5 at the camera’s frame rate and the network link between the two computers would no longer be needed; this has not been tested.

Colour cast

In the live view the image looked reddish. Three causes were identified and a test for each written: the channel order in the detector (a blue object should appear blue), the camera itself (comparing with the operating system’s own camera preview), and a camera module without an infrared-cut filter, which gives a pink or red cast in daylight and has its own tuning file in the camera software. An option to load that tuning file was added to the detector. The test was run on 1 October 2026 (Appendix C). A still taken with the standard tuning showed a white wall as strongly pink, the detector’s own frames showed the same cast, and a still of the same scene with the tuning for a module without an infrared-cut filter showed the wall neutral, so the cast came from the module, which lacks that filter, and not from the detector’s channel order. With that tuning alone, colours were washed out: a blue card measured 120, 124 and 125 on the red, green and blue channels, almost grey, which is expected when infrared light reaches the sensor. Raising the camera’s saturation control to 1.8, through a new detector option, restored them: the blue card then measured 1, 83 and 218, a red box 213, 7 and 56, and the white wall 202, 217 and 214, close to neutral. The detector now starts with both settings. Each printed card was then held about 1 m from the camera in turn and its average colour measured in the detector’s own frames, as red, green and blue values from 0 to 255: blue 1, 86 and 216; red 222, 8 and 77; green 6, 124 and 103; grey 124, 129 and 138; and white 186, 199 and 197 (Table 3.4). The three colours are clearly identified. The grey and white cards show a small, consistent cool shift, red about 12 to 14 below blue or green, which the automatic white balance leaves in place under this indoor light; it is small enough not to matter for detection. With all four colour cards held together in one frame, so that the automatic exposure and white balance had to handle them at once, they remained distinct: blue 1, 94 and 207; red 243, 19 and 44; green 77, 145 and 89; and grey 187, 189 and 176, here slightly warm rather than cool. In daylight the infrared share is larger, so the colours should be checked again outdoors. On 6 October 2026 black objects were seen as purple in the live view: many black materials reflect infrared, which the red and blue pixels pick up, and the saturation of 1.8 strengthens the tint. A correction for dark pixels was added: each pixel’s

saturation is scaled by a weight set by its brightest channel, from fully grey when all three channels are at 50 or below to unchanged at 110 and above, so the colour cards above pass through unchanged while near-black pixels become neutral. Done in numpy it took 171 ms a frame on the Raspberry Pi 4 and 84 ms with Pillow, both with the detector running, which would have cut the detector's rate, so it is applied only to the live view, in the encoder's own thread; the detector's input is unchanged and its rate was the same before and after (6.2 and 6.3 frames per second). That rate was low because the Raspberry Pi 4 was under-voltage and throttled to 600 MHz at the time (status 0x50005), not because of the correction. An infrared-cut filter, or a standard camera module in place of the NoIR one, would remove the cause. The under-voltage itself was traced on 6 October. The Raspberry Pi 4 stopped booting that afternoon and recovered after everything was disconnected. A logger then recorded its power and temperature flags every 2 s. With nothing attached the flags were clear and the processor ran at its full 1.8 GHz. Fourteen seconds after the telemetry unit was plugged into its USB port, when the unit's LTE modem came up, under-voltage began and then came and went every few seconds, with the processor throttled to 600 MHz and the detector slowed from 14.3 to 12 frames per second; the distance sensors were not connected during this test. The modem's transmit current, drawn through the Raspberry Pi's USB port, is therefore the cause of the under-voltage seen all week, and probably of the failure to boot. The fix is to power the telemetry unit separately, for example through a powered USB hub, which applies equally to the Raspberry Pi 5. The same log showed the Raspberry Pi 4 reaching its soft temperature limit at 65 to 69 °C with the detector running and no heatsink, so it also needs cooling.

Table 3.4: Colour cards in the detector's frames with the NoIR tuning and saturation 1.8 (mean red, green and blue, 0 to 255; 1 October 2026, indoor light).

Card	R	G	B
Blue	1	86	216
Red	222	8	77
Green	6	124	103
Grey	124	129	138
White	186	199	197

Because the model was trained on normal-colour images, the cast may also be lowering detection confidence, which the same tests will show by comparing confidence before and after.

A first road-hazard model

Because the general COCO classes include no road-surface hazards, a first pothole detector was trained on 2 October 2026 (CameraDetection/training/). YOLO11n, pretrained on COCO, was fine-tuned in Google Colab on a free GPU on a public 608-image pothole dataset from Roboflow Universe, with no images from this vehicle's camera. Training ran 46 epochs in 15 minutes and stopped by itself 15 epochs after the best validation result. On the dataset's held-out test images it reached a mAP@50 of 59.7% at 640×640 input (precision 66%, recall 50%), and 55.0% at 320×320, the smaller input that needs about a quarter of the computation on the Raspberry Pi. The validation loss stopped improving after about 20 epochs while the training loss kept falling, so more varied images, rather than longer training, are needed for a better model. The test images come from the same source as the training images, so these figures describe the model on that dataset, not on South African roads; testing on a South African pothole dataset (Nienaber, Kroon and Booysen, Stellenbosch University) and on the Raspberry Pi, for speed, is the next step. The model has not yet been run on this vehicle's camera. A first timing of a YOLO model on the Raspberry Pi 4 was made on 2 October 2026. Installing the PyTorch library that the Ultralytics software uses failed over the available network, so the exported model was run with the NCNN library alone, with the letterboxing, box decoding and non-maximum suppression written for

this purpose (`CameraDetection/ncnn_bench.py`), which is also the lighter form a deployment would take. In a 15 s trial with the COCO-trained YOLO11n at a 320-pixel input and four threads, it ran at 7.9 frames per second with a median of 111 ms for inference and 175 ms from sensor readout to result (95th percentile 258 ms), against 19.5 frames per second and 102 ms for the SSD-MobileNet detector at its 300-pixel input. Full one-minute runs, including the polygon-producing segmentation model and the 640-pixel input, are still to be made. The South African dataset, filmed from a vehicle-mounted camera, was converted to the same format: of its 4,337 images of 3680×2760 pixels, 1,987 contain 5,029 labelled potholes. Its potholes are small in the frame: the median is 100 by 26 pixels (10th to 90th percentile 46 to 220 by 12 to 66), which shrinks to about 17 by 4.5 pixels when the whole frame is reduced to the model’s 640-pixel input, close to the smallest objects such a detector can find. Their centres all lie between 48 and 62% of the image height, in the band of road a few tens of metres ahead. Cropping that band and passing it to the model at a higher resolution, in two or three tiles, would make the potholes about three times larger, at the cost of two or three inferences per frame; the same option exists on this vehicle, whose camera sensor has 2592×1944 pixels but is read at 640×480 for speed. The first model was then tested, without retraining, on the South African dataset’s 605 test images, which contain 1,238 potholes. Its mAP@50 was 0.000 at input sizes of 640, 1280 and 1920 pixels, with a recall of at most 1.2%. To rule out a fault in the label conversion, the labels and the model’s detections were drawn on the test image with the most potholes: the converted boxes lie on the potholes, while boxes read as centred coordinates would be visibly offset, so the conversion is correct. On that image, with 12 labelled potholes, the model made one detection, at a confidence of 0.07. A model that reaches 59.7% on test images from its own source therefore finds almost none of the potholes in South African street footage taken through a windscreen, which is close to this vehicle’s view. Training on local images, at a resolution at which the potholes remain visible, is therefore necessary rather than optional. For a second run, prepared on 6 October 2026, the notebook crops each South African frame to the band from 42 to 68% of its height and cuts it into three overlapping tiles 1280 pixels wide, keeping a pothole in a tile when at least half of it lies inside; at the 640-pixel training input a median pothole then measures about 50 by 13 pixels instead of 17 by 4.5. The model is fine-tuned on these tiles, starting from the first run’s weights when they are available. The run was made on 6 October 2026 on a Colab T4 graphics processor and used all 60 epochs; the validation mAP@50 rose from 0.463 after the first epoch to a best of 0.719 at epoch 54. Scored on the validation tiles, the best weights reached a mAP@50 of 0.714 at a 640-pixel input (precision 81%, recall 62%) and 0.566 at 320 pixels (precision 69%, recall 51%), against 0.000 for the first model on the same country’s images. These figures are on the validation split, which training used to choose the best epoch, so they are optimistic, and they are per tile, not per full frame; the held-out test tiles from the dataset’s separate test frames are still to be scored, and the notebook’s record did not state which weights training started from (both corrected in the notebook on 6 October). The model has not yet been run on the Raspberry Pi or on this vehicle’s camera.

A subsequent literature check specifically for camera-based hazard-awareness systems on three-wheeled or auto-rickshaw-class vehicles found no directly comparable academic or open-source system; the closest commercial precedent (a Tier-1 automotive supplier case study describing telematics and HUD products for auto-rickshaws) does not appear to combine camera-based detection with the vehicle-dynamics telemetry described elsewhere in this chapter. This is treated as evidence that the specific integration attempted here is not already a solved problem, rather than as proof of the individual component technologies (pretrained object detection, monocular distance estimation) being novel in themselves, since each of those is independently well preceded in the literature for other vehicle classes.

3.20.8 Camera and time-of-flight fusion

The camera and the time-of-flight sensors are complementary. The camera says what an object is and in which direction, but its distance comes from the uncalibrated width heuristic (Section 3.20.7); each time-of-flight sensor measures distance accurately, but only close by, in one fixed direction, and without saying what it sees. On 4 October 2026 a fusion step was added to the bridge on the Raspberry Pi 4 (`CameraDetection/sensor_fusion.py`, geometry in `CameraDetection/fusion_config.json`). The detector now also serves its latest detections with a bearing for each, computed from the box centre and the camera's horizontal field of view (53.5°), and the bridge polls them five times a second. The rules are:

1. A detection is matched to a time-of-flight sensor when its bearing lies inside that sensor's cone, widened by a 5° gate, the sensor's sample nearest in time to the camera frame is within 200 ms of it, and that sample is valid and within the sensor's range. If several sensors qualify the nearest reading wins, and each sensor is given to at most one detection.
2. A matched detection becomes a "fused" hazard: the camera's class with the time-of-flight distance. An unmatched detection keeps the camera's estimate ("camera"). Both distances are kept for evaluation, and the distance band is recomputed from the distance used.
3. A sensor not matched to any detection whose last three readings are all valid and in range reports an "obstacle" from the time-of-flight sensor alone, the same three-sample debounce as the detector's onset. This covers objects outside the detector's classes.
4. Camera data older than 1 s is ignored, so with the detector stopped only time-of-flight obstacles are reported.

Hazards are sorted nearest first and served in the telemetry record and at a separate address. Since 6 October the dashboard's Sensors page lists them and its hazard banner shows the immediate ones (Section 3.20.7); the driving page's front-obstacle distance still comes from the forward sensor alone, as decided on 2 October.

The sensor directions in the configuration are defaults, not measurements: left, ahead and right at $-35, 0$ and $+35^\circ$, each with a 12.5° half-angle and a 1.2 m range. They are to be calibrated on the bench before any result is claimed. With these defaults the side cones cover -47.5 to -22.5° and 22.5 to 47.5° , while the camera sees only $\pm 26.75^\circ$, so the side sensors overlap the camera's view only at its edges (about the outer 9° of each side, gate included). Most objects to the side can therefore only be reported by the time-of-flight sensor alone, without a class (Figure 3.7).

Other stated limitations: a time-of-flight sensor reports the nearest surface in its cone, which need not be the object the camera saw at that bearing, and detections are matched frame by frame without tracking.

By design, the bridge does not keep a log of fused hazards unless one is asked for. The detector's normal log holds five fields per alert with a coarse distance band, and a continuous record of distances and bearings would collect more than the system needs, so the fusion log is off by default and is switched on only for a validation session; even then it holds numbers only, no images.

Status: 29 offline tests pass (4 October 2026), covering bearing, matching inside and outside a cone, readings out of range, stale timestamps, two objects, the obstacle debounce, band recomputation, the

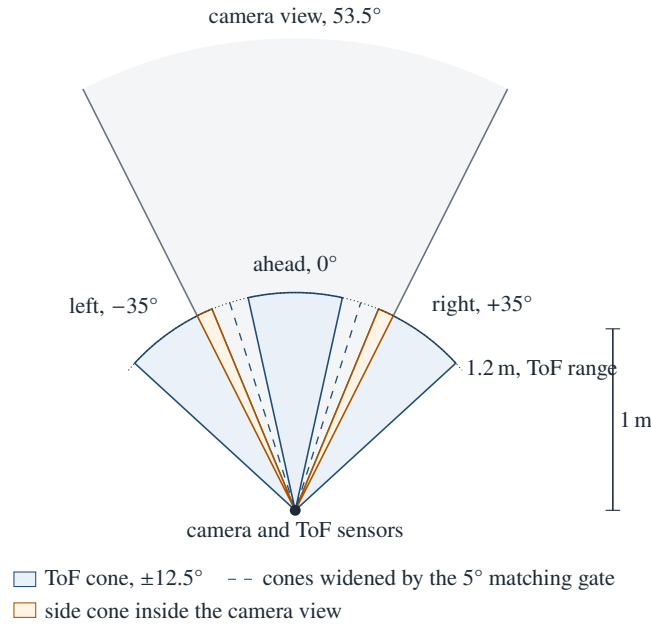


Figure 3.7: Camera and time-of-flight geometry used by the fusion, top view to scale, forward up. The values are the configuration defaults, not yet calibrated. A detection is matched to a sensor when its bearing lies within that sensor's cone widened by the 5° gate.

detections feed and the bridge against a stand-in detector. On 4 October 2026, before any calibration, the fusion was started on the Raspberry Pi 4 with the live detector and sensors: the detector's detections feed answered at 15.8 frames per second, and with no person in view the fused list showed only distance-sensor obstacles from objects on the bench, for example one 0.125 m ahead. A bench validation (test C9 in `BenchTest/TEST_PROCEDURES.md`) is written but has not yet been run: it first calibrates the camera axis, the field of view and each sensor's cone with a narrow target on a marked floor, then checks a person walking across at 1 m, a person standing at 0.5, 1.0 and 1.5 m against a tape measure (fused distance error 5 cm or less), a box outside the camera's classes at 0.8 m (reported as a time-of-flight obstacle), and a person at 2.5 m, beyond the sensors' range (camera only). On 6 October 2026, still before calibration, the first fused hazard was seen in the live record with a person close in front of the camera: class person, ahead sensor, camera estimate 0.874 m, time-of-flight 0.096 m, reported distance 0.096 m, immediate band, while the right sensor gave a time-of-flight-only obstacle at 0.17 m. This shows the matching working on live data, and how far the uncalibrated camera distance can be wrong at close range; the distances were not checked against a tape, so it is not a validation, and no validated result is recorded yet. Later on 6 October all three distances were null. The I²C bus showed only the left sensor, at its new address, and nothing answered when the ahead sensor was woken; the start-up gave every sensor its address in turn and stopped at the first that did not answer, so one loose connection removed all three. The reader now skips a sensor that does not answer, logs which one, and keeps the others running, and the bridge looks for a missing sensor again every 60 s, so a reseated one returns without a restart. The bridge tests pass offline; the change had not yet been run on the Raspberry Pi 4, which was off the network at the time.

3.21 Cellular telemetry publishing

A networking task, running inside the existing modem-handling task rather than as a separate FreeRTOS task, publishes a compact JSON summary of the latest ride-characterisation, BMS, and GNSS state over

MQTT via the A7670X modem's native AT command set (the same AT+CMQTT* family used by other SIMCom-class modems), rather than requiring a TCP/IP socket library on the microcontroller itself. This was deliberately kept inside the existing modem task rather than given its own task: the modem's UART is not protected by any locking mechanism, and a second task issuing AT commands on the same UART concurrently with the existing GNSS polling would interleave and corrupt both command streams.

The broker address and credentials are left as placeholders in the firmware rather than pointed at the existing Tukzie vac-work team's own Mosquitto broker, whose credentials were deliberately not copied into this project's code. Tested against this placeholder broker, the connect sequence behaved exactly as intended: the AT+CMQTTCONNECT command failed cleanly (confirmed via live serial capture, showing the modem's own error response rather than a silent failure), the failure was logged clearly, and the rest of the system continued operating normally afterward with no crash or hang. This confirmed the AT command sequence itself was implemented correctly, independent of network conditions.

3.21.1 Local logging to internal flash

Telemetry is also recorded locally, so that a session leaves a complete record even with no cellular connection at all. The board's microSD slot was the obvious candidate, but its chip-select line reaches the processor only through a resistor (R23) that the schematic marks as not populated, so there is likely no electrical path to the card as the board is assembled. Rather than build on an interface that may not physically exist, logging uses a LittleFS file system on the processor's own 16 MB flash, in the 3.5 MB data partition of the 16 MB partition table.

Every ten seconds, on the same cadence as MQTT publishing, the complete telemetry record is appended to the log as one JSON line, whether or not the MQTT connection is up; logging is deliberately not conditional on connectivity. The file is capped at 3 MB, beyond which it is truncated. This first version has two stated limits: truncation discards the oldest data rather than rotating it into a second file, and the log does not yet track which lines have been published, so the unsent backlog is not replayed when the connection returns. The partition mounts correctly on the target board, and its used space grew from 12 KB to 56 KB across later boots, consistent with records being written. The log was first read back on 1 October 2026, through a console command that prints every stored record with its index and length, so that a laptop script can discard any record damaged by other output on the same serial link and repeat the transfer until every index has arrived intact. The first attempts, without pacing or the index, lost records and characters in transfer; with both, four passes recovered all 6,020 records (2.2 MB, about 16.7 hours of operation at one record per 10 s, accumulated across sessions since logging was added, since the file survives reprogramming). The read-back exposed a fault: 5,545 of the records were not valid JSON, because fields with no value, such as the position before a GNSS fix, are NaN and were printed as nan, which JSON does not allow. The same text was being published over MQTT. Firmware v0.6.3 writes these fields as null; with that substitution all 6,020 stored records parse. The records also preserve two earlier events: the first outdoor GNSS fix as the old parser reported it (0.233 and 0.15 degrees, Section 3.13.3), and the only complete BMS readings so far (below). The copy was saved in the repository and the file on the board was then cleared, so that the road-test record starts empty.

Enabling the log exposed a side effect on acquisition timing. In the recorded bench session of Section 3.15.4, the longest interval between consecutive samples rose to between 25 and 30 ms, against a nominal 5 ms, in one two-second window out of every six, on both IMUs at the same moment, while every other window stayed below 8 ms. The stalls began about 30 seconds after boot, once the modem's start-up sequence had finished, and then recurred every 12 seconds. Because each GNSS poll currently

waits about 3 seconds to time out, the ten-second logging step in the modem loop actually runs every 12 seconds, so the stalls coincide with the flash writes. Writing to flash on the ESP32 briefly suspends code executing from flash on both cores, which would explain both sensors stalling together, although the two are on separate I²C buses. No samples were lost, since the queues absorbed each stall, but the samples around it are unevenly spaced. The link was confirmed on 29 September 2026 with firmware v0.5.0, whose serial console can switch local logging off and on (Section 3.21). With the board still, logging was left on for 120 s and then switched off for 120 s, with MQTT publishing every 10 s throughout. With logging on, both IMUs showed a gap of 33 to 40 ms every 10 s, twelve events in the two minutes; with logging off, the longest gap in any window was 5.9 ms and none exceeded 10 ms. Both sensors stayed at 200.0 to 200.5 Hz with no dropped samples in either half. Because publishing continued while logging was off, the stalls come from the flash writes and not from the cellular publish. The stalls now recur every 10 s rather than 12 s because the GNSS polls no longer time out in this firmware, so the logging step runs on its nominal cadence (Appendix C). Candidate remedies were buffering log records in memory and writing them less often, or placing the time-critical acquisition code in instruction RAM. Firmware v0.6.0 implements the first: records are collected in memory and written to flash in a single append about once a minute, and the file size is kept in memory so that no extra file operations are needed. In a 330 s bench run on 1 October 2026 with logging on throughout, there were five flushes, and every acquisition gap over 10 ms (ten windows, both IMUs at each flush, 37 to 40 ms) fell within 3.5 s after one of them; the worst gap in every other window was 5.9 ms, the same as with logging switched off, and both IMUs stayed at 200.0 to 200.5 Hz with no dropped samples. The pause itself is no shorter, since it comes from the flash operation, but it now occurs about six times less often. The cost is that up to a minute of records is lost if power is cut before a flush, so the buffer is also written out before the recalibration restart. Each flush occupied the modem task for 71 to 202 ms, which does not affect acquisition. Removing the pause entirely would still require the instruction-RAM approach, or the sensors' own hardware FIFO buffers to absorb it. The same run showed that the log file had reached 2.2 MB of its 3 MB cap, so at about 160 KB per hour it would be truncated, and its contents lost, after about five more hours of running; it should be read back and cleared before the road test.

3.21.2 End-to-end validation against a real broker

With the full-system integration described in Section 3.19, the modem was observed reaching genuine LTE registration: `AT+CPSI?` returned `LTE,Online` against a real mobile network operator's PLMN code, and `AT+CREG?/AT+CGREG?` both reported registered status. This registration was observed to be intermittent across repeated boot sequences in this indoor bench-testing location, in at least one case dropping mid-sequence (`+CGEV: NW PDN DEACT`) immediately before the MQTT connect attempt, which is reported honestly here as bench-test behaviour rather than smoothed over, since it is directly relevant to how reliably this subsystem could be expected to behave once mounted on the vehicle and moving through areas of varying signal strength.

With the broker address changed from the placeholder to `test.mosquitto.org` (a public, unauthenticated broker used here only to validate the communication path end-to-end, not as a destination for real vehicle telemetry) and registration present at the time of the attempt, the full sequence succeeded: the firmware logged `[MQTT] Connected.`, and the subsequent publish returned `+CMQTTPUB: 0,0`, indicating a successfully acknowledged publish. This is the first point at which a complete, real path (SIM readiness, cellular network registration, PDN context activation, MQTT connection, and message publish) has been exercised end-to-end on real hardware, rather than validated only up to a placeholder failure. The intermittent registration observed above means this should be read as confirmation the mechanism works correctly when network conditions allow it, not as a claim of reliable connectivity under all conditions; the latter would require repeated testing under the vehicle's actual operating conditions, which remains

outstanding.

3.21.3 Result-code checking and automatic reconnection (firmware v0.5.0)

A closer reading of the modem manual found a latent fault in the connection code. For the commands that start the MQTT service, connect to the broker and publish, the manual gives both success and failure as an OK followed by a separate result line carrying an error code, where zero means success [59]. The firmware had treated the first OK as success, so it could report a connection that had in fact failed. The acknowledged publish reported above is not affected, since it was confirmed by its own result line (+CMQTT PUB: 0,0). In firmware v0.5.0 these commands wait for their result line and check its code; for the start command, an error response is accepted as meaning the service was already running, as the manual states.

The bench log also showed that after a failed first connection the firmware never tried again. Version 0.5.0 retries with a delay that starts at 30 s and doubles to at most five minutes, resetting after a success. A session is treated as lost after three failed publishes in a row, or when the modem reports that the server dropped the client or that the network was lost, which the manual says requires the service to be restarted. Before each retry the old session is shut down in the order the manual requires: disconnect, release the client, then stop the service.

Version 0.5.0 also adds commands on the USB serial console for bench and vehicle tests. A line starting with an exclamation mark is handled by the telemetry unit; everything else is still passed straight to the modem. The commands report the state of logging, MQTT and the calibration factors; switch local flash logging off and on, to test whether the periodic acquisition stalls described in Section 3.21.1 come from the flash writes; request an immediate MQTT reconnection; and erase the stored IMU calibration and restart, so that the sensors can be recalibrated in their mounted position without erasing the whole flash. Version 0.5.0 was loaded onto the target board on 29 September 2026 and tested on the bench with a scripted session (BenchTest/v050_bench_test.py): the status, logging and reconnect commands all responded; MQTT connected on the first attempt and then published 47 times with no failures; and a requested reconnect shut the session down in the manual's order and reconnected in 0.4 s. Recovery from a loss of signal was tested on 30 September 2026 by switching the modem's radio off with AT+CFUN=4 for 60 s and back on with AT+CFUN=1 (BenchTest/signal_loss_test.py). The firmware detected the loss either from the modem's own report, 0.8 s after the radio went off, or after three failed publishes, 20.7 s after; it then retried with a delay of 30 and then 60 s, as designed. In v0.5.0 the reconnect after the radio returned waited for the next scheduled retry and took 36 s, although the network data connection was back within 2.4 s. Firmware v0.5.3 therefore also watches for the modem's report that the data connection is active and reconnects within a few seconds of it; in the repeated test it reconnected about 5 s after the radio came back. In both runs publishing resumed with no failed publishes afterwards. At the start of the repeated test the status showed a live session after two connection attempts, so a failed first connection after boot had been retried successfully; that boot itself was not captured. The recalibration command has not yet been used. When several tasks print at once their serial lines can interleave, which occasionally splits a status line in the log; this affects the readability of logs but not the firmware's behaviour.

Chapter 4

Results

This chapter presents the results by subsystem, followed by a full-system test on the vehicle, and maps each back to the research questions and requirements in Table 1.1. Bench-level validation results are reported alongside their methods in Chapter 3.

4.1 Inertial acquisition integrity (RQ3)

Achieved sample rate, inter-sample jitter distribution, and dropped-sample counts for both IMUs, first on the bench and then on the moving vehicle.

4.2 Sensor calibration

Per-sensor magnitude calibration factors and the resulting agreement between the two IMUs, at rest and under dynamic excitation.

4.3 Ride characterisation (RQ1, RQ4)

Time-domain features (RMS, peak-to-peak, crest factor, jerk, threshold crossings) and the speed-normalised roughness index recorded over known road sections, compared against identifiable road and powertrain events.

4.4 Battery-management telemetry

BLE connection reliability over a sustained session, and a measured discharge curve compared against the pack's rated capacity.

4.5 Camera, synchronisation and proximity sensing

Camera-to-IMU time alignment, object-detection alert events and the fields recorded for them, and time-of-flight distance measurements, including behaviour against light and dark targets.

4.6 Cellular synchronisation and data integrity (RQ3)

Publish success rate, latency, and recovery behaviour after induced or naturally occurring loss of coverage, with local-log completeness as the reference.

4.7 Driver display (RQ2)

Information hierarchy of the dashboard page as implemented, alert update latency to the screen, legibility under representative lighting, and observed glance time.

4.8 Full-system test

All subsystems operating together on the vehicle over a defined route.

Chapter 5

Discussion

This chapter will interpret the results against the literature reviewed in Chapter 2 and the research questions in Chapter 1. Planned themes are: how repeatably low-cost inertial sensors characterise this vehicle's dynamic response, and how strongly that depends on mounting, speed and calibration; whether the chosen time-domain features separate road-induced from powertrain vibration in practice; how the embedded architecture's latency, jitter and data-recovery behaviour compare with what edge-telemetry studies typically report; how the dashboard page's minimal information hierarchy holds up against the complexity and glance-time concerns identified in the literature; and the practical limits of camera-based distance estimation relative to the time-of-flight measurement.

Chapter 6

Conclusions

This chapter will return to each research question in Chapter 1 in turn and state what the evidence supports, with reference to the requirements and verification status in Table 1.1, followed by an overall statement of the contribution and its limitations.

Chapter 7

Recommendations

7.1 Generic road-obstacle detection

The camera-based detector described in Chapter 3 is restricted to a predefined set of trained object categories (motor vehicles, motorcycles, bicycles, pedestrians, animals). It cannot, and does not claim to, recognise an arbitrary unexpected object, such as fallen debris, that was not one of its training classes; a class like “stop sign” would not fire on a twig lying in the road, since the model is a closed-set classifier over its trained categories, not a generic obstacle detector.

Generic detection of this kind is an active, separate research area, referred to in the literature as road-obstacle or road-anomaly detection, distinct from closed-set object detection. HazardNet addresses the scarcity of real hazard training data by augmenting real road images with synthetic 3D debris models under domain randomisation, allowing detection of debris types not explicitly present in training [66]. The SegmentMeIfYouCan benchmark provides datasets and an evaluation protocol specifically for anomalous-object and road-obstacle segmentation, that is, identifying objects that do not belong to the expected road scene at all, rather than classifying among a fixed list [69]. Open-vocabulary detectors such as Grounding DINO, which pair a Transformer-based detector with language-grounded pre-training to detect objects described by an arbitrary text prompt rather than a fixed class list [67], offer an alternative route, but as large Transformer-based models they are unlikely to run in real time on a Raspberry Pi 4 without a hardware accelerator, which would need to be confirmed by measurement. A more Pi-appropriate direction is a lightweight, custom-trained detector: a 2026 study demonstrates a YOLOv8-n model trained specifically on rare roadside obstacles (cones, debris, barrels, fallen trees, rocks), reporting 98.1% mAP@0.5 at 68 fps on GPU hardware [68], though this would need re-validation for real-time performance on the Pi 4’s more limited compute.

None of these approaches is recommended for adoption within the current project. Each would replace the detection model already implemented, and would require its own dataset, training, and on-hardware evaluation before it could be trusted. Generic road-obstacle detection is therefore recorded here as a concrete, evidenced direction for future work, rather than attempted within the scope or timeline of this project.

Were this pursued in a subsequent project phase, a sensible order of work would be: define the operational meaning of “unexpected obstacle” for this vehicle and use case; collect and annotate representative

road-hazard imagery, or evaluate an existing benchmark such as RoadObstacle21; compare a custom lightweight detector against an anomaly-segmentation approach on that data; test detection accuracy across daylight, night, rain, shadow, and glare conditions, since the present system has only been characterised under bench lighting; measure inference speed and power draw on the actual Raspberry Pi 4 rather than the GPU hardware the cited studies were evaluated on; calibrate any resulting distance estimate against the VL53L5CX time-of-flight sensor described in Section 3.20.7 as an independent reference, rather than trusting a vision-only estimate alone; and only then begin live-camera testing of the new model.

Bibliography

- [1] World Health Organization, *Global Status Report on Road Safety 2023: Summary*. Geneva, Switzerland: World Health Organization, 2023.
- [2] United Nations, Department of Economic and Social Affairs, Population Division, *World Urbanization Prospects: The 2018 Revision*. New York, NY, USA: United Nations, 2019.
- [3] O. S. Sebwa, P. V. Chombo, G. B. Gerutu, R. O. Kivugo, J. H. Hosea, and K. A. Greyson, “Energy, economy and environmental assessment of e-tricycle for medium-size cargo deliveries: A case of Dar es Salaam City,” *The Journal of Engineering*, vol. 2025, no. 1, Art. no. e70073, 2025, doi: 10.1049/tje2.70073.
- [4] J. Saenz, M. Figliozzi, and J. Faulin, “Assessment of the carbon footprint reductions of tricycle logistics services,” *Transportation Research Record*, vol. 2570, no. 1, pp. 48–56, 2016, doi: 10.3141/2570-06.
- [5] Y.-C. Liu, “Effects of using head-up display in automobile context on attention demand and driving performance,” *Displays*, vol. 24, nos. 4–5, pp. 157–165, 2003, doi: 10.1016/j.displa.2004.01.001.
- [6] National Highway Traffic Safety Administration, *Visual-Manual NHTSA Driver Distraction Guidelines for In-Vehicle Electronic Devices*, Federal Register, vol. 78, no. 81, pp. 24818–24890, Apr. 2013.
- [7] W. M. Kimmel, C. D. Weiss, and A. M. Mazzuchi, *Technology Readiness Assessment Best Practices Guide*, NASA/SP-20205003605, 2020.
- [8] R. Häuslschmid, S. Osterwald, M. Lang, and A. Butz, “Augmenting the driver’s view with peripheral information on a windshield display,” in *Proc. 20th Int. Conf. Intelligent User Interfaces*, 2015, doi: 10.1145/2678025.2701393.
- [9] M. Barnell, C. Raymond, C. Capraro, D. Isereau, C. Cicotta, and N. Stokes, “High-performance computing (HPC) and machine learning demonstrated in flight using Agile Condor,” in *Proc. IEEE High Performance Extreme Computing Conf.*, 2018, pp. 1–4, doi: 10.1109/HPEC.2018.8547797.
- [10] A. Mareška, T. Kordová, and M. Havlík Míka, “Production of Head-Up Display windshield and its relation with the image quality,” *Transactions on Transport Sciences*, vol. 13, no. 2, pp. 63–70, 2022, doi: 10.5507/tots.2022.011.
- [11] E. Caño-Marín, P. E. González Prieto, M. Maroto Gómez, and D. Villegas, “Head-up displays in driving,” 2016, arXiv:1803.08383.
- [12] Y. I. Alatoom and O. Smadi, “Embedded framework for low-cost pavement condition evaluation using microcontroller and single-board computer platforms,” *Automation in Construction*, vol. 178, Art. no. 106442, 2025, doi: 10.1016/j.autcon.2025.106442.

- [13] Federal Highway Administration, *Measuring and Specifying Pavement Smoothness*, Tech. Brief FHWA-HIF-16-032, Washington, DC, USA, 2016.
- [14] A. González, E. J. O'Brien, Y.-Y. Li, and K. Cashell, "The use of vehicle acceleration measurements to estimate road roughness," *Vehicle System Dynamics*, vol. 46, no. 6, pp. 483–499, 2008, doi: 10.1080/00423110701485050.
- [15] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," *Sensors*, vol. 18, no. 11, Art. no. 3845, 2018, doi: 10.3390/s18113845.
- [16] International Organization for Standardization, *Mechanical Vibration and Shock: Evaluation of Human Exposure to Whole-Body Vibration, Part 1: General Requirements*, ISO 2631-1:1997, including Amendment 1:2010.
- [17] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, doi: 10.1109/IIOT.2016.2579198.
- [18] Y. Zhu, Y. Jing, M. Jiang, Z. Zhang, D. Wang, and W. Liu, "An experimental study of the cognitive load of in-vehicle multiscreen connected HUD," in *Design, User Experience, and Usability: Design for Contemporary Technological Environments*, Lecture Notes in Computer Science, vol. 12781. Cham, Switzerland: Springer, 2021, pp. 268–285, doi: 10.1007/978-3-030-78227-6_20.
- [19] M. Smith, J. L. Gabbard, and C. Conley, "Head-up vs. head-down displays: Examining traditional methods of display assessment while driving," in *Proc. 8th Int. Conf. Automotive User Interfaces and Interactive Vehicular Applications*, 2016, pp. 185–192, doi: 10.1145/3003715.3005419.
- [20] R. Currano, S. Y. Park, D. J. Moore, K. Lyons, and D. Sirkin, "Little Road Driving HUD: Heads-Up Display complexity influences drivers' perceptions of automated vehicles," in *Proc. 2021 CHI Conf. Human Factors in Computing Systems*, 2021, pp. 1–15, doi: 10.1145/3411764.3445575.
- [21] Jiangsu Jinpeng Group Co., Ltd, *Electric Tricycle User Manual*. Vehicle documentation, TUKZIE Rev 0 platform.
- [22] S. Winberg and S. Jayalath, "Development of an Embedded HPEC Telemetry Unit and Windshield HUD for the TUKZIE Rev 0 Platform," EEE4022S project brief, University of Cape Town, Department of Electrical Engineering, 2026.
- [23] V.-H. Bui, D.-N. Tran, P. Q. Huy, N. V. Cong, and T.-H. Dao, "Signal processing-driven real-time rough road detection with GPS accuracy improvement on edge microcontrollers," in *Proc. IEEE 15th Int. Conf. Communication Systems and Network Technologies*, 2026, doi: 10.1109/CSNT69054.2026.11502079.
- [24] B. Yiliguogqi, B. Siriguleng, Arong, G. Guo, and J. Wang, "Research on the evaluation and analysis of road surface roughness based on smartphone sensors and SVM," *Scientific Reports*, vol. 16, Art. no. 4409, 2026, doi: 10.1038/s41598-025-34396-3.
- [25] A. Pérez-González, A. F. Villa-Salazar, I. N. Gomez-Miranda, J. D. Velásquez-Gómez, A. F. Romero-Maya, and A. Jaramillo-Duque, "An IoV-based real-time telemetry and monitoring system for electric racing vehicles: Design, implementation, and field validation," *Vehicles*, vol. 7, no. 4, Art. no. 128, 2025, doi: 10.3390/vehicles7040128.
- [26] K. Park and Y. Im, "Ergonomic guidelines of Head-Up Display user interface during semi-automated driving," *Electronics*, vol. 9, no. 4, Art. no. 611, 2020, doi: 10.3390/electronics9040611.
- [27] Z. Qin, F.-C. Lin, Y.-P. Huang, and H.-P. D. Shieh, "Maximal acceptable ghost images for designing a legible windshield-type vehicle Head-Up Display," *IEEE Photonics Journal*, vol. 9, no. 6, pp. 1–12, 2017, doi: 10.1109/JPHOT.2017.2758820.

- [28] R. Häußlschmid, B. Pfleging, F. Alt, and A. Butz, “A design space to support the development of windshield applications for the car,” in *Proc. 2016 CHI Conf. Human Factors in Computing Systems*, 2016, pp. 5076–5091, doi: 10.1145/2858036.2858336.
- [29] L. M. Pinho, E. Quiñones, M. Bertogna, A. Marongiu, V. Nélis, P. Gai, and J. Sancho, Eds., *High-Performance and Time-Predictable Embedded Computing*. Gistrup, Denmark: River Publishers, 2018, ISBN: 978-87-93609-69-3.
- [30] A. Munir, S. Ranka, and A. Gordon-Ross, “High-performance energy-efficient multicore embedded computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 4, pp. 684–700, 2012, doi: 10.1109/TPDS.2011.214.
- [31] G. Lentaris, K. Maragos, I. Stratakos, L. Papadopoulos, O. Papanikolaou, D. Soudris, M. Lourakis, X. Zabulis, D. Gonzalez-Arjona, and G. Furano, “High-performance embedded computing in space: Evaluation of platforms for vision-based navigation,” *Journal of Aerospace Information Systems*, vol. 15, no. 4, pp. 178–192, 2018, doi: 10.2514/1.1010555.
- [32] Y. Zou, A. Awad, and M. Lin, “DirectNVM: Hardware-accelerated NVMe SSDs for high-performance embedded computing,” *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 1, Art. no. 9, pp. 1–24, 2022, doi: 10.1145/3463911.
- [33] G. Hendry, E. Robinson, V. Gleyzer, J. Chan, L. P. Carloni, N. Bliss, and K. Bergman, “Circuit-switched memory access in photonic interconnection networks for high-performance embedded computing,” in *Proc. 2010 ACM/IEEE Int. Conf. High Performance Computing, Networking, Storage and Analysis*, New Orleans, LA, USA, 2010, pp. 1–12, doi: 10.1109/SC.2010.42.
- [34] C. Petrie, M. Devlin, and K. Regester, “High performance embedded computing using field programmable gate arrays,” Nallatech, Eldersburg, MD, USA, Tech. White Paper, 2004.
- [35] W. Brewer, M. Maiterth, V. Kumar, R. Wojda, S. Bouknight, J. Hines, W. Shin, S. Greenwood, D. Grant, W. Williams, and F. Wang, “A digital twin framework for liquid-cooled supercomputers as demonstrated at exascale,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, doi: 10.1109/SC41406.2024.00029.
- [36] F. Samie, L. Bauer, and J. Henkel, “IoT technologies for embedded computing: A survey,” in *Proc. 2016 Int. Conf. Hardware/Software Codesign and System Synthesis*, Pittsburgh, PA, USA, 2016, Art. no. 8, pp. 1–10, doi: 10.1145/2968456.2974004.
- [37] T. Sipola, J. Alatalo, T. Kokkonen, and M. Rantonen, “Artificial intelligence in the IoT era: A review of edge AI hardware and software,” in *Proc. 31st Conf. Open Innovations Association*, Helsinki, Finland, 2022, pp. 321–331, ISSN: 2305-7254.
- [38] T. K. Chowdhury and S. Ashfaq, “High-performance computing architectures to strengthen cloud infrastructure security,” *American Journal of Interdisciplinary Studies*, vol. 5, no. 3, pp. 1–42, 2024, doi: 10.63125/9hr8qk06.
- [39] M. De Vincenzi, G. Costantino, I. Matteucci, F. Fenzl, C. Plappert, R. Rieke, and D. Zelle, “A systematic review on security attacks and countermeasures in automotive Ethernet,” *ACM Computing Surveys*, vol. 56, no. 6, Art. no. 135, pp. 1–38, 2024, doi: 10.1145/3637059.
- [40] N. Navet, Y. Song, F. Simonot-Lion, and C. Wilwert, “Trends in automotive communication systems,” *Proceedings of the IEEE*, vol. 93, no. 6, pp. 1204–1223, 2005, doi: 10.1109/JPROC.2005.849725.
- [41] J. Huang, M. Zhao, Y. Zhou, and C.-C. Xing, “In-vehicle networking: Protocols, challenges, and solutions,” *IEEE Network*, vol. 33, no. 1, pp. 92–98, 2019, doi: 10.1109/MNET.2018.1700448.

- [42] A. Ben Chehida Douss, R. Abassi, and D. Sauveron, “State-of-the-art survey of in-vehicle protocols and automotive Ethernet security and vulnerabilities,” *Mathematical Biosciences and Engineering*, vol. 20, no. 9, pp. 17057–17095, 2023, doi: 10.3934/mbe.2023761.
- [43] S. Seifert and R. Obermaisser, “Secure automotive gateway: Secure communication for future cars,” in *Proc. 12th IEEE Int. Conf. Industrial Informatics*, Porto Alegre, Brazil, 2014, pp. 213–220, doi: 10.1109/INDIN.2014.6945510.
- [44] Y.-C. Liu and M.-H. Wen, “Comparison of Head-Up Display (HUD) vs. Head-Down Display (HDD): Driving performance of commercial vehicle operators in Taiwan,” *International Journal of Human-Computer Studies*, vol. 61, no. 5, pp. 679–697, 2004, doi: 10.1016/j.ijhcs.2004.06.002.
- [45] J. Li, C. Wang, and M. Chen, “Effects of driving background complexity and interface opacity on visual cognition in AR-HUD systems,” *Journal of the Society for Information Display*, vol. 33, no. 8, pp. 919–936, 2025, doi: 10.1002/jsid.2096.
- [46] R. E. Bailey, J. J. Arthur III, S. P. Williams, and L. J. Kramer, “Latency in visionic systems: Test methods and requirements,” presented at the RTO/HFM-125 Symp. Human Factors in the Design of Visionics Systems, 2004, Paper 8, pp. 8-1–8-14.
- [47] C. A. Estrada, H. S. Rosman, N. K. Prasad, G. Battilana, M. Alexander, A. C. Held, and M. J. Young, “Role of telemetry monitoring in the non-intensive care unit,” *The American Journal of Cardiology*, vol. 76, no. 12, pp. 960–965, 1995, doi: 10.1016/S0002-9149(99)80268-X.
- [48] R. Kandler, M. Lai, A. Ponnusamy, J. Bland, and C. Pang, “The safety of UK video telemetry units: Results of a national service evaluation,” *Seizure: European Journal of Epilepsy*, vol. 22, no. 10, pp. 872–876, 2013, doi: 10.1016/j.seizure.2013.08.001.
- [49] C. Zhang, L.-F. Wang, J.-Q. Huang, and Q.-A. Huang, “An LC-type passive wireless humidity sensor system with portable telemetry unit,” *J. Microelectromech. Syst.*, vol. 24, no. 3, pp. 575–581, 2015, doi: 10.1109/JMEMS.2014.2333747.
- [50] S. Udkar, R. Sharma, and U. Krishnan M, “Design and implementation of an embedded AI-driven telemetry and self-reconfiguration framework for high-power electrical platform,” *Procedia Computer Science*, vol. 283, pp. 1357–1366, 2026, presented at the Int. Conf. Machine Learning and Data Engineering, ISSN: 1877-0509.
- [51] A. Elshazly, A. Elliethy, and M. A. Elshafey, “Tactics overview for implementing high-performance computing on embedded platforms,” *IOP Conference Series: Materials Science and Engineering*, vol. 1172, Art. no. 012034, 2021, doi: 10.1088/1757-899X/1172/1/012034.
- [52] B. Eckman, D. Irvin, K. Lokshin, and A. Puri, “New approaches to the implementation of embedded telemetry processing systems,” in *Proc. Int. Telemetering Conf.*, International Foundation for Telemetering, 2015, HDL: 10150/624261.
- [53] M. H. Baig, J. A. Ayala Garcia, W. S. Qureshi, and I. Ullah, “Towards assessing cycleway pavement surface roughness using an action camera with IMU and GPS,” in *Proc. 11th Int. Conf. Vehicle Technology and Intelligent Transport Systems*, 2025, pp. 247–255, doi: 10.5220/0013504900003941.
- [54] T. Lei, A. A. Mohamed, and C. Claudel, “An IMU-based traffic and road condition monitoring system,” *HardwareX*, vol. 4, Art. no. e00045, 2018, doi: 10.1016/j.ohx.2018.e00045.
- [55] N. Certad, A. Aghanouri, J. Gorospe, and C. Olaverri-Monreal, “Multi-modal assessment of road roughness using smartphone applications, acceleration, and passenger ratings,” arXiv:2606.03427, 2026.

- [56] C. Choubassi, D. P. K. Seedah, N. Jiang, and C. M. Walton, “Economic analysis of cargo cycles for urban mail delivery,” *Transportation Research Record*, no. 2547, pp. 102–110, 2016, doi: 10.3141/2547-14.
- [57] J. C. Tapson, *Instrumentation*. Cape Town, South Africa: UCT Press, 2010.
- [58] Makerfabs, “ESP32S3 4G LTE CAT1 A7670X,” product page and hardware repository. [Online]. Available: <https://www.makerfabs.com/esp32s3-4g-lte.html> and <https://github.com/Makerfabs/ESP32-S3-4G-LTE-CAT1-A7670X> (accessed 24 Sep. 2026).
- [59] SIMCom Wireless Solutions, *A76XX Series AT Command Manual*, V1.09, Apr. 2023. [Online]. Available: https://files.waveshare.com/wiki/A7670E-Cat-1-GNSS-HAT/A76XX_Series_AT_Command_Manual_V1.09.pdf (accessed 24 Sep. 2026).
- [60] Raspberry Pi Ltd, *The Picamera2 Library*, Sep. 2026. [Online]. Available: <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf> (accessed 28 Sep. 2026).
- [61] N. Adams and S. Rasara, “Jinpeng Electric Tricycle Drivetrain: Characterization & Modelling,” TUKZIE vacation work report, University of Cape Town, Department of Electrical Engineering, Jul. 2026.
- [62] Z. Li, W. Yu, and X. Cui, “Online Classification of Road Roughness Conditions with Vehicle Unsprung Mass Acceleration by Sliding Time Window,” *Shock and Vibration*, vol. 2018, Art. no. 5131434, 2018, doi: 10.1155/2018/5131434.
- [63] G. Caroleo, A. Albin, and P. Maiolino, “On the Characterisation of the Time-of-Flight VL53L5CX Sensor by STMicroelectronics for Indoor Robotics Applications,” *Sensors*, vol. 26, no. 5, Art. no. 1639, 2026, doi: 10.3390/s26051639.
- [64] M. De Simone, Z. B. Rivera Chavez, and D. Guida, “Obstacle Avoidance System for Unmanned Ground Vehicles by Using Ultrasonic Sensors,” *Machines*, vol. 6, no. 2, Art. no. 18, 2018, doi: 10.3390/machines6020018.
- [65] N. Noomwongs, K. Kiataramgul, S. Chantranuwathana, and G. Phanomchoeng, “GNSS-RTK-Based Navigation with Real-Time Obstacle Avoidance for Low-Speed Micro Electric Vehicles,” *Machines*, vol. 13, no. 6, Art. no. 471, 2025, doi: 10.3390/machines13060471.
- [66] T. E. Choe, J. Wu, X. Lin, K. Kwon, and M. Park, “HazardNet: Road Debris Detection by Augmentation of Synthetic Models,” arXiv:2303.07547, 2023.
- [67] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” arXiv:2303.05499, 2023.
- [68] A. Bint-e Tanveer, M. A. Kamal, M. M. Alam, and M. Mohd Su’ud, “Real-time detection of rare roadside obstacles using YOLOv8-n in autonomous vehicles,” *PLOS ONE*, 2026, doi: 10.1371/journal.pone.0350732.
- [69] R. Chan, K. Lis, S. Uhlemeyer, H. Blum, S. Honari, R. Siegwart, P. Fua, M. Salzmann, and M. Rottmann, “SegmentMeIfYouCan: A Benchmark for Anomaly Segmentation,” in *Proc. Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2021.
- [70] B. Reimer, B. Mehler, J. Dobres, J. F. Coughlin, S. Matteson, D. Gould, N. Chahine, and V. Levantovsky, “Assessing the impact of typeface design in a text-rich automotive user interface,” *Ergonomics*, vol. 57, no. 11, pp. 1643–1658, 2014, doi: 10.1080/00140139.2014.940000.

- [71] F. You, Y.-F. Yang, M.-T. Fu, J. Zhang, and J.-M. Wang, “Design guidelines for the size and length of Chinese characters displayed in the intelligent vehicle’s central console interface,” *Information*, vol. 12, no. 5, p. 213, 2021, doi: 10.3390/info12050213.
- [72] H. Kim, S. Kwon, J. Heo, H. Lee, and M. K. Chung, “The effect of touch-key size on the usability of in-vehicle information systems and driving safety during simulated driving,” *Applied Ergonomics*, vol. 45, no. 3, pp. 379–388, 2014, doi: 10.1016/j.apergo.2013.05.006.
- [73] Euro NCAP, *Safe Driving: Driver Engagement Protocol*, version 1.2. Leuven, Belgium: Euro NCAP, 2026.
- [74] D. L. Strayer, J. M. Cooper, R. M. Goethe, M. M. McCarty, D. J. Getty, and F. Biondi, *Visual and Cognitive Demands of Using In-Vehicle Infotainment Systems*. Washington, DC, USA: AAA Foundation for Traffic Safety, 2017.

Appendix A

Graduate Attributes, AI Use and Project Repository

A.1 Graduate attribute evidence

This section summarises how the work reported here meets the six graduate attributes assessed in this course, with the sections that hold the evidence.

GA1: Problem solving

The research questions were framed around measurable vehicle dynamic response rather than subjective ride comfort, and each was tied to a requirement and a verification method (Section 1.2.2). The design choices were reasoned from first principles: a 200 Hz sampling rate from the ISO 2631-1 band and the sampling theorem, drift-free task timing and non-blocking bounded queues so that acquisition is never held up (Section 3.11), the acceleration magnitude as an orientation-tolerant signal (Section 3.15), and speed normalisation after the literature showed that raw RMS mixes road roughness with speed (Section 3.15.3). Faults were root-caused from evidence rather than worked around, including a sync-pin choice checked against the board schematic (Section 3.12), a serial-routing fault (Section 3.19) and the flash-write acquisition stall (Section 3.21.1).

GA4: Investigations, experiments and data analysis

Assumptions were treated as hypotheses and tested on the real hardware: the GNSS reply format (assumed 13 fields, measured 9, Section 3.13), the BMS service layout (Section 3.16), and the cause of the periodic acquisition stall, confirmed by a controlled comparison with local logging on and off (Section 3.21.1). The feature computation was validated against a synthetic signal with known answers, the two IMUs were calibrated against gravity and checked under disturbance (Section 3.15.2), the dual-IMU fusion was verified in a recorded session (Section 3.15.4), and the camera-to-IMU timing offset was bounded (Section 3.12). The raw evidence is indexed in Appendix C.

GA5: Use of engineering tools

PlatformIO, the Arduino framework and FreeRTOS were used for the firmware, with esptool and serial diagnostics for hardware verification (Section 3.10). Python tools were written to support testing: a scripted bench test, a browser bench console that parses and replays the telemetry, and schematic checks. On the camera side, LiteRT and Picamera2 run the detector, and per-stage latency is measured from the sensor's own frame timestamps (Section 3.20.7). A parametric bracket was modelled in OpenSCAD and a camera casing was designed and 3D printed (Section 3.20.6). The limits of each tool were checked rather than assumed, for example a BMS checksum formula that no vendor document confirms (Section 3.16).

GA6: Professional and technical communication

This report documents the engineering work, including the faults found and how they were resolved. The written work was developed with the supervisor through regular meetings and a draft Introduction and Methodology, and inconsistencies between chapters were corrected as the work changed. The raw evidence behind each result is kept and indexed (Appendix C), and the firmware, tools and planning documents are in the project repository.

GA8: Individual work

The hardware bring-up, firmware, bench tools and report were done individually, with the supervisor consulted on specific questions. The components were selected, compared against the requirements and ordered by the author, for example the 8×8-zone time-of-flight sensor chosen over a single-point one to separate left, centre and right (Section 3.20.7); each component was tested on the bench before being relied on; and the camera casing was designed and 3D printed by the author (Section 3.20.6). Inherited work is identified where it is used: the vehicle dashboard and its team, and a teammate's BMS parser used only as an independent cross-check (Section 3.16). The use of AI tools is declared in Section A.3.

GA9: Independent learning ability

The work required learning FreeRTOS timing, BLE GATT, the A76XX AT command set and on-device inference on the Raspberry Pi, largely from primary sources: manufacturer schematics and manuals, and live tests on the hardware. Assumptions were repeatedly corrected against those sources, for example GPIO assignments against the schematic and the MQTT result handling against the modem manual (Section 3.21). A continuously running literature search was kept up so that the research-gap claims were tested rather than asserted once, and it narrowed two of them (Chapter 2).

A.2 Project repository

Firmware, camera software, planning documents and supporting tools for this project are available at <https://github.com/0geder/Tukzie-HPEC-HUD>.

A.3 Declaration of Artificial Intelligence Use

Artificial intelligence tools were used to assist with clarifying the organisation of this report, improving the alignment and consistency of its presentation, refining wording, and checking LaTeX formatting. All AI-assisted suggestions were reviewed by the author. The author remains responsible for the report's technical content, analysis, interpretations, and final editorial decisions.

Appendix B

Supporting Figures

The figures below support the text of Chapters 1 and 3, where each is referenced.



Figure B.1: Manufacturer identification plate on the TUKZIE Rev 0 platform, photographed directly during vehicle inspection.

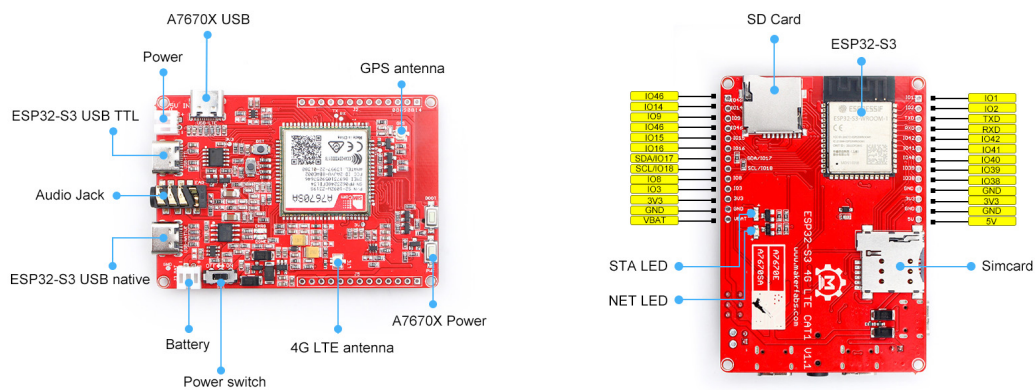


Figure B.2: Makerfabs ESP32-S3 4G LTE CAT1 A7670X board: manufacturer-labelled ports (left) and pinout (right) [58].



Figure B.3: The live telemetry dashboard connected to the telemetry unit over USB during bench testing, showing both IMUs fused at 200 Hz with no dropped samples and two hand disturbances in the vibration plot.

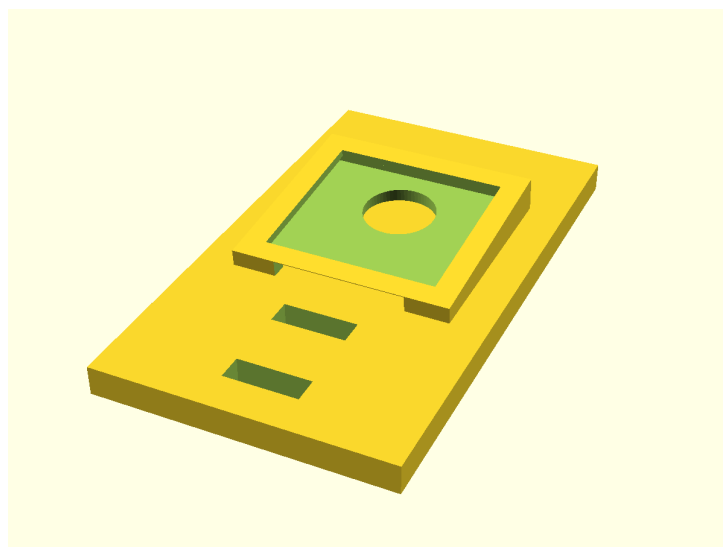


Figure B.4: Initial parametric camera bracket (OpenSCAD, full render), showing the corrected lens-retention lip and cable-routing slots. This design was not printed.

Appendix C

Test Evidence

This appendix indexes the raw evidence behind the bench results reported in Chapter 3. Every item is the unedited output of the test it names and is kept in the project repository, under `Report/evidence/` or `BenchTest/logs/`; Table C.1 gives each file. Figure B.3 in Appendix B and Figure 3.3 in Chapter 3 belong to the same record.

Table C.1: Index of test evidence.

Date	Test	Result shown	File
25 Sep 2026	Recorded bench session, telemetry unit, 150 s from power-up	200 Hz on both IMUs with no dropped samples; 149 fused windows, all from both sensors; five of five fusion self-tests passed; acquisition gaps of 25 to 30 ms every 12 s	<code>BenchTest/logs/2026-09-25_bench_-fusion_shake.log</code>
25 Sep 2026	The same log replayed through the bench console (Figure C.1)	Every subsystem’s state at the end of the session, and the vibration and disagreement traces for the whole recording	<code>Report/evidence/2026-09-25_bench_-console_replay.png</code>
28 Sep 2026	Hazard detector, first run on live camera frames	430 frames in 60.1 s (7.2 fps); a person in view logged as “motorcycle”, which exposed the label-offset fault	<code>Report/evidence/2026-09-28_-camera_run1_-label_bug.txt</code>
28 Sep 2026	Hazard detector after the label and colour fixes	419 frames in 60.1 s (7.0 fps); person detected at confidence 0.50 to 0.73; 16 alert activations in 47 s for one person in view, which led to clear-side hysteresis	<code>Report/evidence/2026-09-28_-camera_run2_-after_fixes.txt</code>
29 Sep 2026	Firmware v0.5.0 on the telemetry unit: commands, flash-write stall test, MQTT reconnect	With logging on, 33 to 40 ms gaps on both IMUs every 10 s; with logging off, worst gap 5.9 ms; no dropped samples; 47 MQTT publishes with no failures; reconnect in 0.4 s	<code>BenchTest/logs/2026-09-29_v050_-bench_test.log</code>

Date	Test	Result shown	File
29 Sep 2026	GNSS outdoor fix on a rooftop, firmware v0.5.0, raw replies requested every 15 s	3D fix after 51 s; 17-field reply with decimal degrees; old parser read satellite counts as the position; well-formed replies 2.4 and 5.5 m from the phone's position; replies garbled by the competing query	BenchTest/logs/2026-09-29_gnss_-rooftop.log
30 Sep 2026	GNSS outdoor fix with the corrected parser (v0.5.2), raw replies printed by the firmware	3D fix after 105 s from a cold start; 23 to 25 satellites; over 104 fixes, median 2.7 m and 95% within 11.0 m of the phone's position	BenchTest/logs/2026-09-30_gnss_-rooftop_v052.log
30 Sep 2026	MQTT recovery from a 60 s radio-off period (AT+CFUN=4, then AT+CFUN=1), v0.5.0 and v0.5.3	Loss detected in 0.8 to 20.7 s; retries at 30 and 60 s; reconnect 36 s after the radio returned in v0.5.0, about 5 s in v0.5.3; no failed publishes afterwards	BenchTest/logs/2026-09-30_-signal_loss_-test.log, .._v053.log
30 Sep 2026	Sync-pulse edges logged on the Pi (GPIO17, kernel timestamps), two 60 s runs	Run 1: 16 s burst of spurious edges, consistent with a disturbed contact; run 2: 121 edges, median interval 500.06 ms, range 499.72 to 500.41 ms	BenchTest/logs/2026-09-30_sync_-edges_run1_-noisy.csv, .._run2.csv
30 Sep 2026	Camera detector latency on the Pi, three 60 s runs (4 threads, 4 threads with live view, 1 thread)	Median 102, 105 and 186 ms from sensor readout to result; 19.5, 17.3 and 7.0 frames per second; no throttling	BenchTest/logs/2026-09-30_c4_-latency/
1 Oct 2026	Firmware v0.6.0 bench run, 330 s, logging on throughout (buffered log, signal quality, Hall counter in loopback)	Five flushes; all ten gaps over 10 ms (37 to 40 ms) within 3.5 s after a flush; worst gap otherwise 5.9 ms; no dropped samples; Hall counter 2.00 edges/s; CSQ 24 to 26	BenchTest/logs/2026-10-01_v060_-bench_test.log
1 Oct 2026	Read-back of the stored telemetry log (firmware v0.6.3, BenchTest/log_-dump.py, four passes)	6,020 records recovered complete; 5,545 contained nan (invalid JSON, fixed in v0.6.3); 14 records with a valid BMS reading (74.78 V, 60%) from 30 Sep; log then cleared	BenchTest/logs/2026-10-01_-telemetry_log_-dump.jsonl, ..clean.jsonl
1 Oct 2026	Stand-in ToF: micromouse sensor board on the Pi (three VL53L0X, tof_probe.py, tof_reader.py)	Chip confirmed VL53L0X; three sensors addressed and read at about 31 per second each; centre 267 to 300 mm, right 24 to 38 mm, left 8190 (no target) throughout	BenchTest/logs/2026-10-01_tof_-three_sensors.csv

Date	Test	Result shown	File
1 Oct 2026	Dashboard camera page (laptop, offscreen) against the live detector on the Pi 4, 60 s	Live after 0.6 s; 638 frames decoded in 45 s (14.2 per second); offline state shown when the detector stopped; no alerts (nothing in view)	Report/evidence/2026-10-01_-dashboard_page_-live.png, DashboardIntegration/tests/test_front_-camera_page_-live.py
1 Oct 2026	Dashboard camera page against the live detector with a person in view, 40 s	Red “Person, Immediate” alert listed 0.5 s after start, matching the box on the video; 15.4 frames per second, 109 to 118 ms; clearing not seen (person stayed in view)	Report/evidence/2026-10-01_-dashboard_page_-person.png
1 Oct 2026	Camera colour check (C3): stills with the standard and the NoIR tuning, detector frames with NoIR tuning and saturation 1.8	Standard tuning: white wall pink; NoIR tuning: neutral but washed out (blue card 120, 124, 125); with saturation 1.8: blue 1, 86, 216; red 222, 8, 77; green 6, 124, 103; grey 124, 129, 138; white 186, 199, 197 (slightly cool)	Report/evidence/c3/
2 Oct 2026	First pothole model: YOLO11n fine-tuned in Colab (T4) on a public 608-image pothole dataset, test split at 640 and 320	mAP@50 59.7% (P 0.66, R 0.50) at 640; 55.0% at 320; 46 epochs, 15 min	CameraDetection/training/runs/run1_-gerapothole_640/
2 Oct 2026	First pothole model on the South African test split (605 images, 1,238 potholes) at 640, 1280 and 1920 input; labels checked visually	mAP@50 0.000 at all three sizes, recall at most 1.2%; label positions confirmed correct	Colab session notebook (to be saved)
2 Oct 2026	Firmware v0.7.0 DASH line on the bench (40 s) and through the Raspberry Pi 4 bridge to a laptop	40 lines in 40 s, none missing, all valid JSON, max 272 bytes; IMUs 200 Hz, 0 drops; bridge served live record (0.37 s old) and ToF distances	BenchTest/TEST_-PROCEDURES.md T11, T12
2 Oct 2026	YOLO11n (COCO) via NCNN on the Raspberry Pi 4, 320 input, 4 threads, 15 s trial	7.9 fps; inference 111 ms median; 175 ms sensor to result (p95 258 ms)	CameraDetection/ncnn_bench.py
2 Oct 2026	Dashboard map crash on the Raspberry Pi 5, diagnosed over ssh without changing the team’s installation	Segmentation fault about 2 s after the dashboard is shown with the web map (online or offline data); no crash with the native map after 30 s; the web engine alone loads normally	Dashboard Integration/PI5_MAP_CRASH.md

Date	Test	Result shown	File
3 Oct 2026	Dashboard live-only mode and automatic Raspberry Pi 4 finding, offscreen against a simulated bridge (laptop)	With the bridge down every page shows -- or Unavailable and no simulated value; live when the bridge appears, back to no data when it stops; host found by localhost, override and beacon, and followed after it moved	Dashboard Integration/ tests/test_live_- data_provider.py
4 Oct 2026	Street-tile map, offscreen, fed the generated UCT replay track (not recorded data) against a stand-in tile server	Tiles requested at most two at a time and cached; heading-up follow; "Waiting for GPS fix"; REPLAY badge; vector roads with the tile server down	Dashboard Integration/ tests/tile_map_- *.png
4 Oct 2026	Camera and time-of-flight fusion, offline tests	29 of 29 pass; not yet run on the Raspberry Pi 4 (test C9 planned)	CameraDetection/ tests/test_- sensor_fusion.py
6 Oct 2026	Sensors page and hazard banner, whole dashboard offscreen against the live bridge	ToF gauges (ahead 0.43 m red, right 0.69 m amber, left no reading), both listed as immediate; banner over the driving page, hidden on the Sensors page	DashboardIntegration/ tests/sensors_- page.png, hazard_banner.png
6 Oct 2026	First fused camera and time-of-flight hazard, live, before calibration	Person, ahead sensor: camera 0.874 m, time-of-flight 0.096 m, reported 0.096 m; not checked against a tape	BenchTest/ TEST_- PROCEDURES.md, test C9 notes
6 Oct 2026	Pothole model run 2, South African road-band tiles (Colab)	Validation mAP@50 0.714 at 640 and 0.566 at 320; held-out test tiles not yet scored	CameraDetection/ training/runs/ run2_sa_band_- tiles_640/ training_- record.json
6 Oct 2026	Simulated drive on the street map, offscreen tests	Starts without a fix, 25 km/h along the route, heading-up, arrives, a real fix stops it	DashboardIntegration/ tests/tile_map_- simulated_- drive.png
6 Oct 2026	Raspberry Pi 4 power and temperature watcher, 20 min	Clear with nothing attached; soft temperature limit from 17:45; under-voltage 14 s after the telemetry unit was plugged into its USB port	BenchTest/logs/ 2026-10-06_pi4_- power_watch.log

The two detector runs end with the following lines, copied from the Raspberry Pi terminal. Before the fixes, every alert carried the wrong class:

```
Stopped. 430 frames in 60.1s (7.2 fps average).
{"class": "motorcycle", "distance_band": "immediate", "confidence": 0.66,
 "timestamp": "2026-09-28T11:58:32", "alert_status": "active"}
```

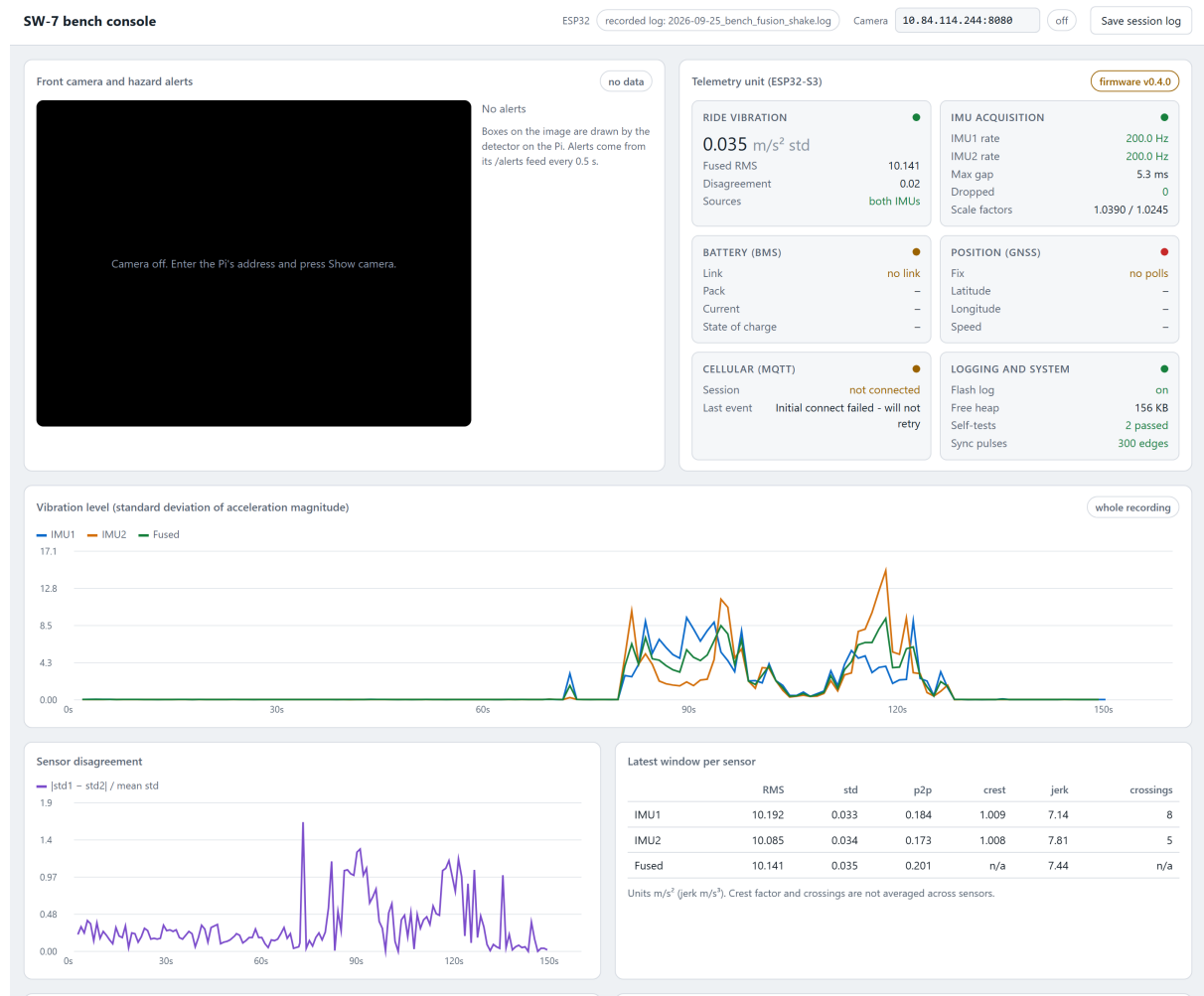


Figure C.1: The recorded bench session of 25 September 2026 replayed through the bench console, which parses the telemetry unit's serial output. Only recorded data is shown; the camera panel is off because the camera was not part of this session.

After the fixes, the class is correct, and the log shows the alert clearing and re-activating within a few seconds while the same person stayed in view:

```
[14:31:28] person band=immediate conf=0.59 streak=3/3
-> ALERT logged: person (immediate)
Stopped. 419 frames in 60.1s (7.0 fps average).
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:28", "alert_status": "active"}
{"class": "person", "distance_band": null, "confidence": 0.0,
 "timestamp": "2026-09-28T14:31:30", "alert_status": "cleared"}
{"class": "person", "distance_band": "immediate", "confidence": 0.59,
 "timestamp": "2026-09-28T14:31:31", "alert_status": "active"}
```